



科学研究的人工智能方法学

# 物理信息融合的科学计算

复旦大学人工智能创新与产业研究院

林晨森

Email: [linchensen@fudan.edu.cn](mailto:linchensen@fudan.edu.cn)

2024.11.15



□ **Overview of ODEs/PDEs**

□ **From DNN to physics-informed neural networks (PINNs)**



# Overview of ODE / PDE

- **Partial differential equation (PDE)** is an equation which imposes relations between the various partial derivatives of a multivariable function.
- Ordinary differential equations form a subclass of partial differential equations, corresponding to functions of a single variable.

$$m \frac{d^2 x(t)}{dt^2} = F(x(t))$$

ODE

经典的牛顿第二定律表达式

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = 0$$

PDE

拉普拉斯算子

静电学中，电势满足拉普拉斯方程  
温度场中，温度满足拉普拉斯方程



# Overview of PDE

➤ A differential equation without nonlinear terms of the unknown function  $y$  and its derivatives is known as a linear differential equation

□  $y$  cannot have higher index terms such as  $y^2$ ,  $y^3$ , and derivative multiples

$$\left(\frac{dy}{dx}\right)^2, \frac{dy}{dx} \left(\frac{d^2y}{dx^2}\right), \dots$$

□  $y$  cannot contain non-linear terms such as

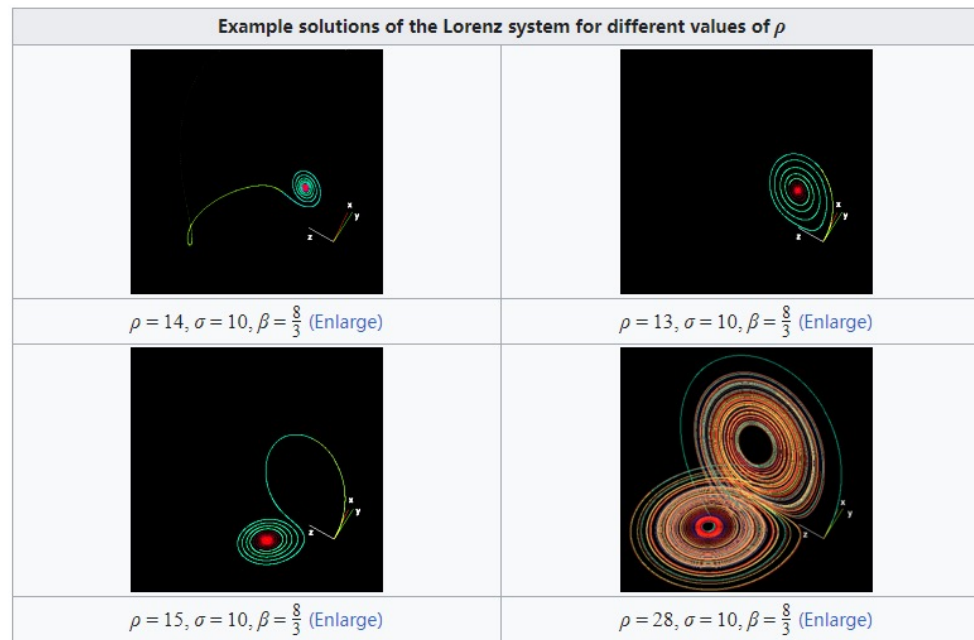
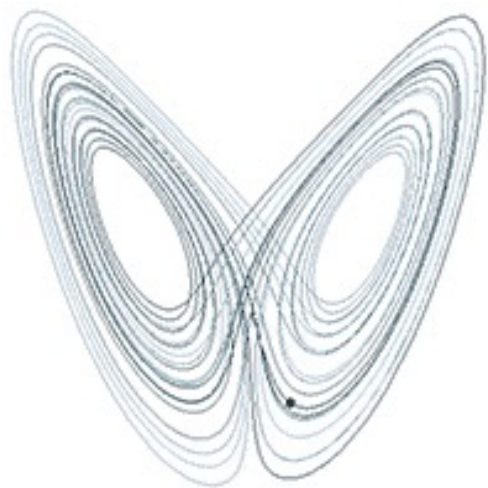
$$(y)^2, \sin(y), \ln(y) \dots$$

1.  $y \frac{dy}{dx} + y^2 = 0$
2.  $\frac{d\sin(y)}{dx} + xy \frac{dy}{dx} = \sin(x) \left(\frac{d^2y}{dx^2}\right)^3$
3.  $\frac{\ln(x)}{xy} + \frac{1}{y^2} \frac{dy}{dx} = \cos(y)$



# PDE: Dynamic system

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(\rho - z) - y, \\ \frac{dz}{dt} &= xy - \beta z.\end{aligned}$$



- 非线性动力系统
- 奇异性：既不发散也不收敛，围绕着吸引区域不断循环，形成蝴蝶效应
- 简单方程也能产生混沌效应

A sample solution in the **Lorenz attractor**  
when  $\rho = 28$ ,  $\sigma = 10$ , and  $\beta = 8/3$



# PDE: Dynamic system

$$\frac{\partial A}{\partial t} = D_a \nabla^2 A - AB^2 + f(1 - A)$$

$$\frac{\partial B}{\partial t} = D_b \nabla^2 B + AB^2 - (f + k)B$$

这里， $D_a$  和  $D_b$  分别是A和B的扩散系数， $\nabla^2$  是拉普拉斯算子，它描述了物质如何在空间中扩散。 $f$  是A的供给速率， $k$  是B转化为P的速率。

**Gray-Scott模型**不仅在理论化学中有应用，在生物学、生态学、物理学等多个领域也有广泛的应用。例如，它可以用来模拟细胞分裂、肿瘤生长、甚至某些社会行为模式的形成。

<https://www.lanevol.org/resources/gray-scott>

# PDE: Navier-Stokes Equations

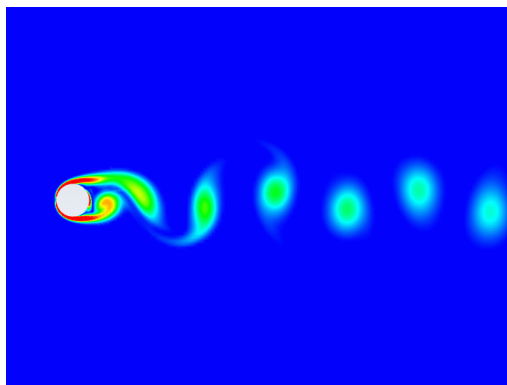
连续性方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

动量方程

$$\begin{aligned} \rho \left( \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} \right) &= \rho f_x - \frac{\partial p}{\partial x} + \mu \left( \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \\ \rho \left( \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} \right) &= \rho f_y - \frac{\partial p}{\partial y} + \mu \left( \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \\ \rho \left( \frac{\partial w}{\partial t} + u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} \right) &= \rho f_z - \frac{\partial p}{\partial z} + \mu \left( \frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \end{aligned}$$

- 数学上：解的存在性和光滑性（千禧年大奖难题）
- 实际应用上：已经取得了巨大成功（航空航天）（2003年波音787“梦想客机” Boeing 787 Dreamliner，第一架完全依靠CFD替代风洞实验的飞机）（理论18世纪，CFD推动进步等到2003年）



“When I meet  
God, I am going  
to ask him two  
questions:  
Why relativity?  
And why  
turbulence?  
I really believe he  
will have an  
answer for the  
first.”

- werner heisenberg



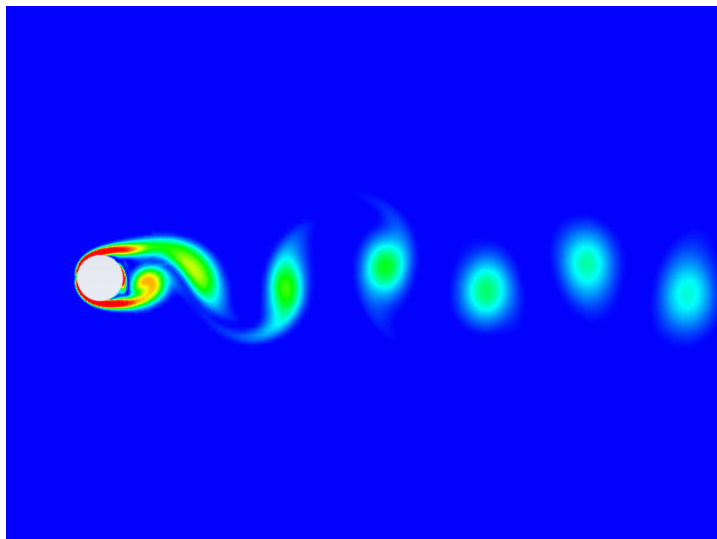


# PDE: Navier-Stokes Equations

## 纯流动

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

$$\begin{aligned} \rho \left( \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} \right) &= \rho f_x - \frac{\partial p}{\partial x} + \mu \left( \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \\ \rho \left( \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} \right) &= \rho f_y - \frac{\partial p}{\partial y} + \mu \left( \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \\ \rho \left( \frac{\partial w}{\partial t} + u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} \right) &= \rho f_z - \frac{\partial p}{\partial z} + \mu \left( \frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \end{aligned}$$



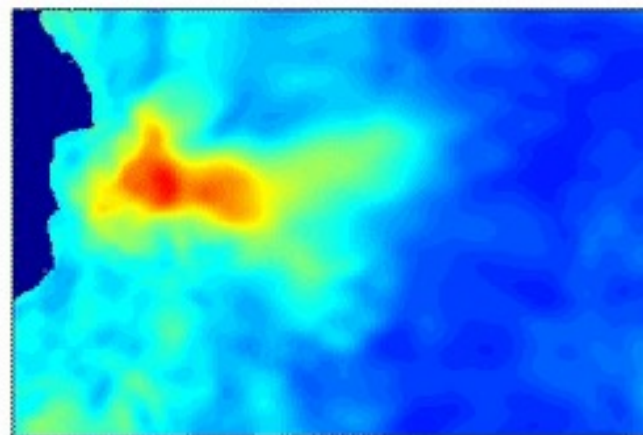
## 纯流流动与温度耦合

$$\partial_t T + \mathbf{u} \cdot \nabla T = \alpha \nabla^2 T$$

能量方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

$$\begin{aligned} \rho \left( \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} \right) &= \rho f_x - \frac{\partial p}{\partial x} + \mu \left( \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \\ \rho \left( \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} \right) &= \rho f_y - \frac{\partial p}{\partial y} + \mu \left( \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \\ \rho \left( \frac{\partial w}{\partial t} + u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} \right) &= \rho f_z - \frac{\partial p}{\partial z} + \mu \left( \frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \end{aligned}$$



# Forward and inverse problem

- 确定性
- 可解性
- 计算效率

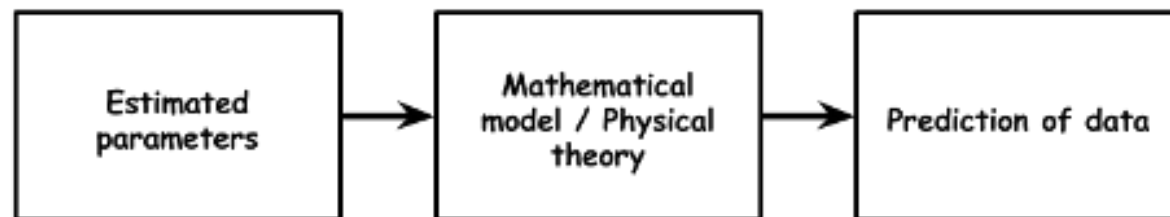
## Inverse Problem

One of these is known

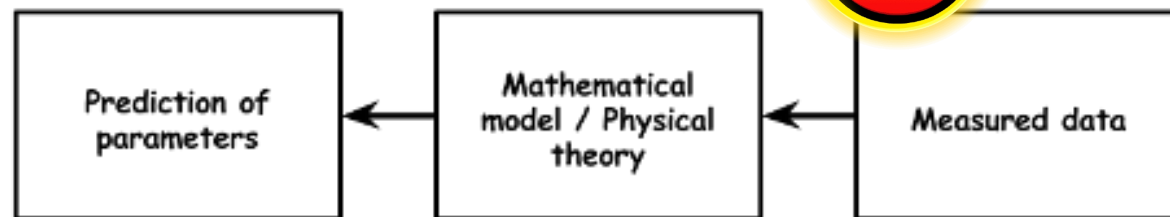
Known  
but with  
errors



## The forward problem



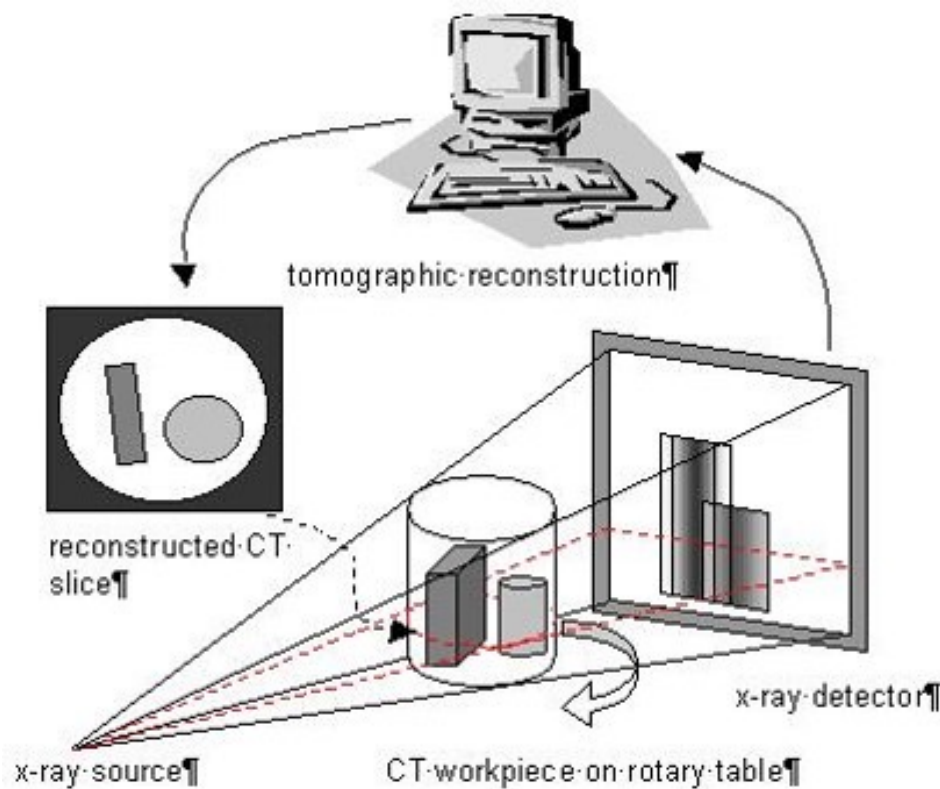
## The inverse problem



- 计算不确定性
- 复杂性，需要反复迭代和优化

# Forward and inverse problem

## 医学成像

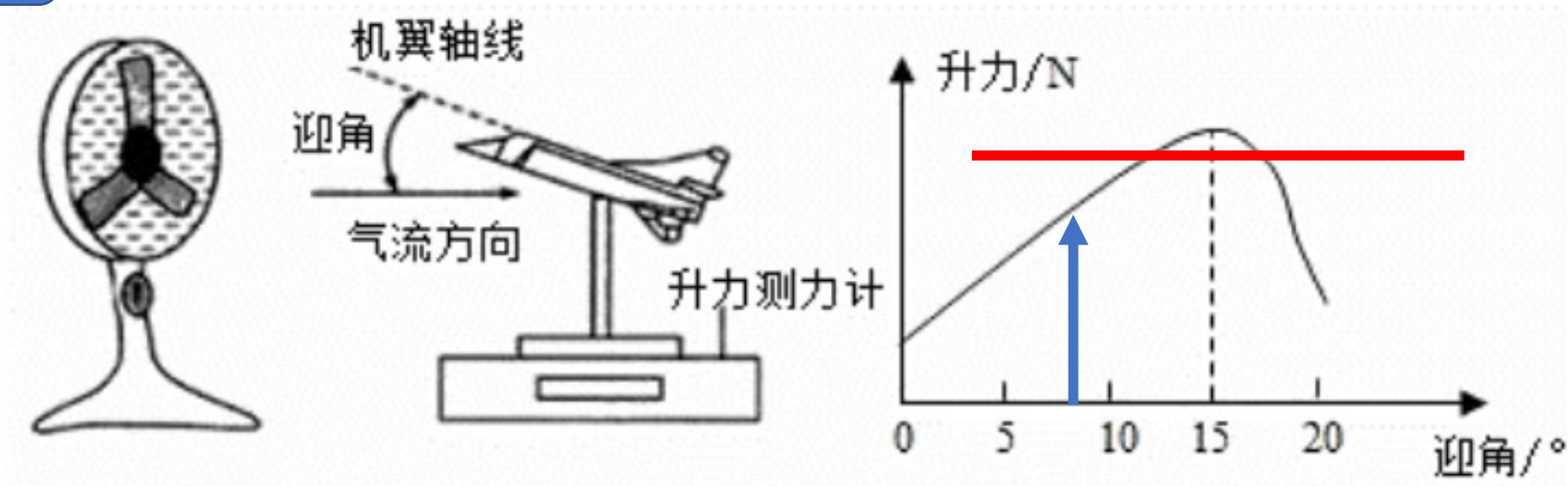


- 正问题：从已知物理模型和输入条件（射线强度）预测图像
- 反问题：从已知图像重建物体的内部结构



# Forward and inverse problem

## 流体分析与设计



正问题:  $x \rightarrow y$  已知迎角, 求解升力  
 反问题:  $y \rightarrow x$  已知升力, 求迎角

□ **Overview on ODEs/PDEs**

□ **From DNN to physics-informed neural networks (PINNs)**



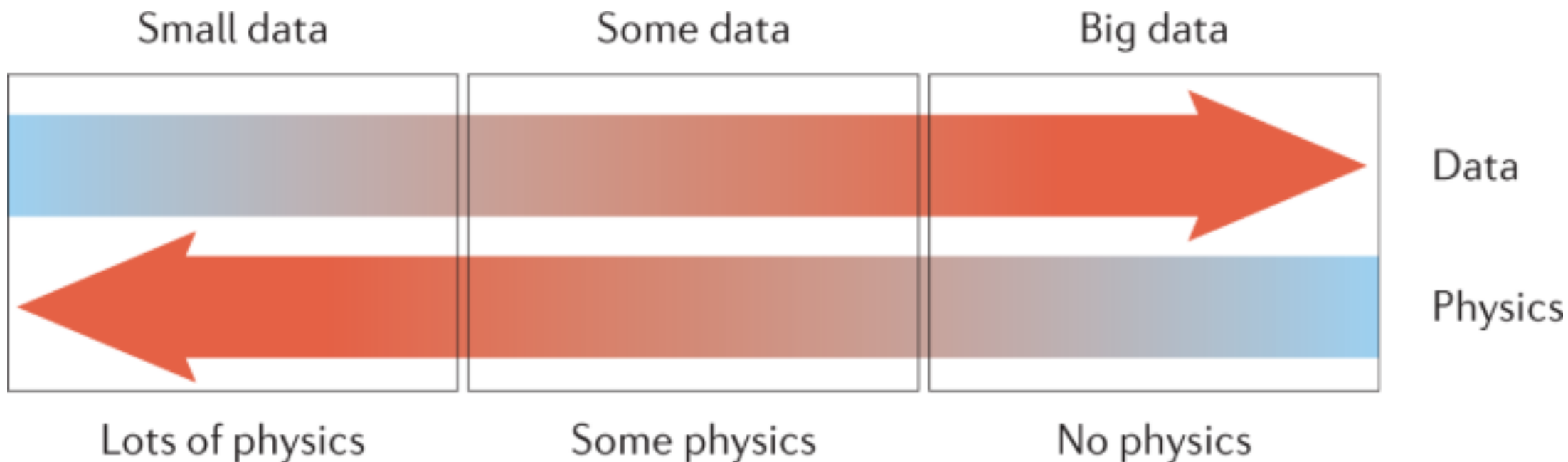


# From DNNs to PINNs

Data + Physical Laws

*The 5D Law: Dinky, Dirty, Dynamic, Deceptive Data*

## Three scenarios of Physics-Informed Learning Machines





# Conventional numerical solvers for ODEs/PDEs

## Methods

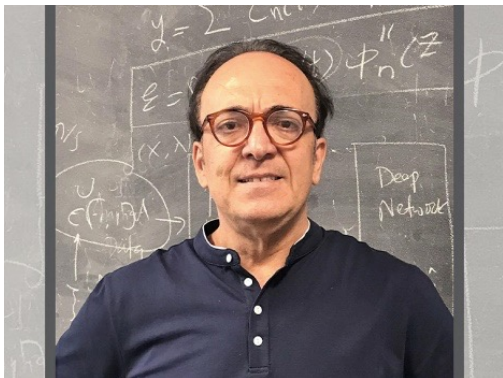
- Finite difference method (FDM)
- Finite element method (FEM)
- Finite volume method (FVM)
- Spectral method (SM)
- $h$ - $p$  method
- .....

## Limitations

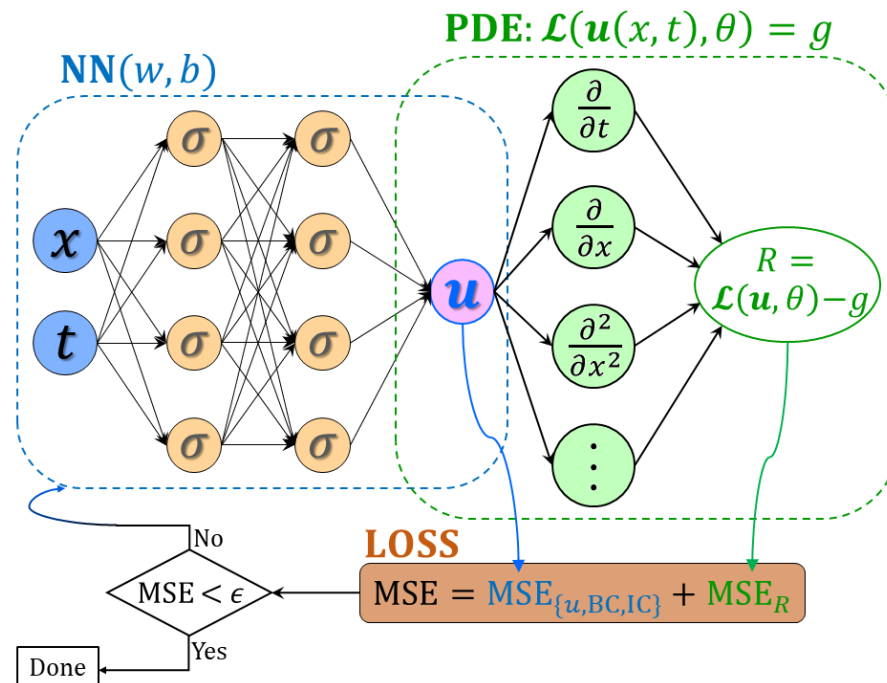
- ✓ Mesh generation
- ✓ High-dimensional problems
- ✓ Solving inverse problems is often prohibitively expensive
- ✓ Cannot seamlessly incorporate noisy data
- ✓ ...

# From DNNs to PINNs

Physics-informed neural networks (PINNs) 2019



G. Karniadakis  
(Brown U)



$$(1) \quad u_1 = \sigma(W_1^T X + b_1)$$

$$u_2 = \sigma(W_2^T u_1 + b_2)$$

...

$$u = W_l^T u_{l-1} + b_l$$

(2) PDE part: *Chain rule*

$$\partial_t u, \partial_x u, \dots$$

**Loss function:**

Forward problem:  $L = \min_{\theta} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$

Inverse problem:  $L = \min_{\theta, \lambda} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2 + |u^* - u|^2)$

# From DNNs to PINNs

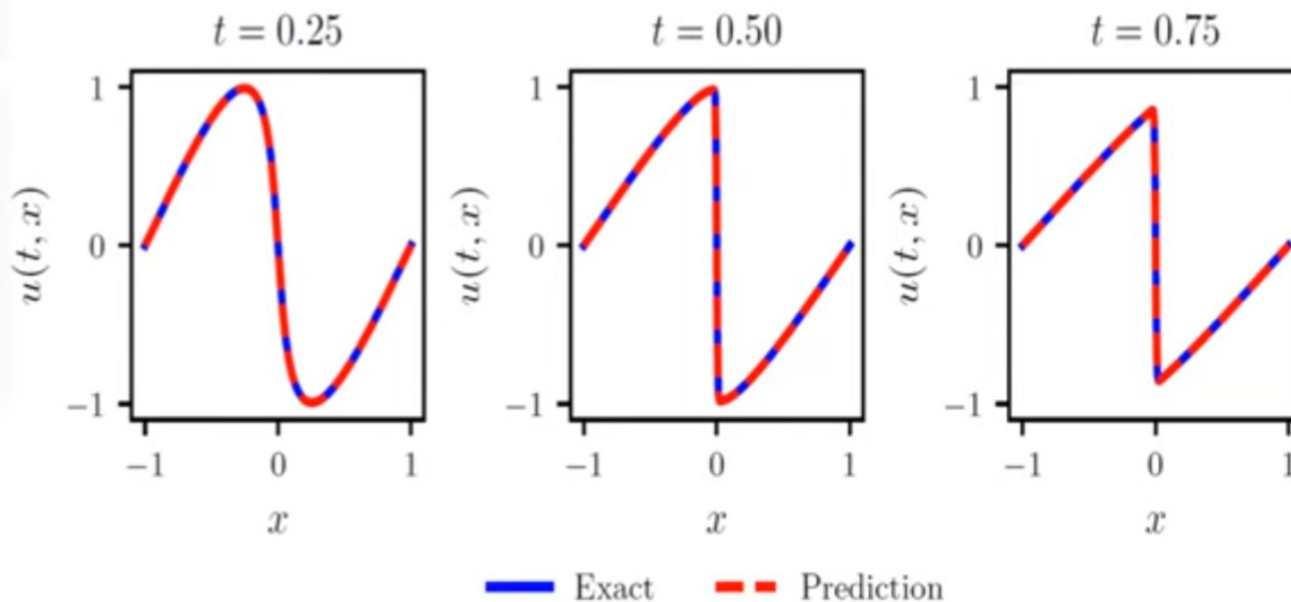
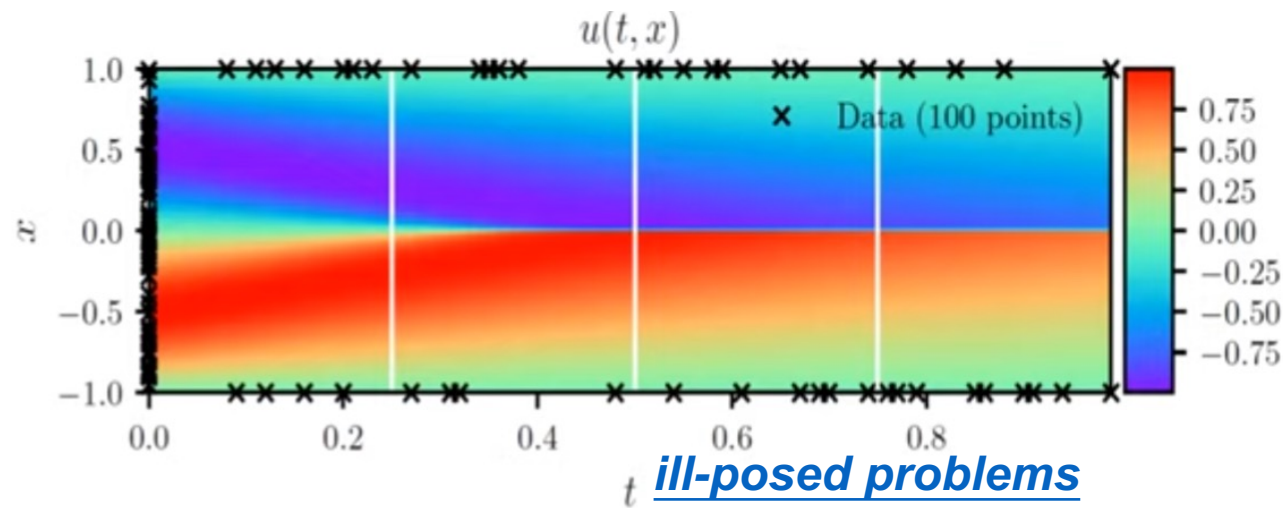
## Physics Informed Neural Networks

$$u_t + uu_x - (0.01/\pi)u_{xx} = 0$$

```
def u(t, x):
    u = neural_net(tf.concat([t,x],1), weights, biases)
    return u
```

```
def f(t, x):
    u = u(t, x)
    u_t = tf.gradients(u, t)[0]
    u_x = tf.gradients(u, x)[0]
    u_xx = tf.gradients(u_x, x)[0]
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx
    return f
```

$$\sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2 + \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2$$





# Forward problems: Poisson equation

泊松方程 (Poisson's equation) 是数学和物理学中一个重要的偏微分方程，广泛应用于电动力学、流体力学、热传导、弹性力学等领域。泊松方程的形式如下：

$$\nabla^2 u = f$$

其中：

- $\nabla^2$  是拉普拉斯算子 (Laplacian operator)，在直角坐标系中表示为：  $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}$
- $u$  是未知函数，表示待求解的物理量（如电势、温度、位移等）。
- $f$  是已知的源项或右端项，表示外部源或分布。

## 泊松方程的应用

### 1. 电动力学：

- 在静电学中，泊松方程描述了电势  $\phi$  与电荷密度  $\rho$  之间的关系：  $\nabla^2 \phi = -\frac{\rho}{\epsilon_0}$
- 其中  $\epsilon_0$  是真空介电常数。

### 2. 流体力学：

- 在不可压缩流体的纳维-斯托克斯方程中，压力  $p$  满足泊松方程：  $\nabla^2 p = -\nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u})$
- 其中  $\mathbf{u}$  是流速场。

### 3. 热传导：

- 在稳态热传导问题中，温度  $T$  满足泊松方程：  $\nabla^2 T = -\frac{Q}{k}$
- 其中  $Q$  是热源密度， $k$  是热导率。

### 4. 弹性力学：

- 在弹性力学中，位移  $u$  满足泊松方程：  $\nabla^2 u = -\frac{F}{\lambda + 2\mu}$
- 其中  $F$  是外力密度， $\lambda$  和  $\mu$  是材料的拉梅常数。



# Forward problems: Poisson equation

## 泊松方程的解法

泊松方程的求解方法可以根据问题的复杂性和边界条件选择不同的数值和解析方法。

### 1. 解析解法：

- 对于简单的几何形状和边界条件，可以尝试寻找解析解。常用的方法包括分离变量法、傅里叶级数展开等。

### 2. 数值解法：

- **有限差分法** (Finite Difference Method, FDM)：将偏微分方程离散化为代数方程组，通过迭代方法求解。
- **有限元法** (Finite Element Method, FEM)：将求解区域划分为小单元，通过近似函数在每个单元内求解。
- **谱方法** (Spectral Method)：利用基函数（如三角函数、多项式等）的线性组合来近似解。
- **边界元法** (Boundary Element Method, BEM)：将问题转化为边界上的积分方程，适用于边界条件复杂的问题。

## 边界条件

泊松方程的求解还需要指定适当的边界条件，常见的边界条件包括：

1. **Dirichlet 边界条件**：指定边界上的函数值  $u$ 。
2. **Neumann 边界条件**：指定边界上的法向导数  $\frac{\partial u}{\partial n}$ 。
3. **Robin 边界条件**：指定边界上的线性组合  $au + b\frac{\partial u}{\partial n} = g$ 。



# Forward problems: Poisson equation

- Consider the 1D Poisson equation with homogeneous boundary conditions

$$\begin{cases} -u''(x) = f(x), x \in (-1,1), \\ u(-1) = u(1) = 0. \end{cases}$$

- Let  $f(x) = \pi^2 \sin(\pi x)$ , then

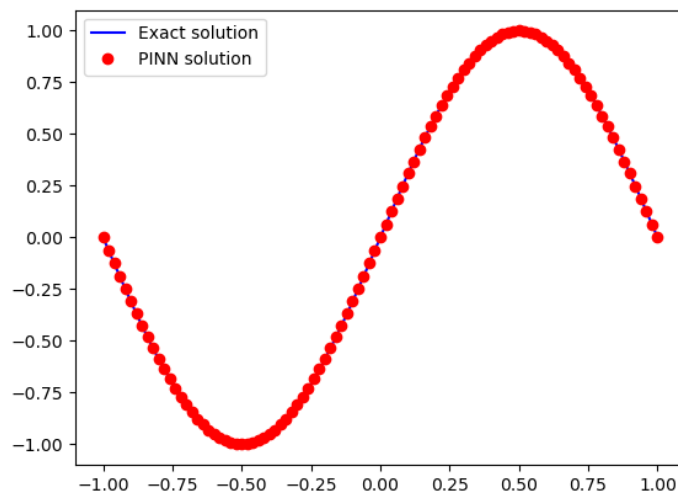
$$u(x) = \sin(\pi x).$$

# Forward problems: Poisson equation

- Loss function

$$Loss = \frac{\omega_1}{N_{res}} \sum_{k=1}^{N_{res}} (u''_{NN}(x_k) + f(x_k))^2 + \omega_2 (u_{NN}(-1)^2 + u_{NN}(1)^2)$$

$$\omega_1 = \omega_2 = 1$$



Test error  $\sim 10^{-4}$

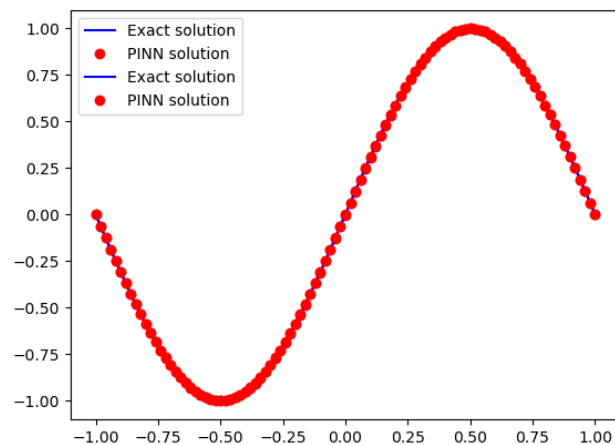


# Forward problems: Poisson equation

- Let  $\bar{u}_{NN}(x) = (x^2 - 1) * u_{NN}(x)$ , then

$$\bar{u}_{NN}(-1) = \bar{u}_{NN}(1) = 0.$$

- Loss function 
$$Loss = \frac{1}{N_{res}} \sum_{k=1}^{N_{res}} (\bar{u}_{NN}''(x_k) + f(x_k))^2$$



Test error  $\sim 10^{-5}$

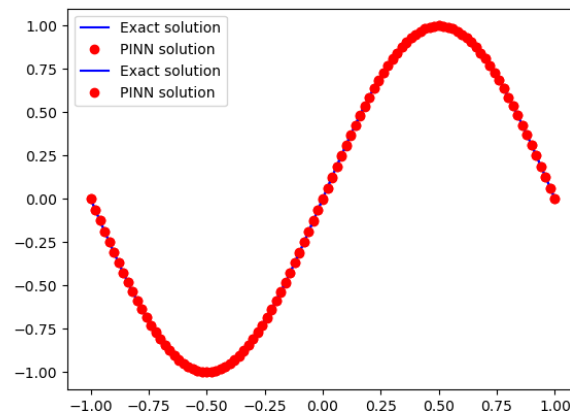
# Forward problems: Poisson equation

- Let  $\bar{u}_{NN}(x) = u_{NN}(x) - (kx + b)$ , where

$$\bar{u}_{NN}(-1) = \bar{u}_{NN}(1) = 0.$$

- Then we have

- Loss function  $Loss = \frac{1}{N_{res}} \sum_{k=1}^{N_{res}} (\bar{u}_{NN}''(x_k) + f(x_k))^2$



Test error  $\sim 10^{-5}$



# Forward problems: Burgers equation

**Burgers 方程 (Burgers' Equation)** 是一个非线性偏微分方程, 最初由荷兰数学家和物理学家约翰·马丁努斯·伯格斯 (J.M. Burgers) 在1940年代提出。这个方程在流体力学、交通流、非线性波传播等领域有广泛的应用。Burgers 方程可以用来描述粘性流体的一维非线性对流和扩散过程。

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

其中:

- $u(x, t)$  是未知函数, 通常表示速度场。
- $t$  是时间变量。
- $x$  是空间变量。
- $\nu$  是粘性系数, 表示流体的粘性。

## 物理意义

### 1. 对流项 $u \frac{\partial u}{\partial x}$ :

- 描述了速度场  $u$  的非线性对流效应。这个项反映了流体在运动过程中速度的变化。

### 2. 扩散项 $\nu \frac{\partial^2 u}{\partial x^2}$ :

- 描述了速度场  $u$  的扩散效应。这个项反映了流体内部的粘性作用, 使得速度场趋于平滑。

## 特殊情况

### 1. 无粘性 Burgers 方程:

- 当粘性系数  $\nu = 0$  时, Burgers 方程退化为无粘性 Burgers 方程:  $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0$
- 这个方程描述了纯对流过程, 解可能会在有限时间内形成激波 (shock wave), 即速度场出现不连续性。

### 2. 粘性 Burgers 方程:

- 当  $\nu > 0$  时, 粘性项起到平滑作用, 可以防止激波的形成。解会逐渐变得光滑, 并且随着时间的推移, 速度场会趋向于均匀分布。

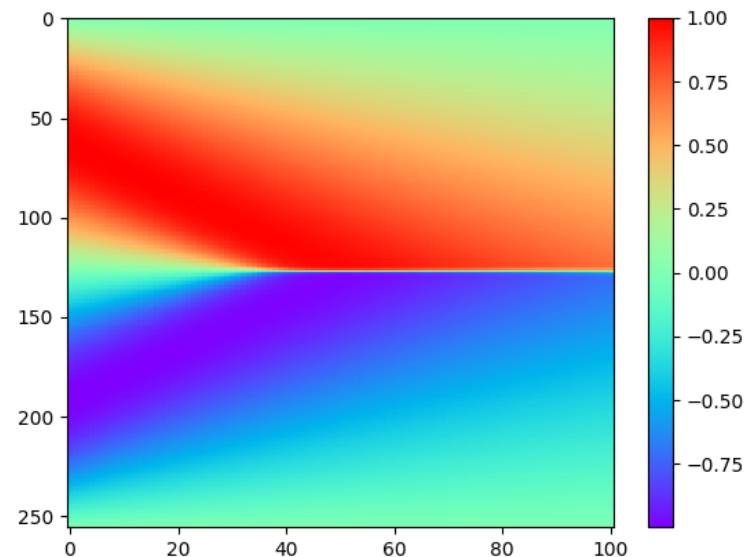
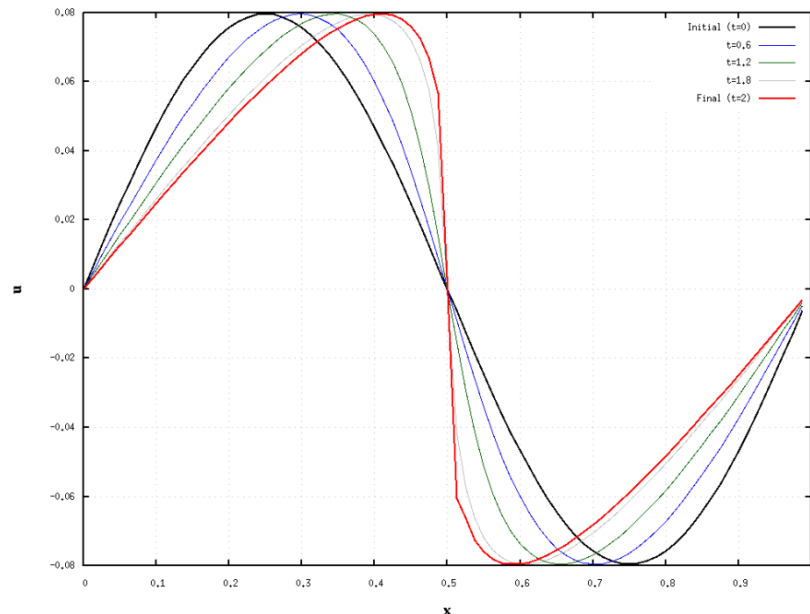
# Forward problems: Burgers equation

- Consider the following Burgers equation

$$u_t + uu_x = \nu u_{xx}, x \in (-1,1), t \in (0, T],$$

$$u(x, 0) = u_0(x), u(-1, t) = u(1, t) = 0.$$

- Let  $u_0(x) = -\sin(\pi x)$



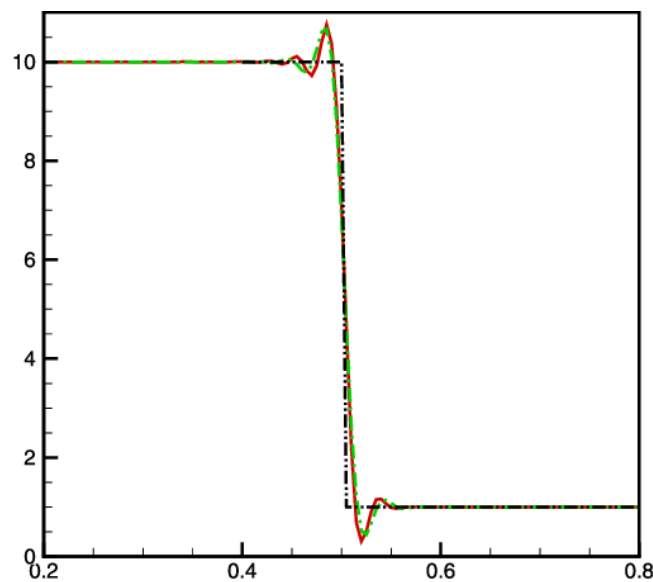
# Forward problems: Burgers equation

- Consider the following Burgers equation

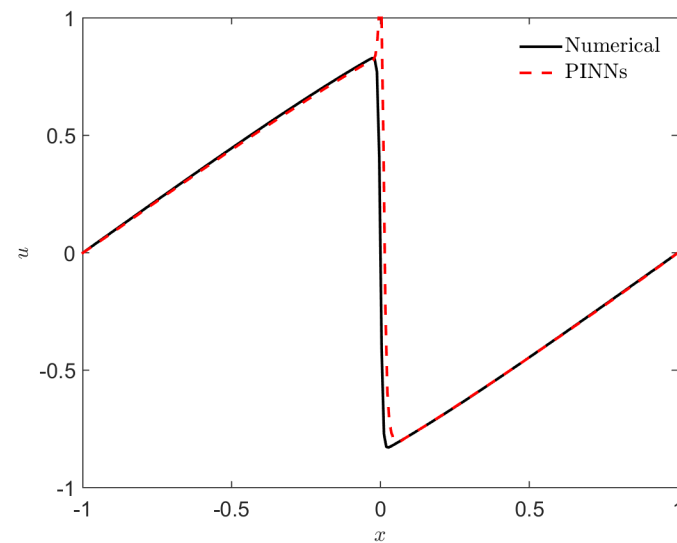
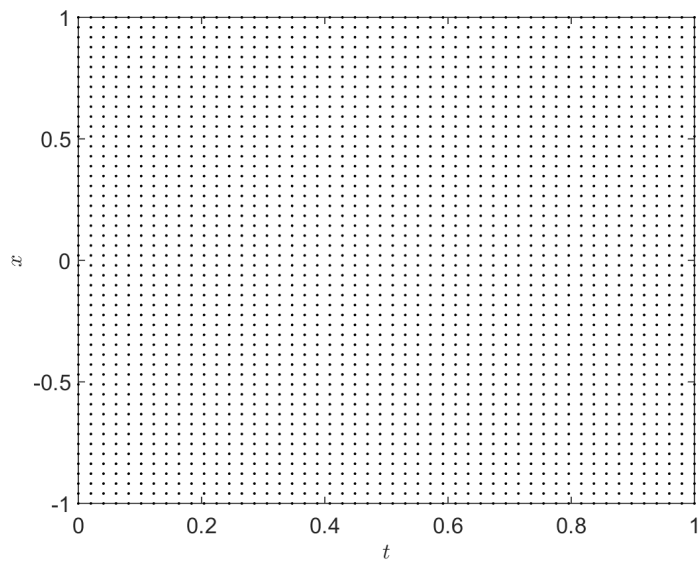
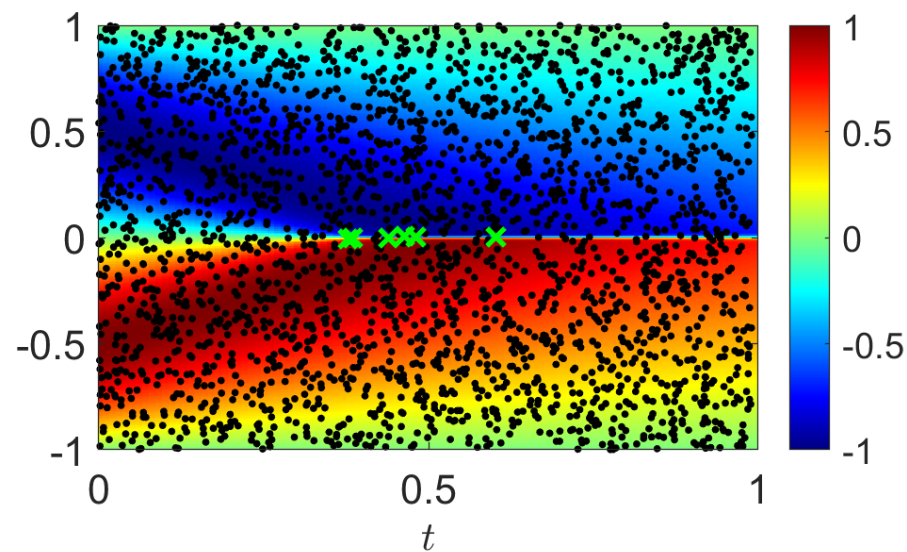
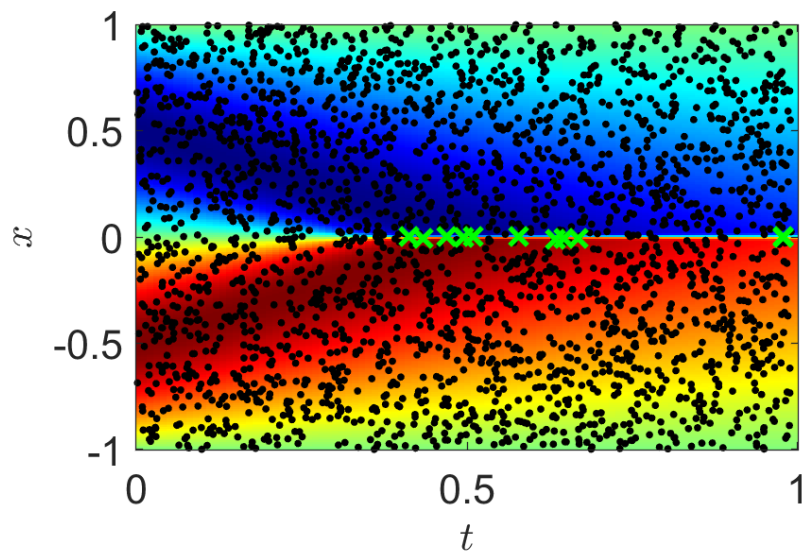
$$u_t + uu_x = \nu u_{xx}, x \in (-1,1), t \in (0, T],$$

$$u(x, 0) = u_0(x), u(-1, t) = u(1, t) = 0.$$

- Let  $u_0(x) = -\sin(\pi x)$



# Forward problems: Burgers equation





# Residual-based adaptive sampling

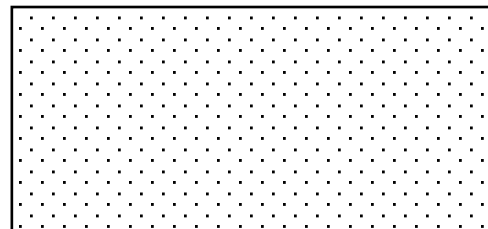
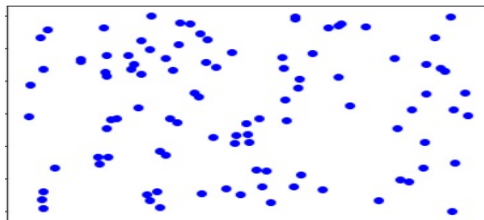
- ❑ **Residual-based adaptive sampling**
- ❑ **Residual/gradient-based adaptive sampling**
- ❑ **Weighted loss function**

# Residual-based adaptive sampling

$$L = \min_{\theta} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$$

$$R(\mathbf{x}, t) = 0$$

Step 1.



Step 2. Randomly select # testing points  $R(\mathbf{x}, t) = 0$  ?

Step 3. Add new training data at locations  $R(\mathbf{x}, t) > 0$



# Residual-based adaptive sampling

---

**Procedure 2.2** RAR for improving the distribution of residual points for training.

---

Step 1 Select the initial residual points  $\mathcal{T}$ , and train the neural network for a limited number of iterations.

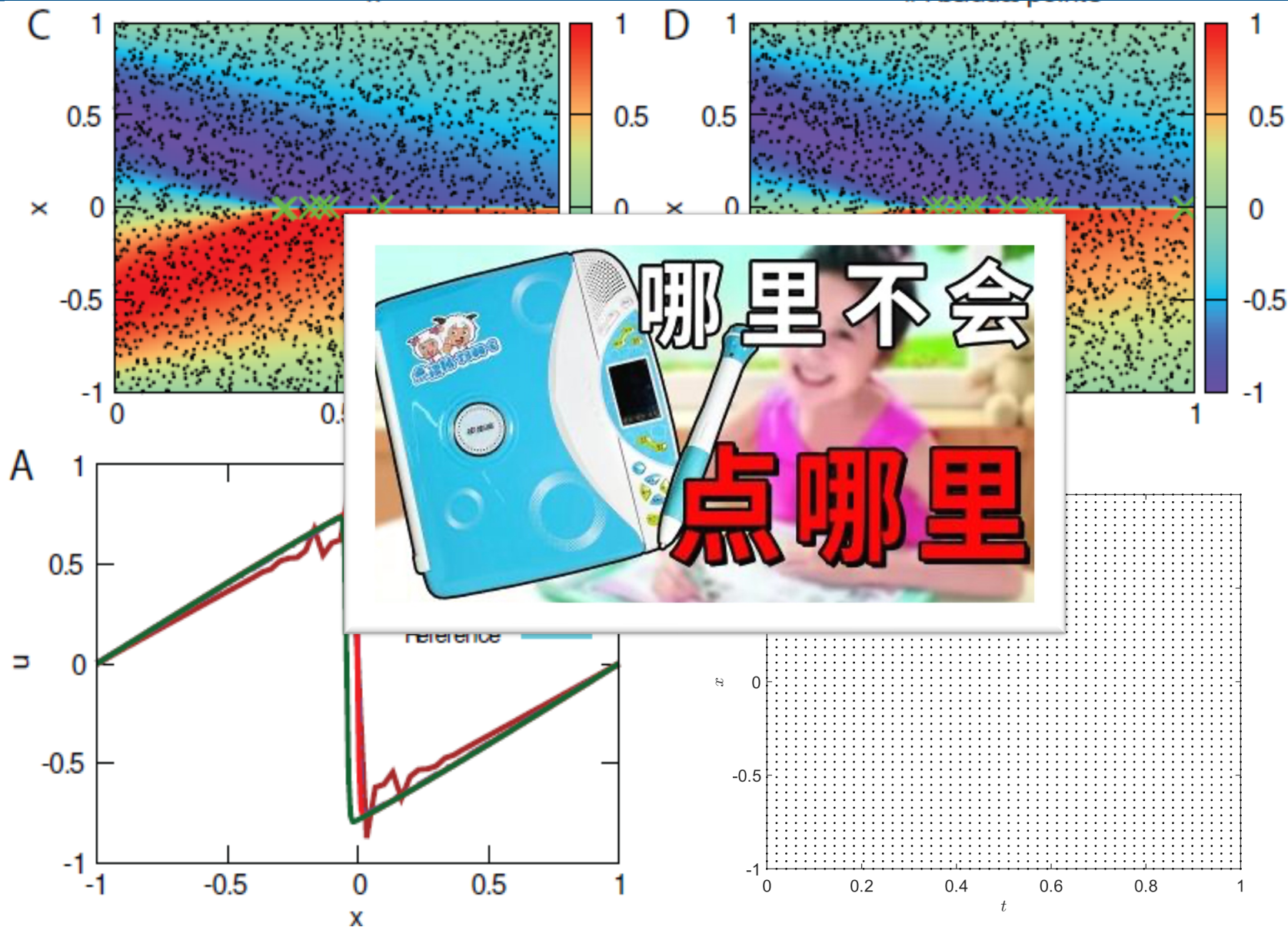
Step 2 Estimate the mean PDE residual  $\mathcal{E}_r$  in (2.3) by Monte Carlo integration, i.e., by the average of values at a set of randomly sampled dense locations  $\mathcal{S} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|\mathcal{S}|}\}$ :

$$\mathcal{E}_r \approx \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x} \in \mathcal{S}} \left\| f \left( \mathbf{x}; \frac{\partial \hat{u}}{\partial x_1}, \dots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_1}, \dots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \dots; \lambda \right) \right\|.$$

Step 3 Stop if  $\mathcal{E}_r < \mathcal{E}_0$ . Otherwise, add  $m$  new points with the largest residuals in  $\mathcal{S}$  to  $\mathcal{T}$ , retrain the network, and go to Step 2.

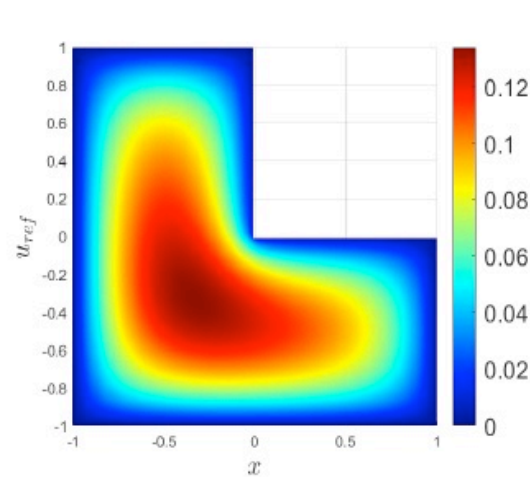
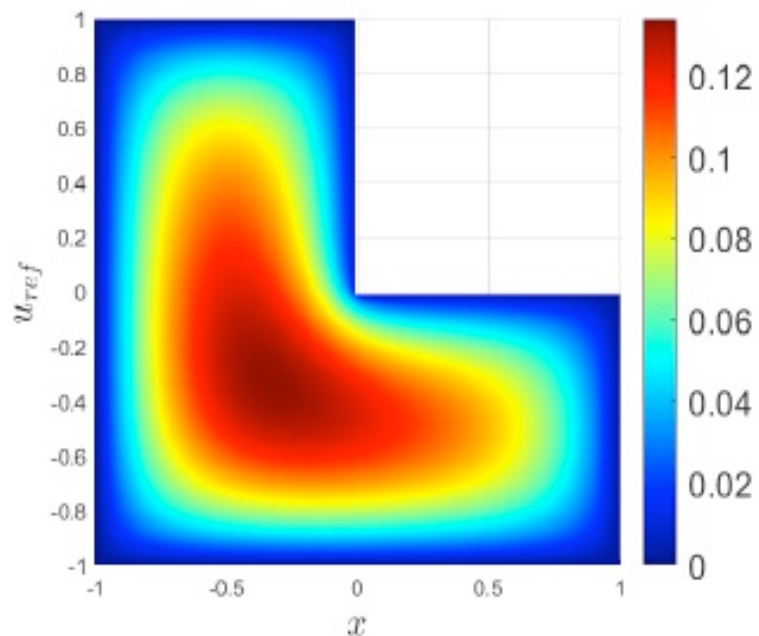
---

# Residual-based adaptive sampling

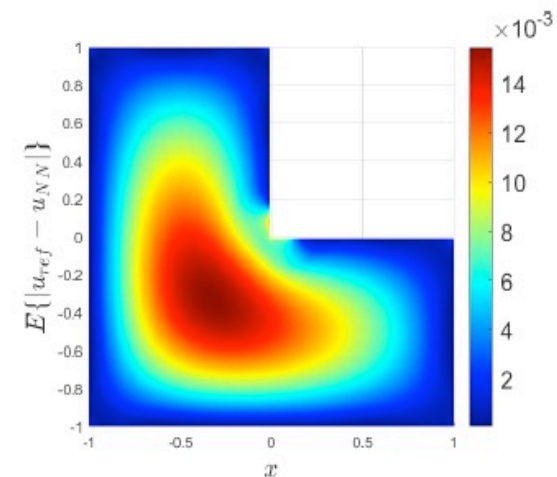


# Residual-based adaptive sampling

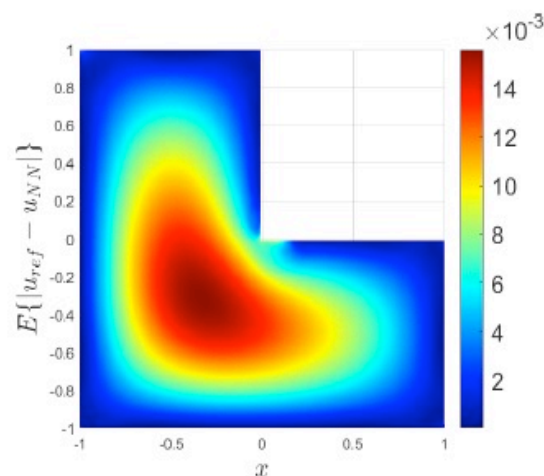
$$\begin{aligned} -\Delta u(x, y) &= 1, & (x, y) \in \Omega, \\ u(x, y) &= 0, & (x, y) \in \partial\Omega. \end{aligned} \quad x, y \in [-1, 1]$$



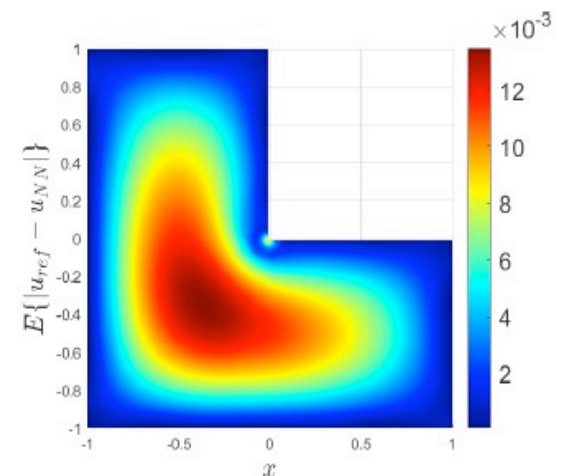
(a) Reference solution



(b) ASM I:  $E\{|u_{ref} - u_{NN}|\}$



(c) ASM II:  $E\{|u_{ref} - u_{NN}|\}$



(d) ASM III:  $E\{|u_{ref} - u_{NN}|\}$

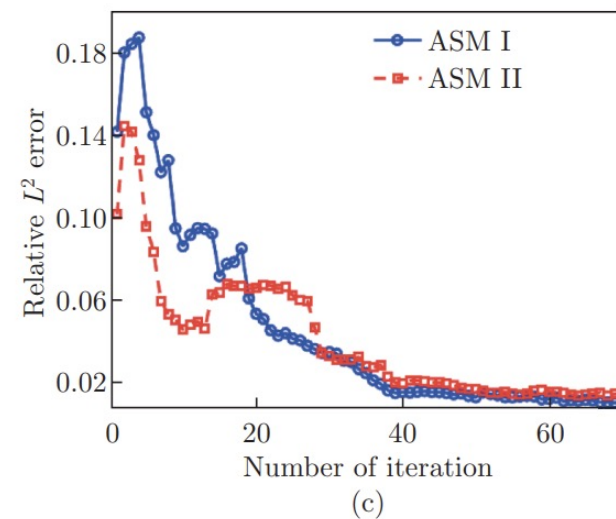
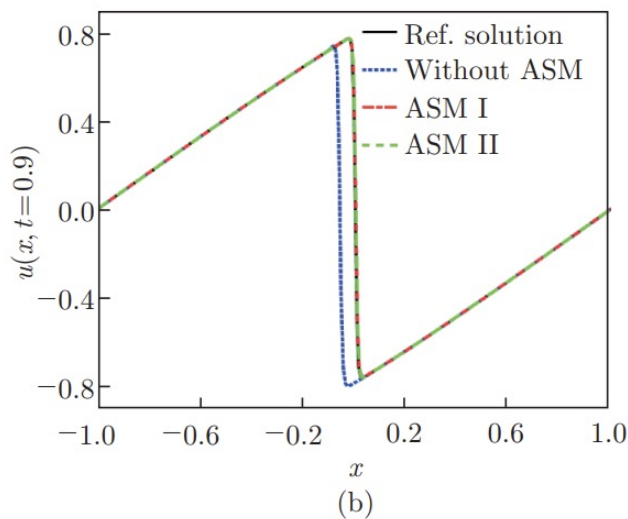
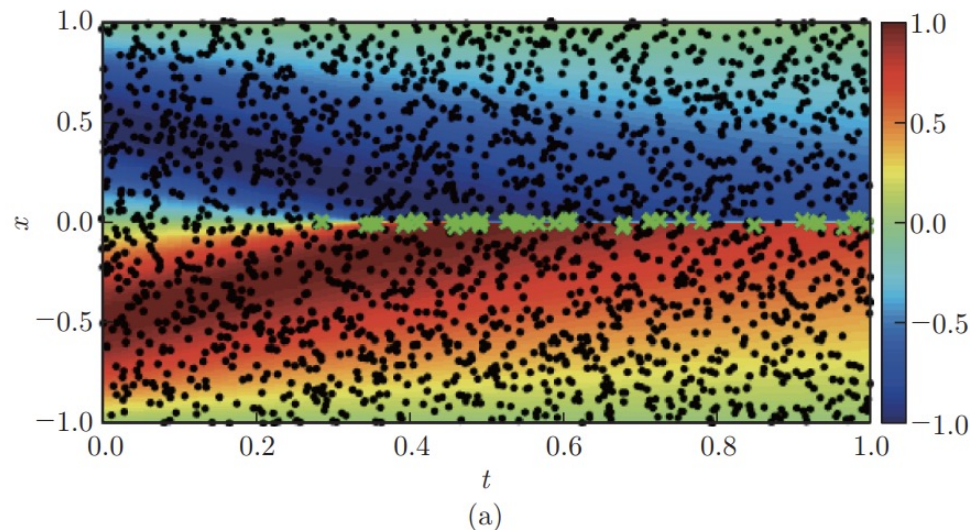


# Residual-based adaptive sampling

- Easy to implement
- Computationally efficient
- Effective for both spatial and temporal domain

# Residual/gradient-based adaptive sampling

The gradient of the solution can be used to measure the sharpness of the solution.  
A new adaptive sampling based on both the residual and the gradient of the solution.





# Residual/gradient-based adaptive sampling

$$\begin{aligned} -\Delta u(x) &= f(x), \quad x \in \Omega := [-1, 1]^d, \\ u(x) &= 0, \quad x \in \partial\Omega. \end{aligned}$$

ASM = Adaptive Sampling Methods

$$u(x) = e^{-10\|x\|^2} \quad d = 9$$

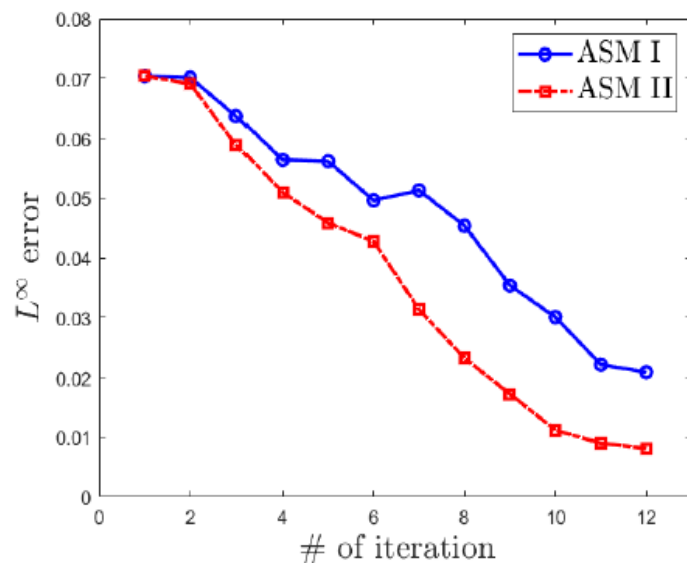


Figure 9: Convergence of the  $L^\infty$  error for the high dimensional Poisson problem using ASM I and ASM II.  $m_R = 500$  for ASM I while  $m_R = 300$ ,  $m_{\nabla u} = 200$  for ASM II.



□ **Residual-based adaptive sampling**

□ **Residual/gradient-based adaptive sampling**

□ **Weighted loss function**



# Weighted loss function

$$L = \min_{\theta} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$$



$$L = \min_{\theta} (w_R R^2 + w_{bc} |u_{bc}^* - u_{bc}|^2 + w_{ic} |u_{ic}^* - u_{ic}|^2)$$



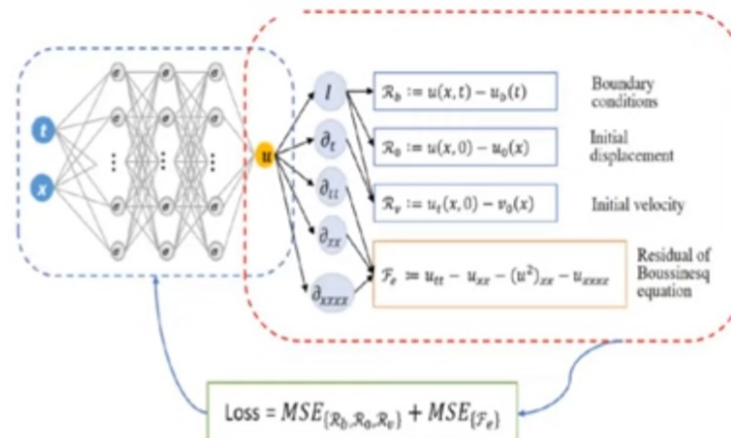
# Weighted loss function

Levi D. McClenny\*   Ulisses Braga-Neto  
 Department of Electrical and Computer Engineering  
 Texas A&M University  
 College Station, TX USA  
 {levimcclenny, ulisses}@tamu.edu

Boussinesq equation  $\mathcal{F} = u_{tt} - u_{xx} - (u^2)_{xx} - u_{xxxx}$

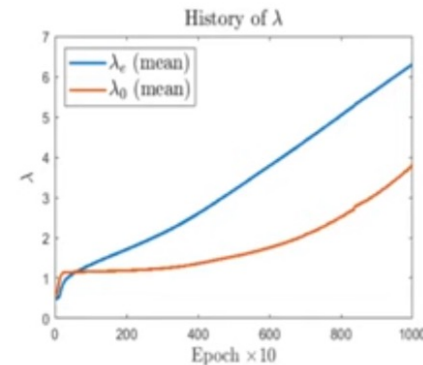
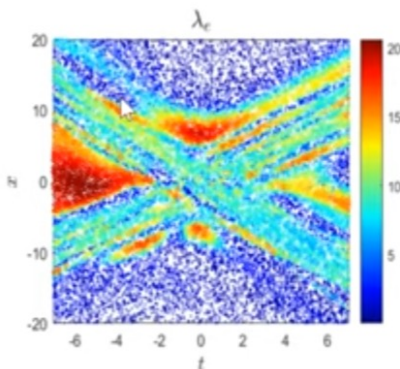
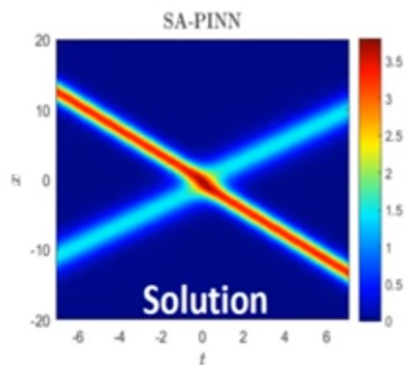
**Vanilla PINN:** PINN without any weighting coefficient in the loss function, i.e.  $L = L_b + L_0 + L_e$

**Self-adaptive PINN:**  $L = L_b + \lambda_0 L_0 + \lambda_e L_e$ , where  $\lambda_0$  and  $\lambda_e$  (point-wise) are trainable



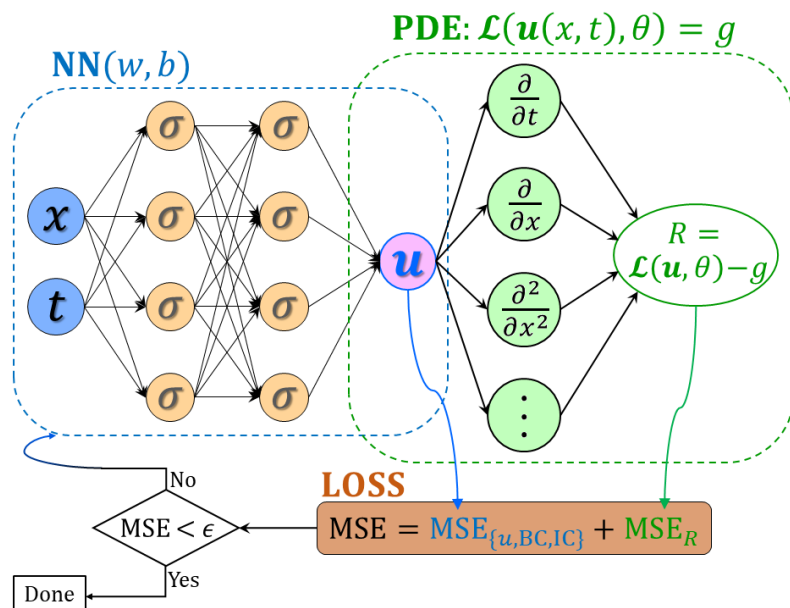
Relative L2-norm error

|                    |
|--------------------|
| Vanilla PINN       |
| 0.99+0.28%         |
| Self adaptive PINN |
| 0.43+0.18%         |



# PINN for Inverse Problems

## PINN for Inverse Problems



Physics-informed neural networks (PINNs)

$$(1) \quad u_1 = \sigma(W_1^T X + b_1)$$

$$u_2 = \sigma(W_2^T u_1 + b_2)$$

...

$$u = W_l^T u_{l-1} + b_l$$

(2) PDE part: *Chain rule*

$$\partial_t u, \partial_x u, \dots$$

Loss function:

Forward problem:  $L = \min_{\theta} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$

Inverse problem:  $L = \min_{\theta, \lambda} (R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2 + |u^* - u|^2)$



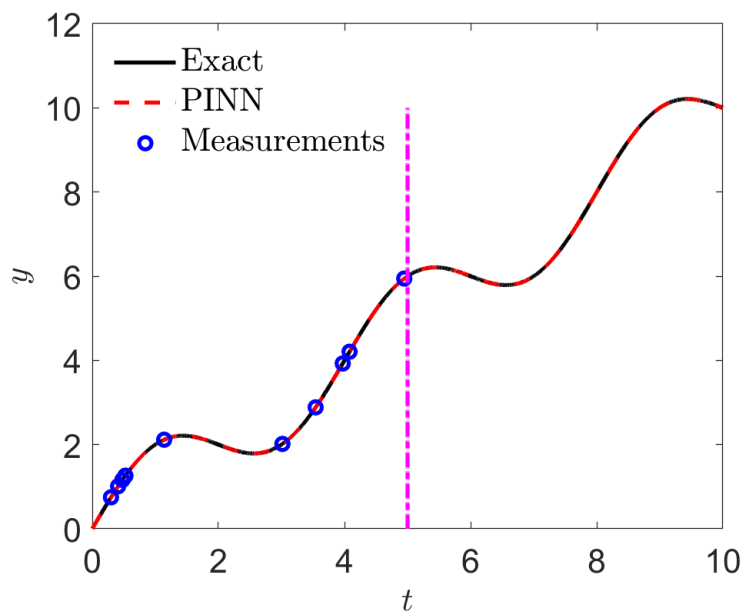
# PINN for Inverse Problems

$$\frac{dy}{dt} = \gamma \left( a + \frac{\pi}{2} \cos \left( \frac{\pi t}{2} \right) \right), t \in [0, 10], a = 1$$

Inverse:  $\gamma$  *unknown*, *partial observations* on  $y$

Solution:  $y = t + \sin(\pi t/2)$

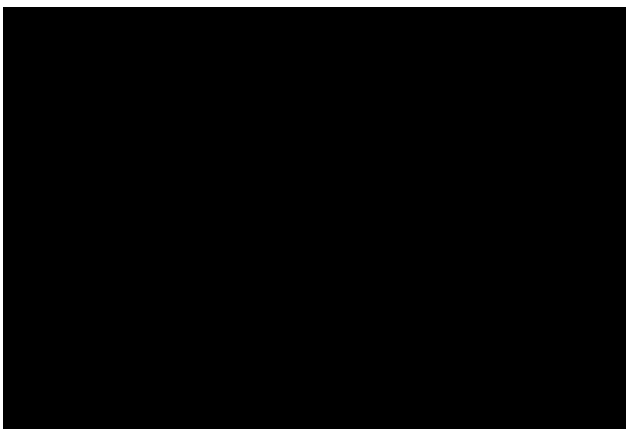
Loss:  $L = |R|^2 + |y_{ic} - y_{ic}^*|^2 + |y - y^*|^2$



$$\gamma = 1.000018$$

# PINN for Inverse Problems - VIV

## Vortex Induced Vibrations



### Inverse problem: PINN for VIV

Physical Model:

$$\begin{aligned} m \frac{\partial^2 Y}{\partial t^2} + \beta \frac{\partial Y}{\partial t} - T \frac{\partial^2 Y}{\partial z^2} - EI \frac{\partial^4 Y}{\partial z^4} &= f_l \\ m \frac{\partial^2 X}{\partial t^2} + \beta \frac{\partial X}{\partial t} - T \frac{\partial^2 X}{\partial z^2} - EI \frac{\partial^4 X}{\partial z^4} &= f_d \end{aligned}$$

$Y$ : cross-flow displacement  
 $f_l$ : lift

$X$ : in-line displacement  
 $f_d$ : drag

Model Parameters

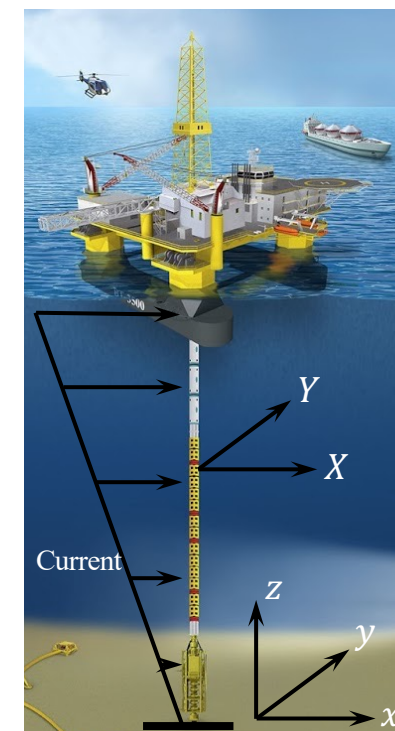
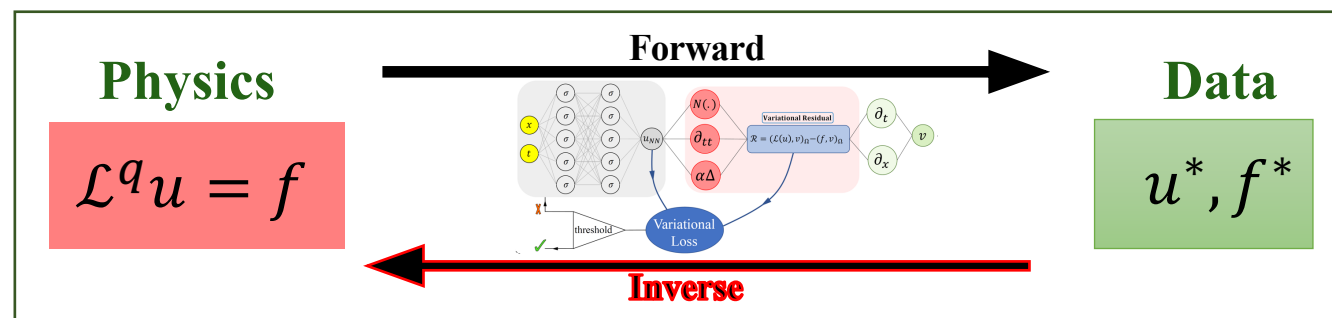
mass per  
unit length

damping  
coefficient

tension

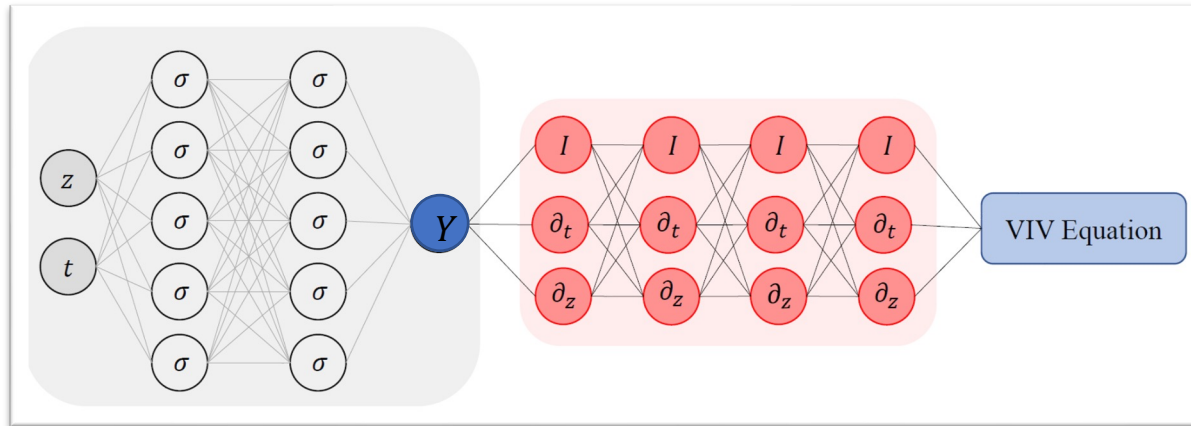
flexural  
rigidity

Distributed damping coefficient: parametrize with a neural network  $\beta(z) = \beta_{NN}(z)$



# PINN for Inverse Problems - VIV

[Kharazmi et al, JFS, 2020]



## Network Parameters

|                              |                            |
|------------------------------|----------------------------|
| structure = fully connected  | # layers = 4               |
| # neurons in each layer = 20 | activation function = sine |
| optimizer = Adam             | learning rate = $10^{-3}$  |

NN parameters:  $\theta$

Model parameters:  $q = \{m, \beta, T, EI\}$

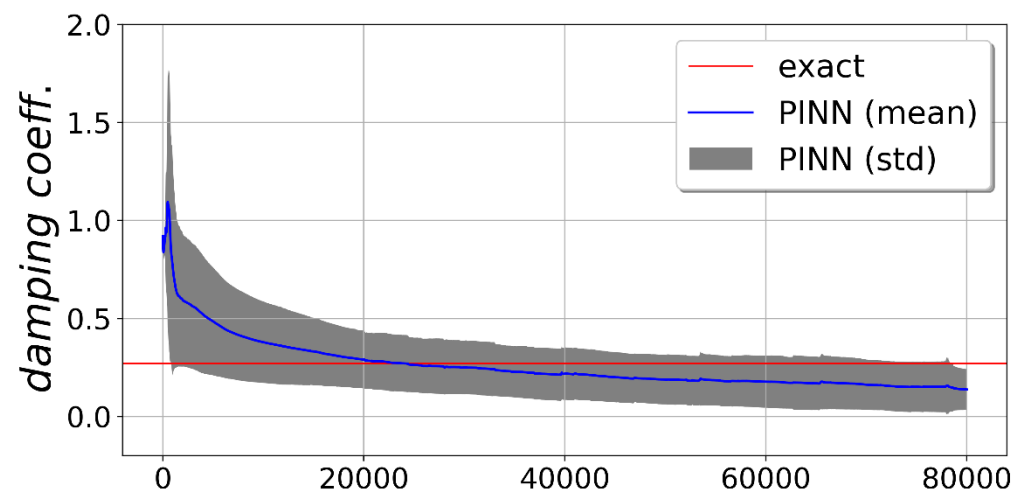
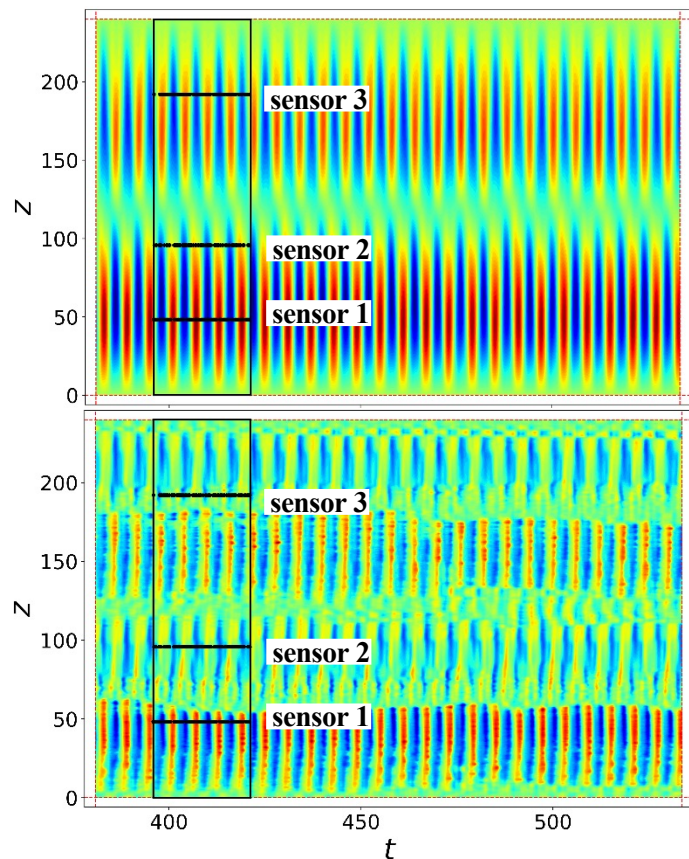
PDE residual:

$$r_{NN} = m \frac{\partial^2 Y}{\partial t^2} + \beta \frac{\partial Y}{\partial t} - T \frac{\partial^2 Y}{\partial z^2} - EI \frac{\partial^4 Y}{\partial z^4} - f_l$$

Loss function:

$$loss = \underbrace{\frac{1}{N_Y} \sum_{i=1}^{N_Y} |Y_{NN}(z_i, t_i; \theta) - Y(z_i, t_i)|^2}_{\substack{\text{Loss } Y(z_i, t_i : \text{Training points}) \\ Y \text{ Displacement}}} + \underbrace{\frac{1}{N_r} \sum_{j=1}^{N_r} |r_{NN}(z_j, t_j; \theta, q)|^2}_{\substack{\text{Loss PDE } (z_j, t_j : \text{Residual points}) \\ f_l \text{ Lift force}}}$$

# PINN for Inverse Problems - VIV

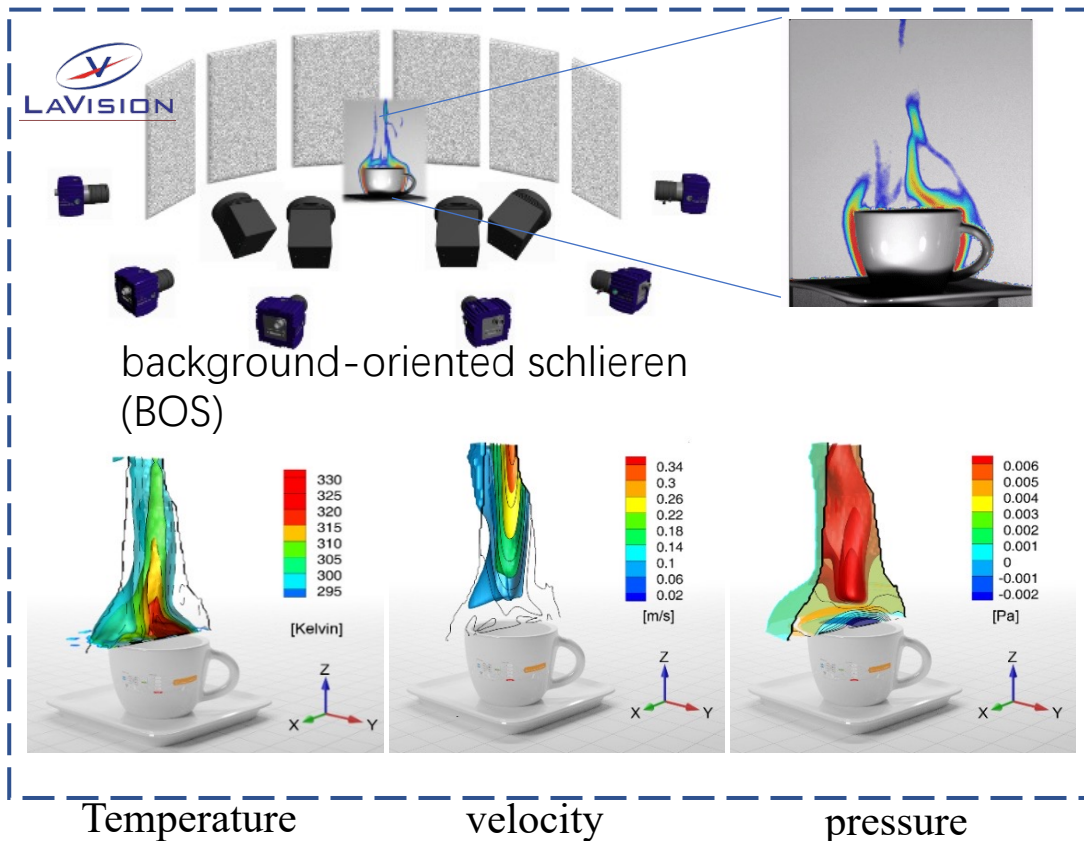


The response with significant noise in the input.  
20% added noise to the lift force.  
Mean/std is based on 30 realizations.



# PINN for Inverse Problems – Thermal Flow

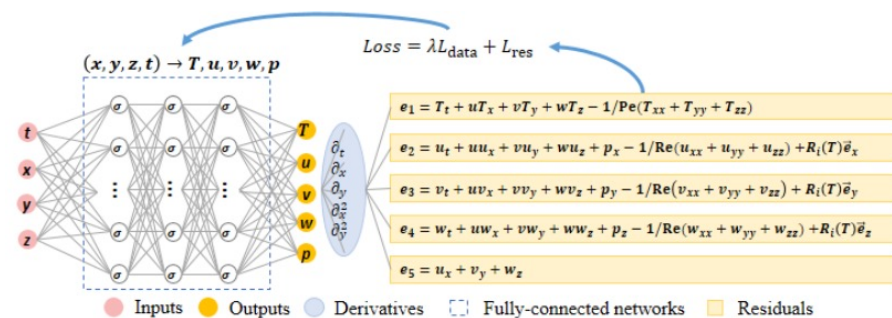
## PINN for Inverse Problems: Flow Over an Espresso Cup



$$\partial_t T + \mathbf{u} \cdot \nabla T = \alpha \nabla^2 T$$

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

$$\begin{aligned} \rho \left( \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} \right) &= \rho f_x - \frac{\partial p}{\partial x} + \mu \left( \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \\ \rho \left( \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} \right) &= \rho f_y - \frac{\partial p}{\partial y} + \mu \left( \frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \\ \rho \left( \frac{\partial w}{\partial t} + u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} \right) &= \rho f_z - \frac{\partial p}{\partial z} + \mu \left( \frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \end{aligned}$$



Tomo-BOS（层析背景纹影）实验，六个相机，可重构三维的温度场。

PINNs以温度场作为训练数据，计算出三维+时间变化的速度和压力。



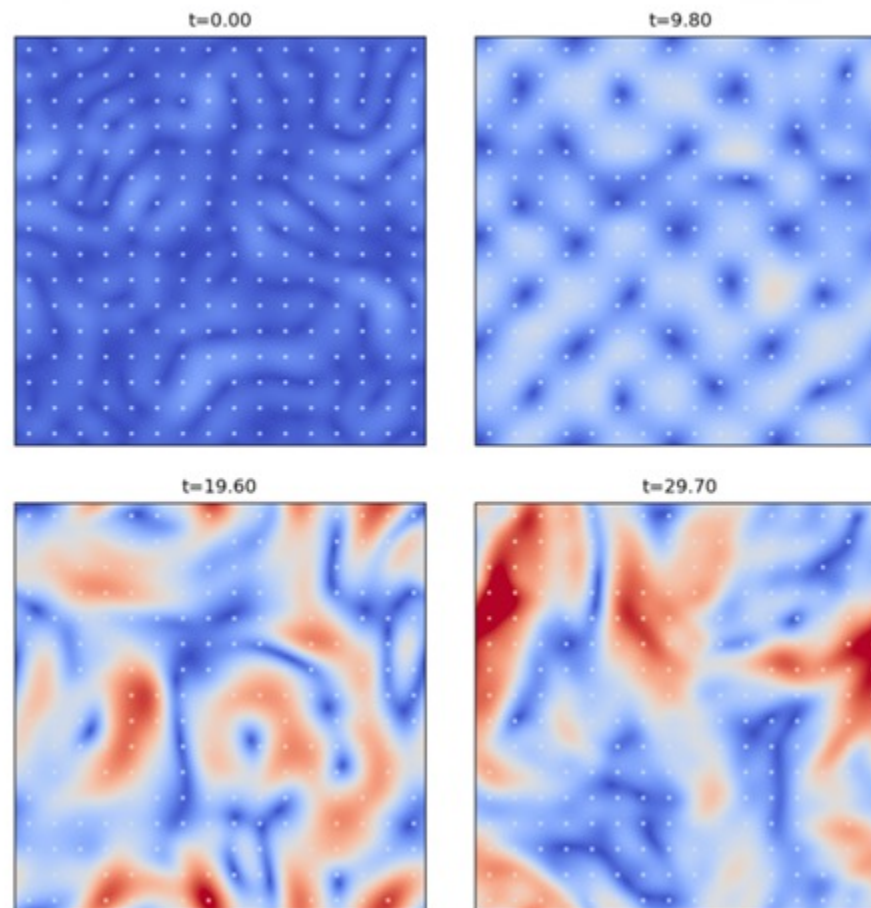
# PINN applications - Super-Resolution





## 赛题描述

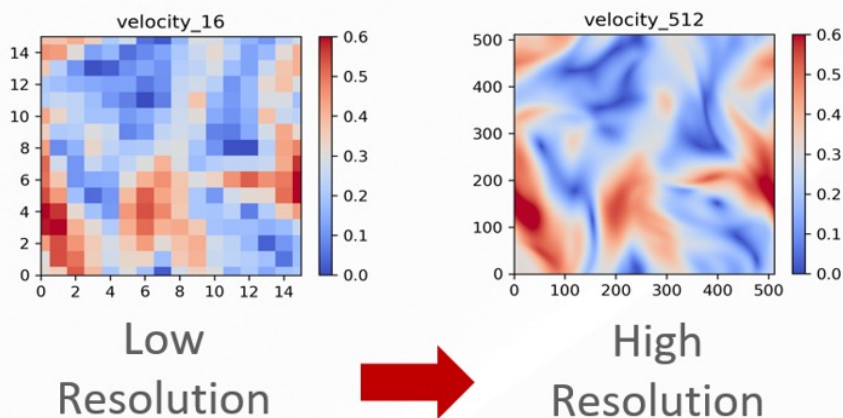
- 第一类任务 (**反问题**) :  
在0-30个时间单位, 已知固定时间间隔的切片上稀疏物理场, 重构出高分辨率的物理场数据
- 第二类任务 (**正问题**) :  
在30.3-36个时间单位, 根据前面求解得到的物理场信息, 推理后续没有任何监督点的物理场数据



# PINN applications - Super-Resolution

## PINNs+空间超分

### 空间超分



LR data  
(3\*101\*16\*16)

+

NS equation

↓

HR data(3\*101\*512\*512)

### Methods

Important tips(hard constrain, smoothness constrain, stream function, random dynamic sampling,...)

$$\mathbf{u}_t + (\mathbf{u} \cdot \nabla) \mathbf{u} = \nu \Delta \mathbf{u} - \frac{1}{\rho} \nabla p + \mathbf{f}$$

Inputs

$x, y, t$

Physics  
informed  
NNs

Outputs

$\mathbf{u}_{pred}$

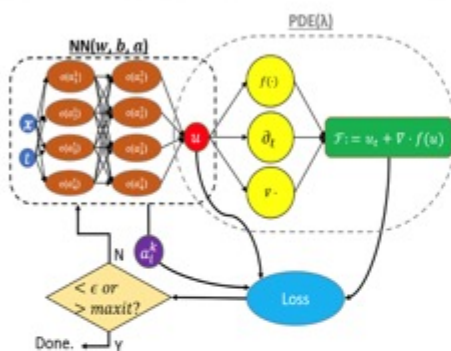
$data_{low}$

eg: smoothness constrain  
(Poisson equation)

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = -\rho \left[ \left( \frac{\partial u}{\partial x} \right)^2 + 2 \frac{\partial u}{\partial y} \frac{\partial v}{\partial x} + \left( \frac{\partial v}{\partial y} \right)^2 \right]$$

# PINN applications - Super-Resolution

激活函数

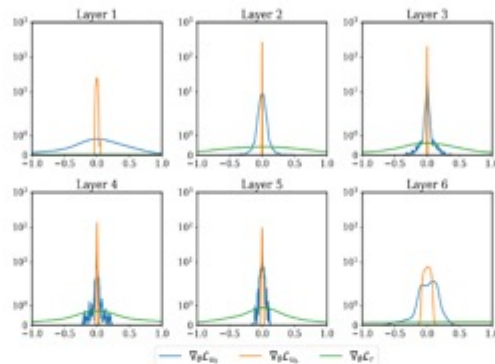


自适应激活函数

$$\sigma(a_k L_k(x^{k-1}))$$

自适应激活因子

损失权重

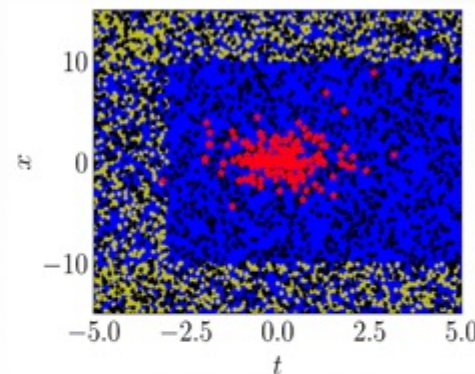


自适应加权

$$L(\theta) := L_r(\theta) + \lambda_i \sum_i^k L_i(\theta)$$

自适应权重

配点采样



自适应采样

损失/梯度概率分布

PINNs优化求解：主要有效手段（仅列举3种）

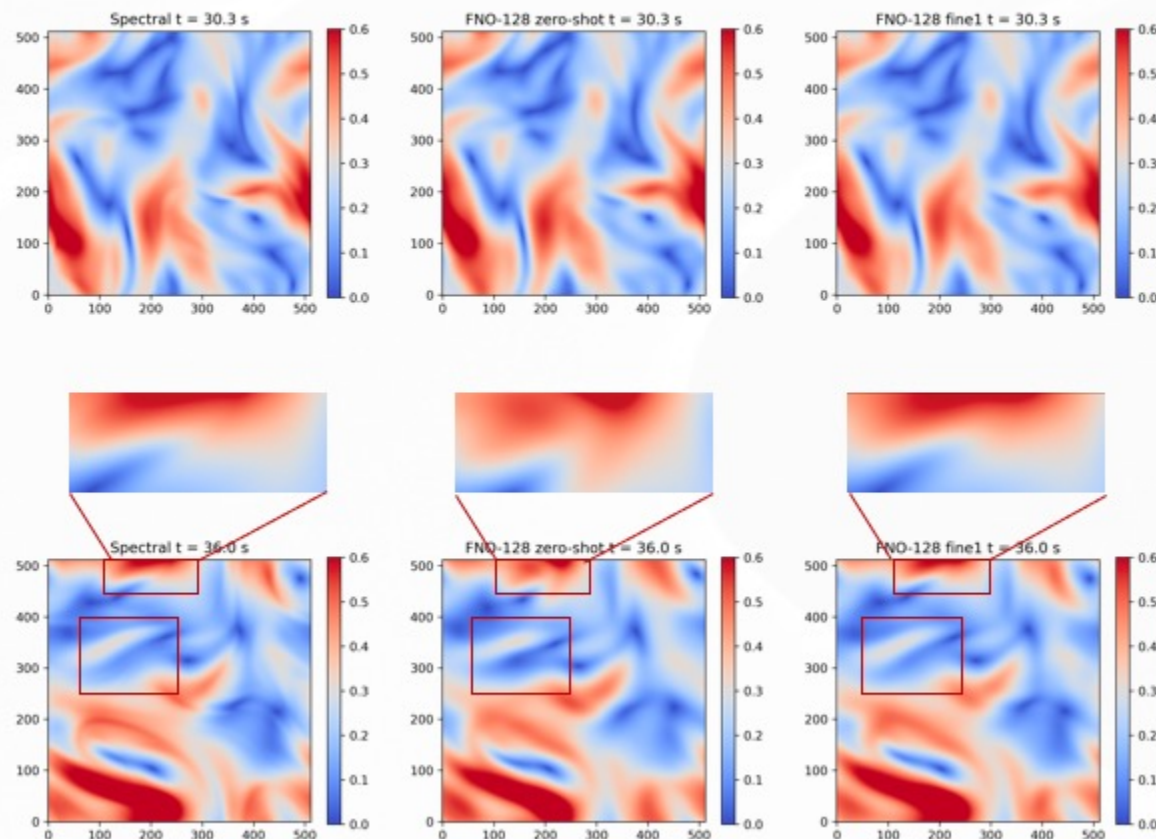


# PINN applications - Super-Resolution

## The results of spectral method, zero-shot, and fine-tune

- 如右图所示，无论是定量计算（与谱方法结果的相对误差在 $10^{-3}$ 量级），还是可视化，我们发现，**PI-FNO**可以达到与谱方法相近的效果

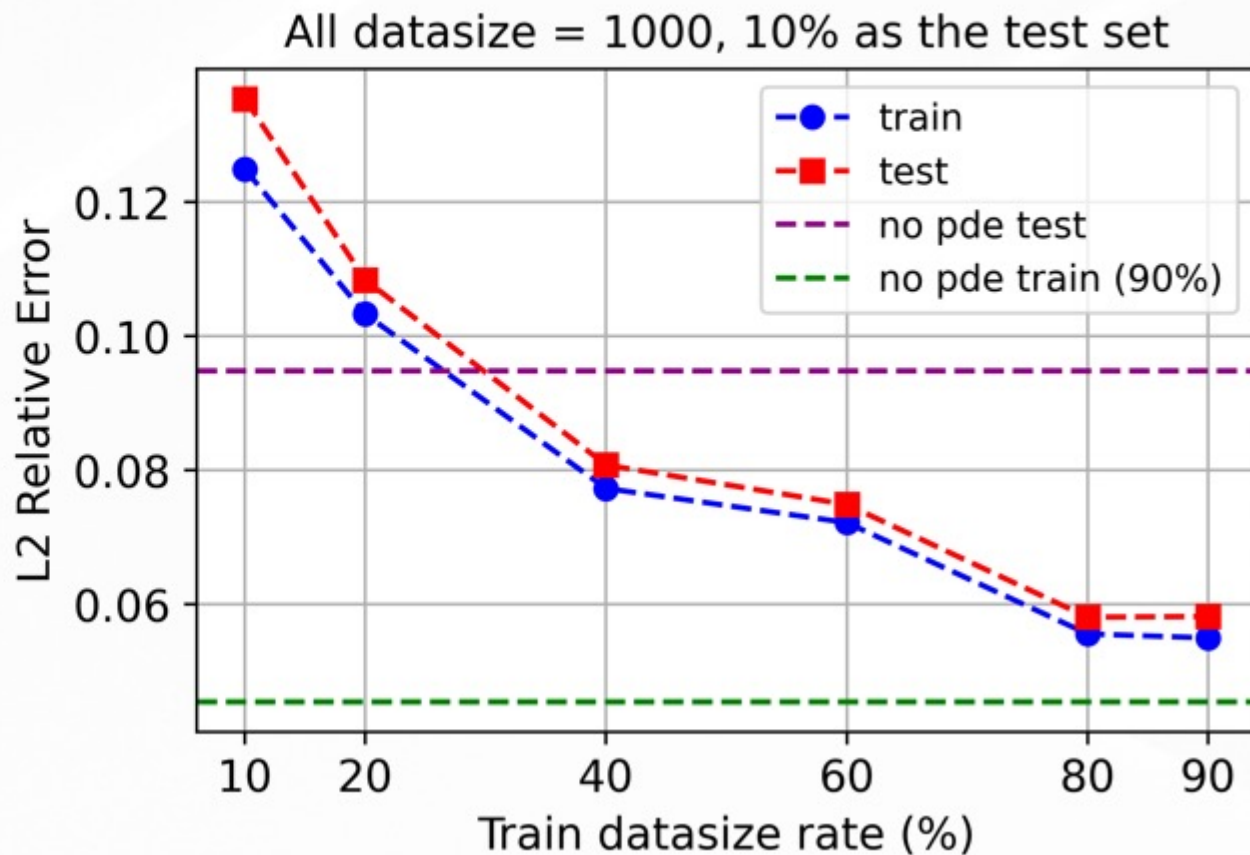
- 从效率上看，PI-FNO速度比谱方法快 $10^3$ 数量级倍



## 数据量消融实验

两个发现:

1. **PDE loss+30% data  $\approx$  90% data**
2. 加入PDE loss后可以明显缩小训练误差曲线与测试误差曲线之间的差距 (**Learning Biases**)，降低了训练集的数据量要求



# PINN applications - aneurysm

Science

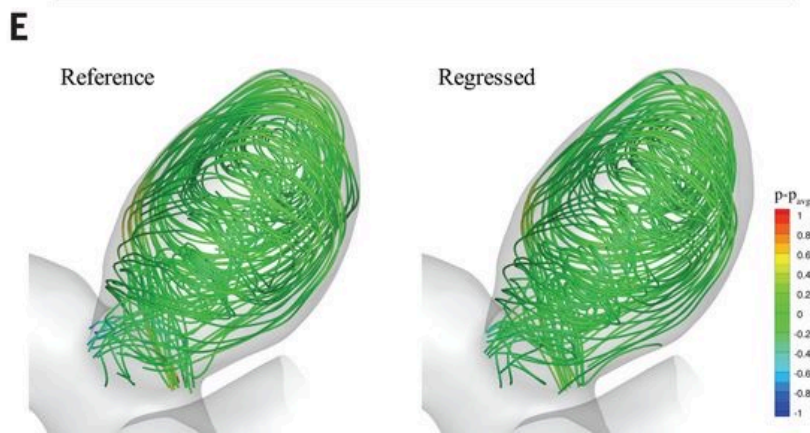
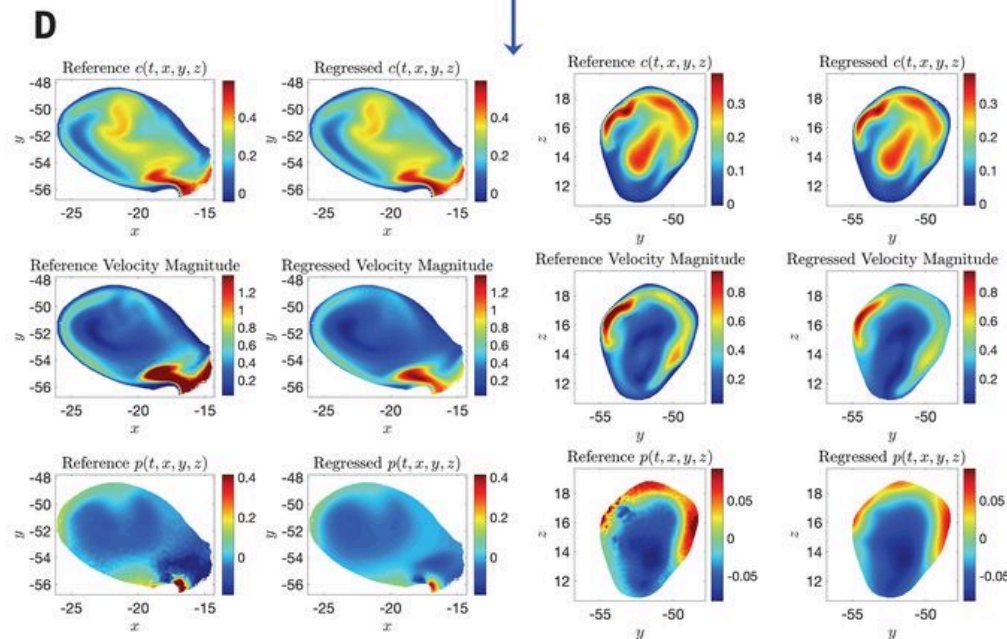
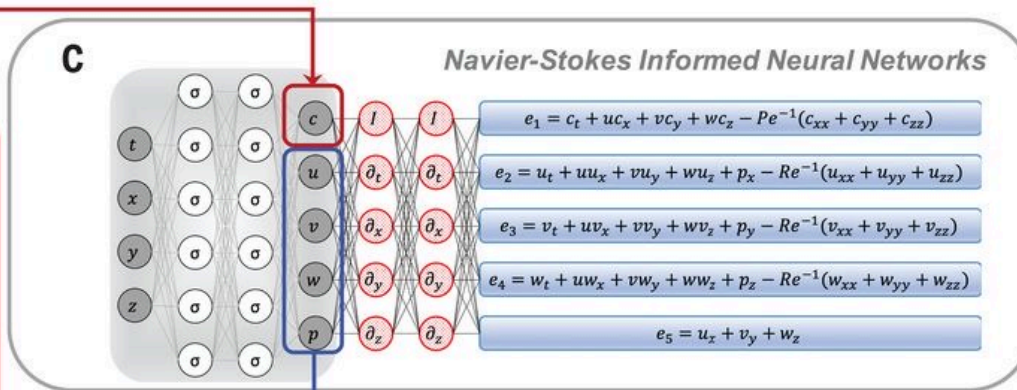
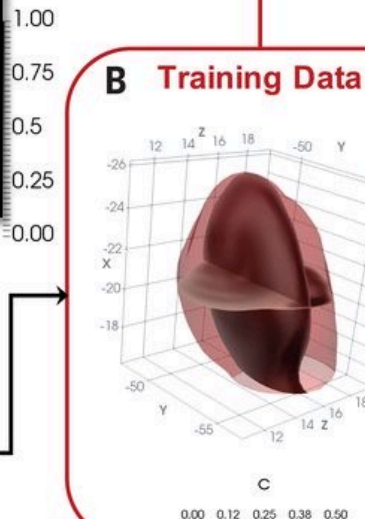
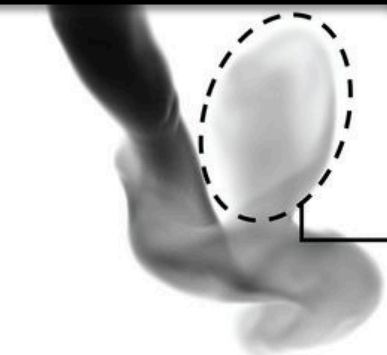
Current Issue First release papers Archive About Submit manuscript

HOME > SCIENCE > VOL. 367, NO. 6481 > HIDDEN FLUID MECHANICS: LEARNING VELOCITY AND PRESSURE FIELDS FROM...

REPORT

## Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations

MAZIAR RAISSI, ALIREZA YAZDANI, AND GEORGE EM KARNIADAKIS Authors Info & Affiliations

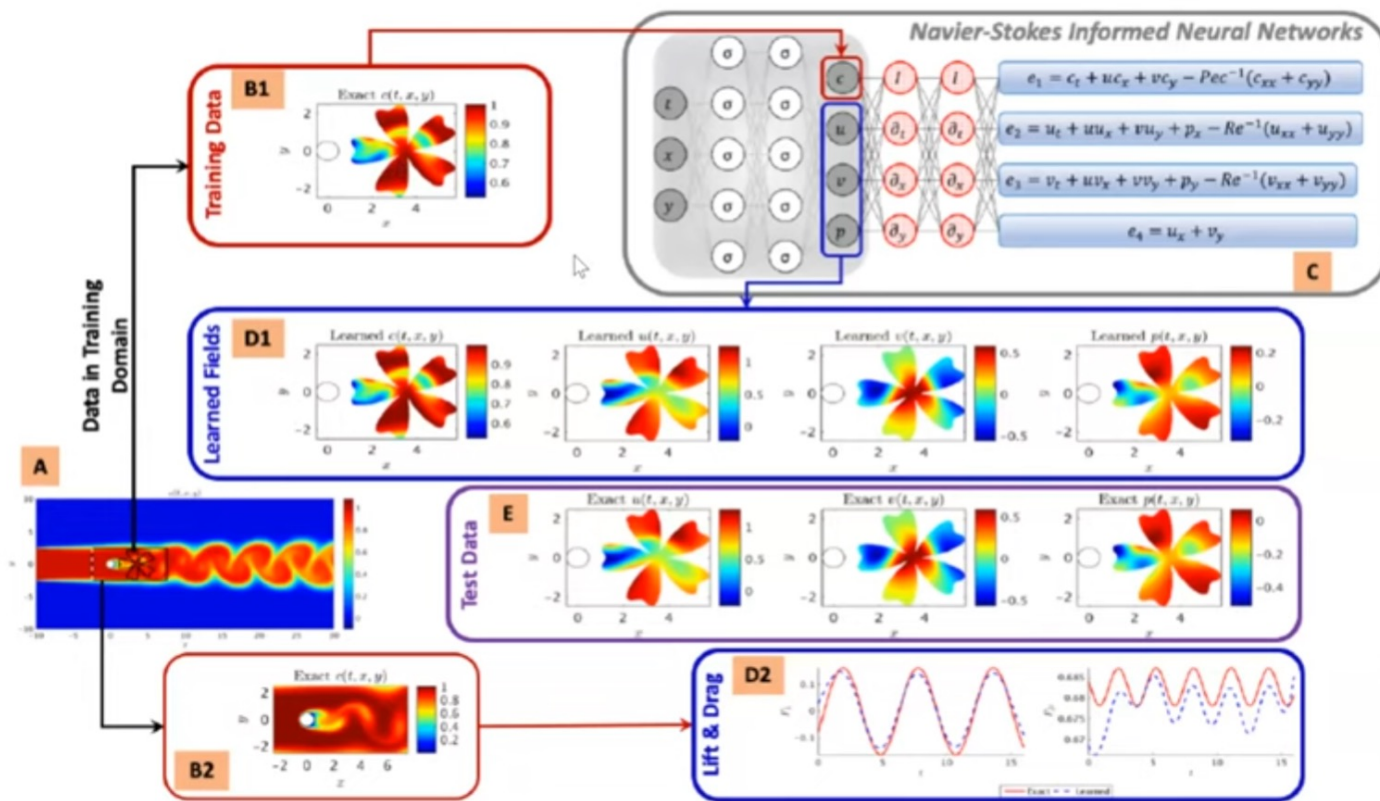




# PINN applications - aneurysm

## Hidden Fluid Mechanics

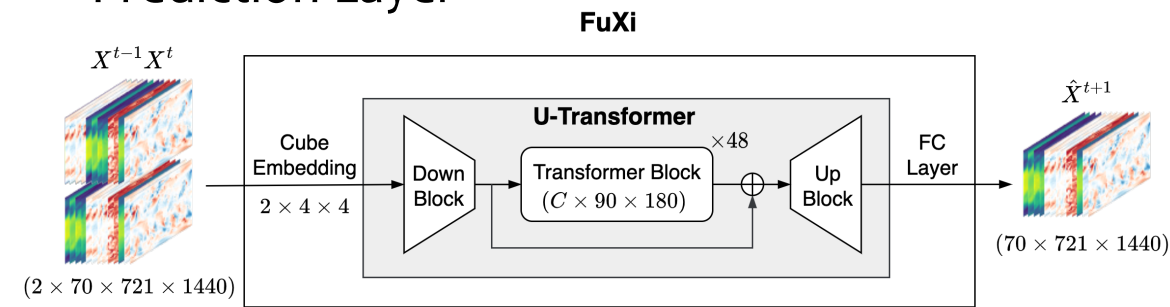
M. Raissi, A. Yazdani and G.E. Karniadakis, [arXiv:1808.04327](https://arxiv.org/abs/1808.04327)





# PINN applications – Super resolution for FuXi

- Cube embedding 编码
- U-Net style block: Down & Up Block
- 48 Swin Transformer blocks
- Prediction Layer

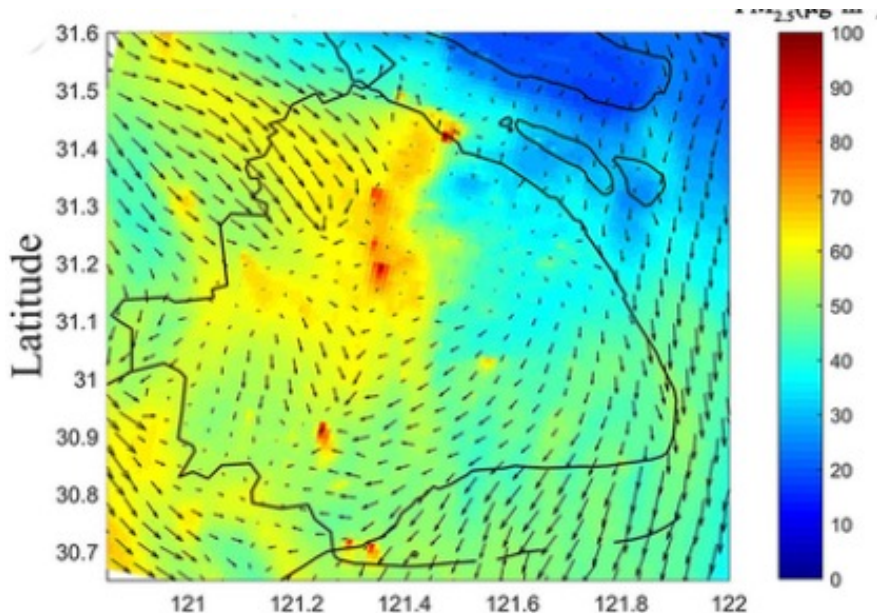


- **ERA5**
  - 逐小时的日均值
  - 训练集: **1950–2016**年 (67年)
  - 测试集: **2017–2021**年 (与ECMWF次季节模式历史回报对比)
  - 测试集: **2022**年 (与ECMWF的次季节模式实时预报对比)

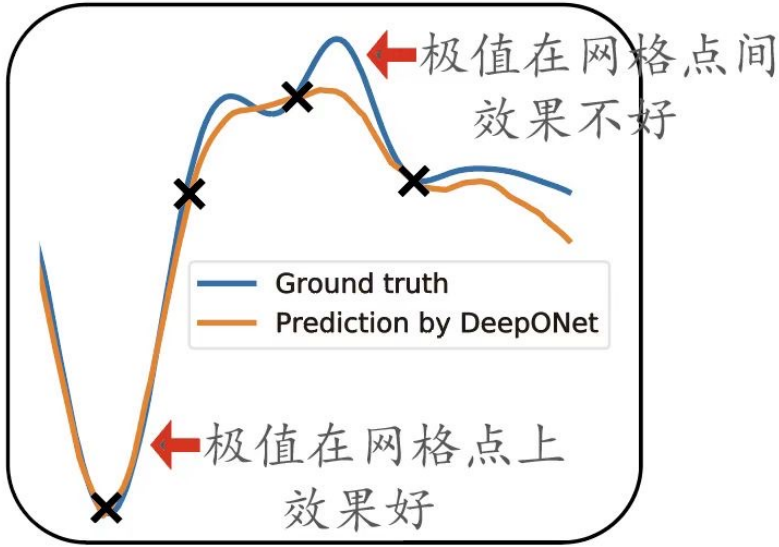
| Type                | Name                         | Short Name |
|---------------------|------------------------------|------------|
| Upper-air variables | Geopotential                 | Z          |
|                     | Temperature                  | T          |
|                     | U component of wind          | U          |
|                     | V component of wind          | V          |
|                     | Specific humidity            | Q          |
| Surface variables   | 2 meter temperature          | T2M        |
|                     | 2-meter dewpoint temperature | D2M        |
|                     | sea surface temperature      | SST        |
|                     | outgoing longwave radiation  | OLR        |
|                     | 10 meter u wind component    | U10        |
|                     | 10 meter v wind component    | V10        |
|                     | 100 meter u wind component   | U100       |
|                     | 100 meter v wind component   | V100       |
|                     | Mean sea level pressure      | MSL        |
|                     | total column water vapor     | TCWV       |
|                     | Total precipitation(6h)      | TP         |



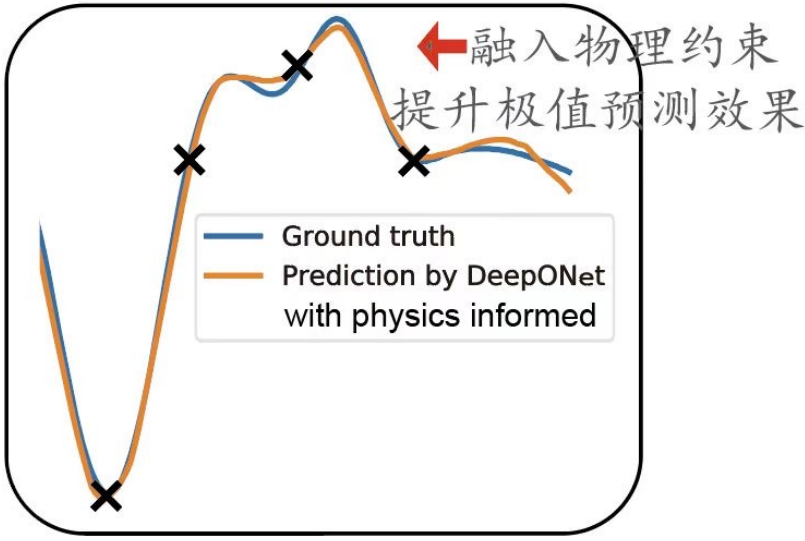
# PINN applications – Super resolution



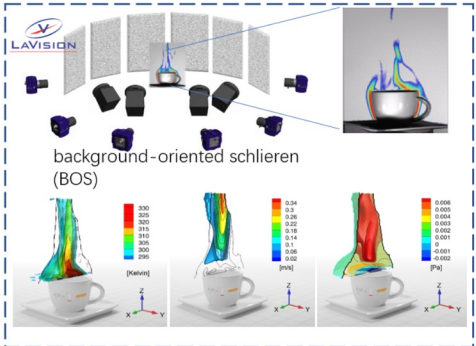
纯数据驱动



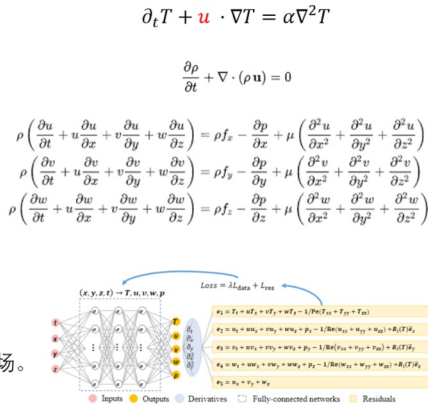
物理+  
数据双驱动



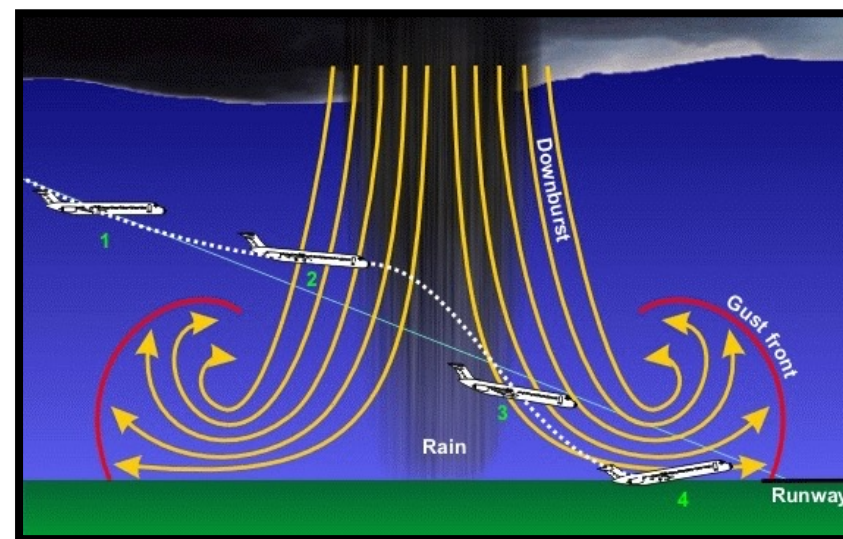
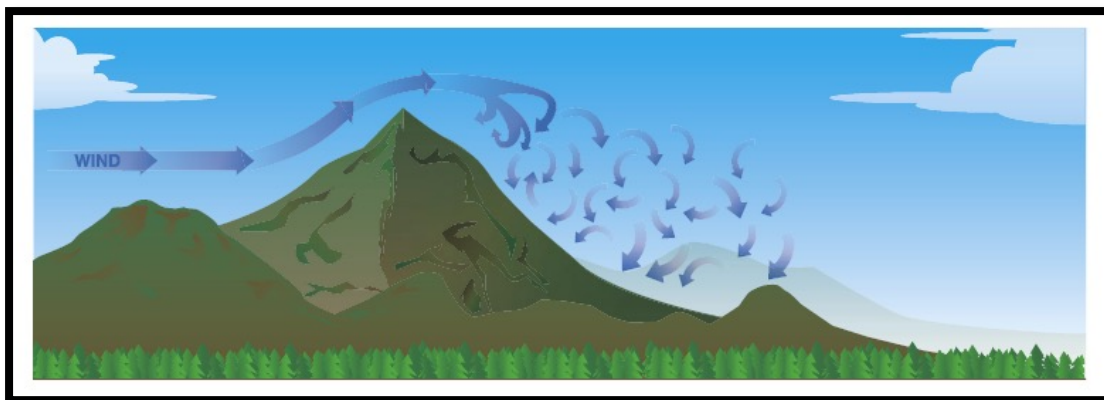
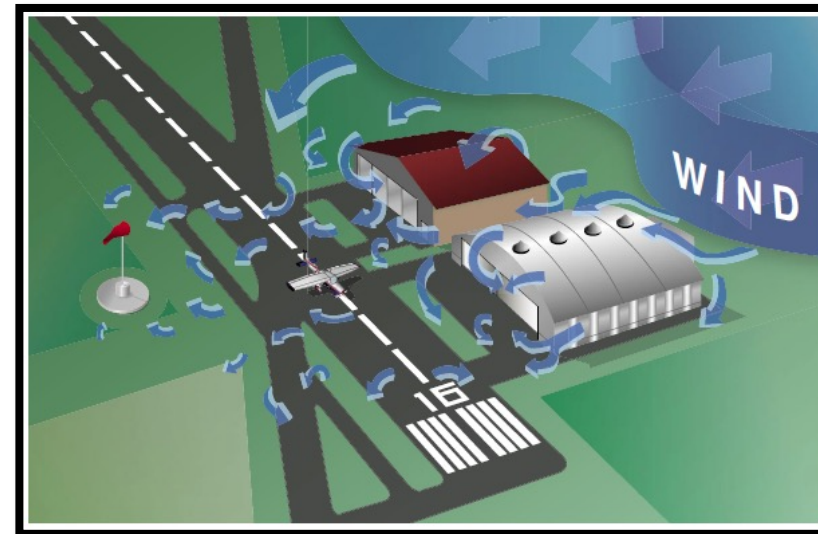
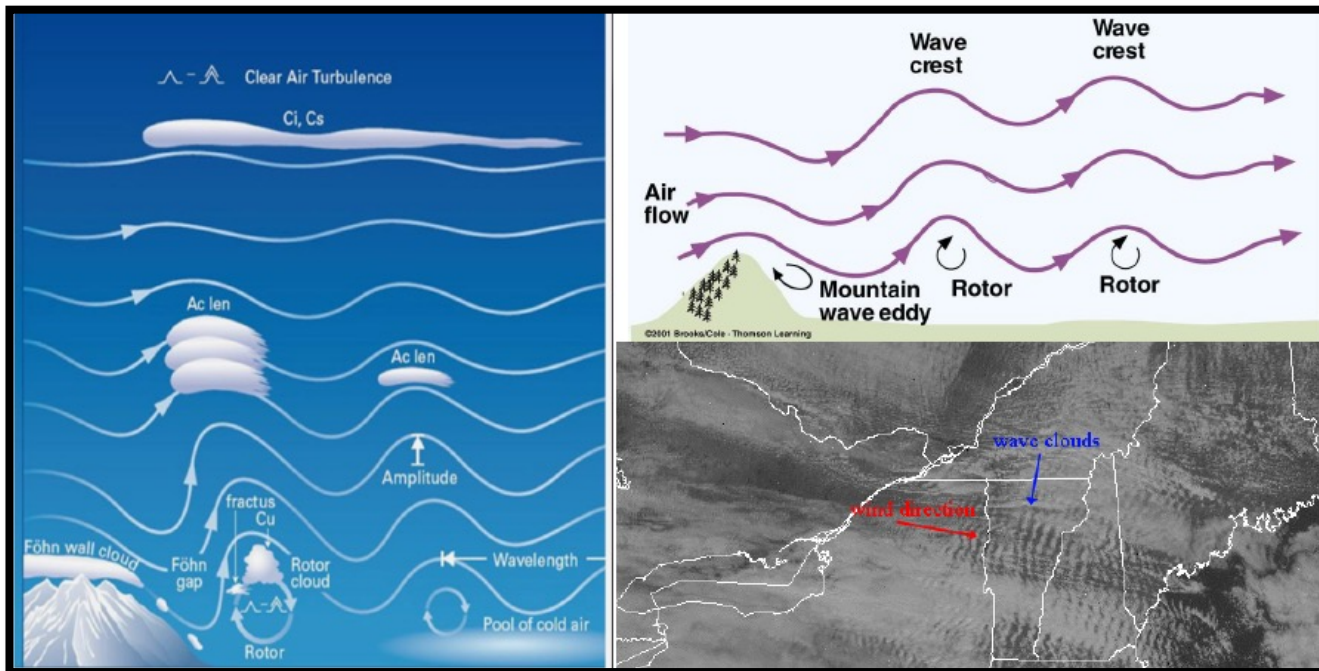
## PINN for Inverse Problems: Flow Over an Espresso Cup



Temperature velocity pressure  
Tomo-BOS (层析背景纹影) 实验, 六个相机, 可重构三维的温度场。  
PINNs以温度场作为训练数据, 计算出三维+时间变化的速度和压力。

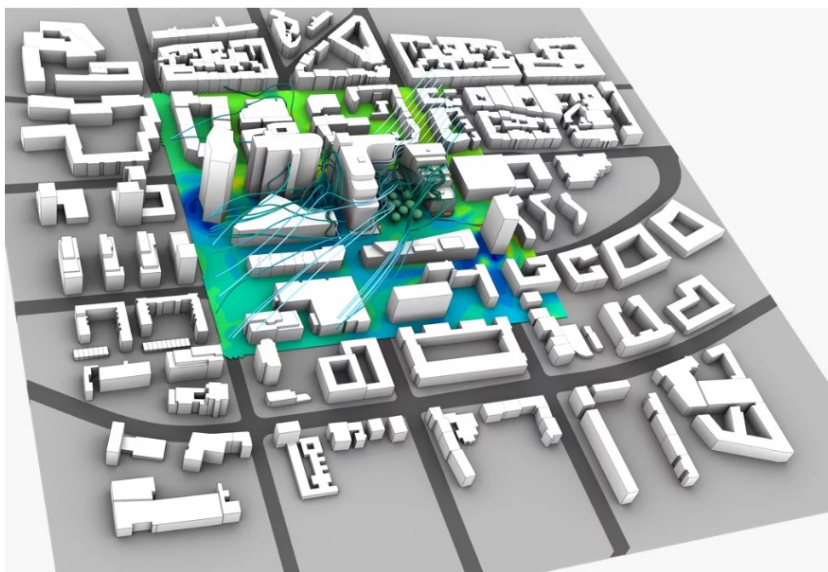


# PINN applications – Super resolution

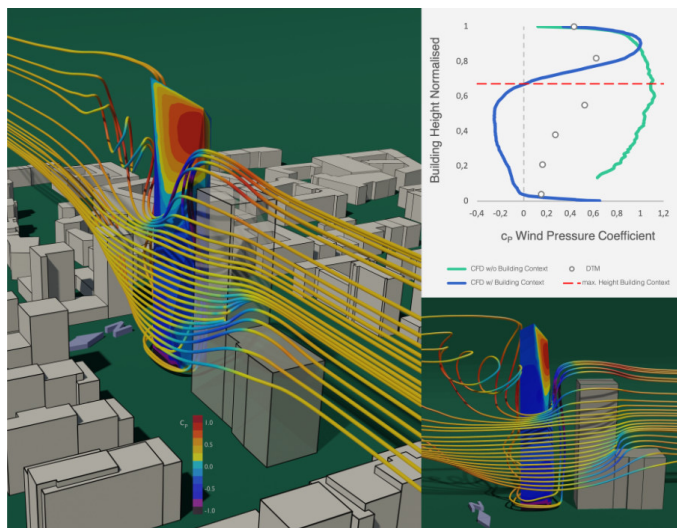
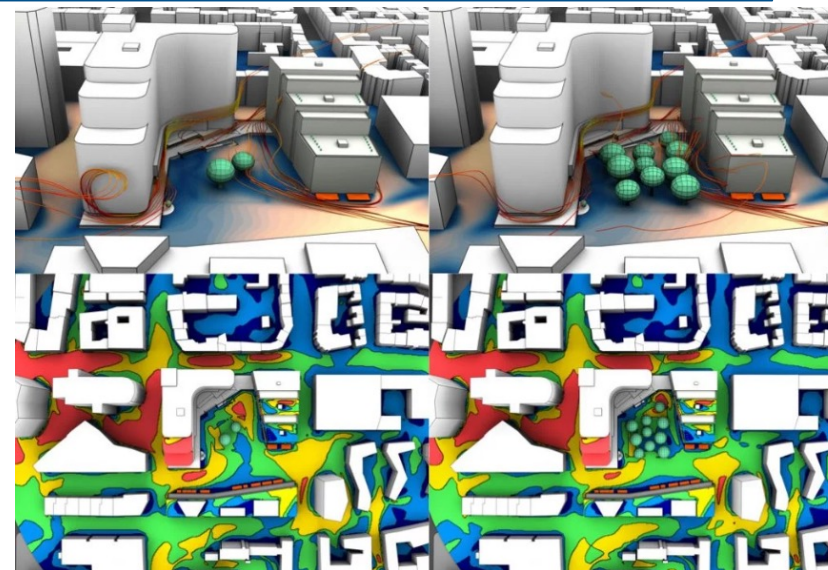




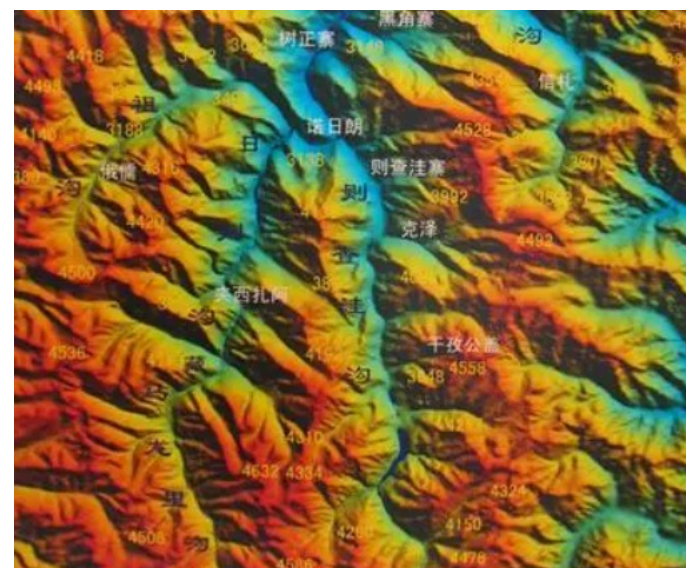
# PINN applications – Super resolution



街区风舒适管理



高层建筑风  
压快速预测

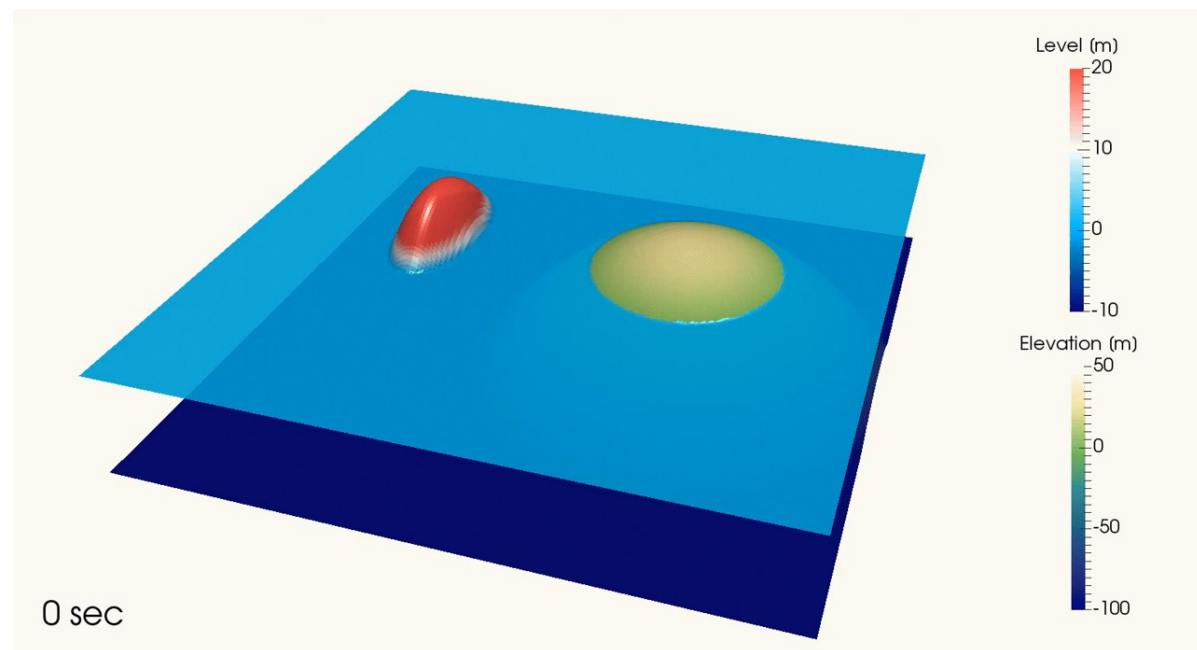


风力发电机  
微观选址

PI\_DeepONet求解浅水方程

## Shallow Water Equation

- 海洋环流与近岸波浪
- 风暴潮与气象海洋学
- 潮汐与潮流预测
- 海啸模拟
- 河流和洪水模拟





# Data-Physics Dual-Driven Model

## 一维SWE

浅水方程（SWE）是描述浅水流动的数学模型，它是一组描述自由表面流体流动的非线性偏微分方程，常用于模拟河流、湖泊、海洋表面波等场景下的水流运动。

$$\begin{cases} \frac{\partial}{\partial t} h + \frac{\partial}{\partial x} h v = 0 & (1) \\ \frac{\partial}{\partial t} h v + \frac{\partial}{\partial x} \left( h v^2 + \frac{g h^2}{2} \right) = 0 & (2) \end{cases}$$

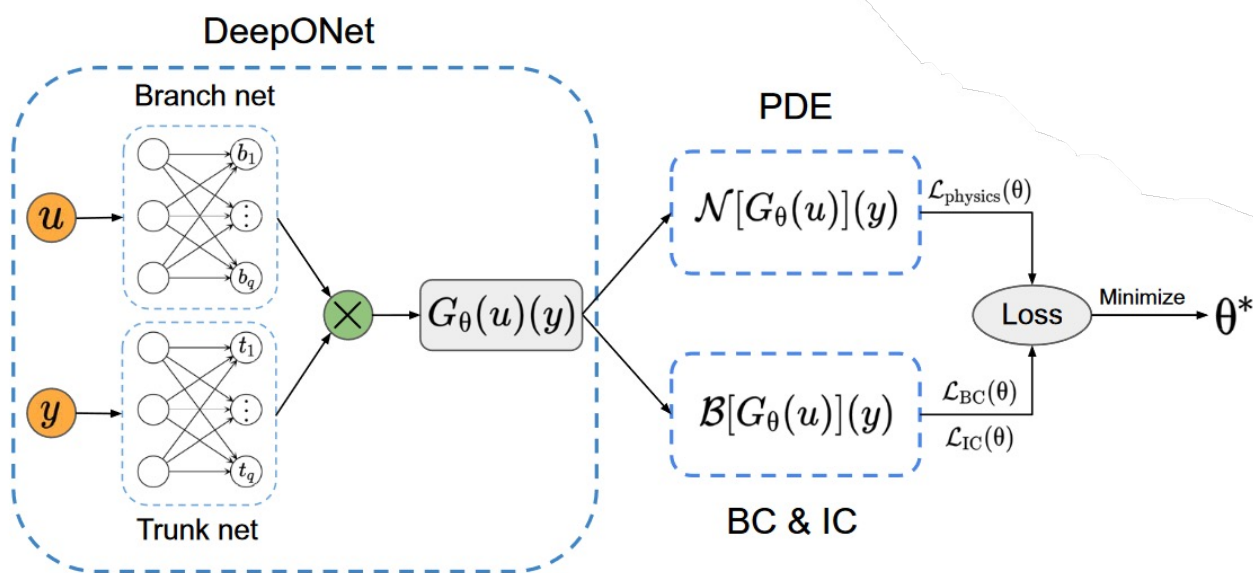
$h(x,t)$  为水在时间  $t$ 、位置  $x$  的深（高）度； $v(x,t)$  为在时间  $t$ 、位置  $x$  上水体的流速，向右为正方向。

初始条件：  $h(x, 0) = u(x)$

边界条件：  $\begin{cases} v(0, t) = 0 \\ v(1, t) = 0 \end{cases}$

# Data-Physics Dual-Driven Model

## 网络框架



图|在DeepONet中加入物理约束

总损失函数:  $\mathcal{L}(\theta) = \mathcal{L}_{\text{operator}}(\theta) + \mathcal{L}_{\text{physics}}(\theta)$

边界/初始条件损失函数

$$\mathcal{L}_{\text{operator}}(\theta) = \frac{1}{NP} \sum_{i=1}^N \sum_{j=1}^P |G_{\theta}(\mathbf{u}^{(i)})(\mathbf{y}_{u,j}^{(i)}) - G(\mathbf{u}^{(i)})(\mathbf{y}_{u,j}^{(i)})|^2$$

物理约束损失函数

$$\mathcal{L}_{\text{physics}}(\theta) = \frac{1}{NQm} \sum_{i=1}^N \sum_{j=1}^Q \sum_{k=1}^m | \mathcal{N}(u^{(i)}(\mathbf{x}_k), G_{\theta}(\mathbf{u}^{(i)})(\mathbf{y}_{r,j}^{(i)})) |^2$$

PI-DeepONet用于解决偏微分方程（PDEs）的数值求解问题。它结合了两类神经网络模型：Deep Operator Network (DeepONet) 和物理信息神经网络 (Physics-Informed Neural Networks, PINNs)。其中，使用 DeepONet 的结构来捕捉输入和输出之间的复杂关系；利用 PINN 的思想，通过在损失函数中加入PDE的残差项来约束网络的学习过程，从而更好地逼近真实的物理行为。

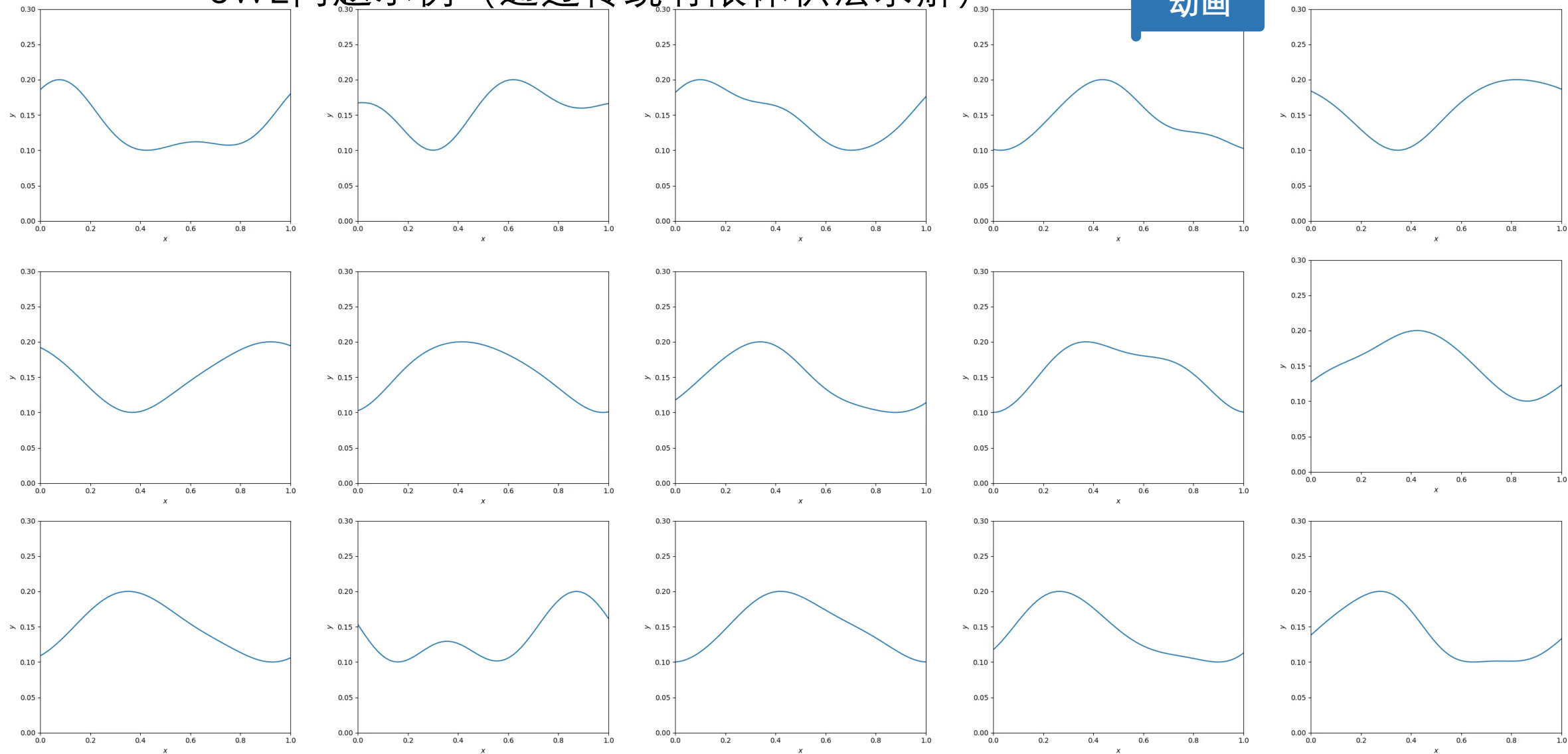




# Data-Physics Dual-Driven Model

SWE问题示例（通过传统有限体积法求解）

动画

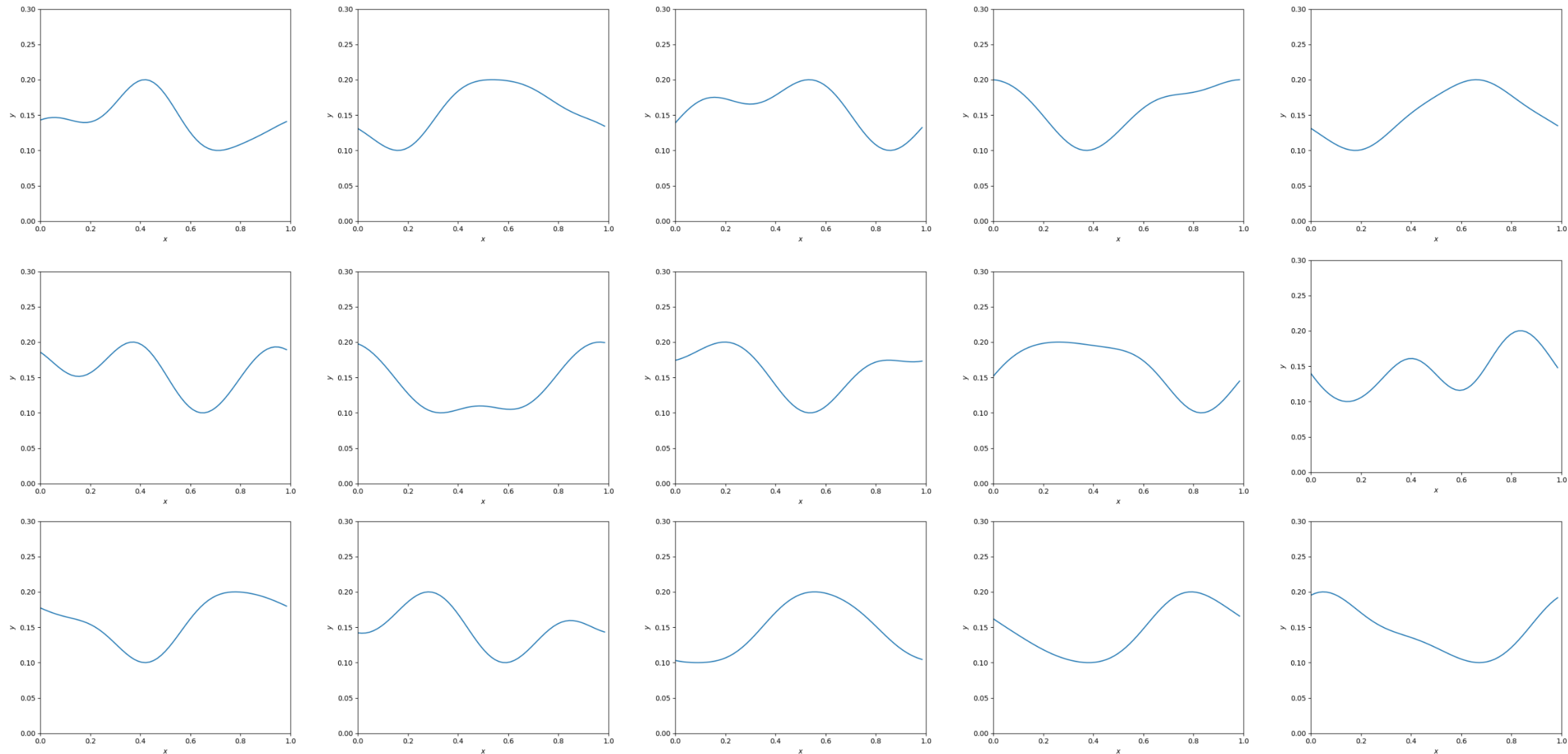




# Data-Physics Dual-Driven Model

只需2000个初始条件，如图静态（无需知道真实解的时间序列）

## PI\_DeepONet的训练数据





# Data-Physics Dual-Driven Model

## PI\_DeepONet训练细节

模型  
设置

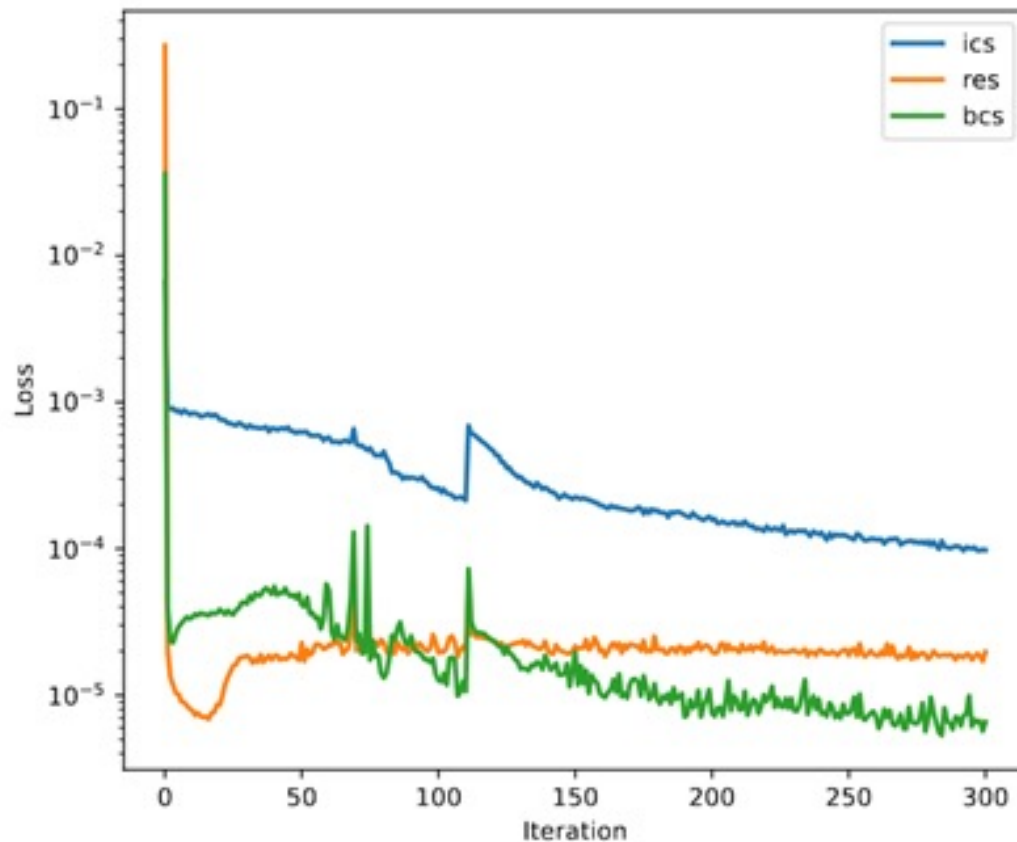
```
m = 64 # number of sensors for input samples
P_ics_train = 64 # number of locations for evaluating the initial condition
P_bcs_train = 100 # number of locations for evaluating the boundary condition
P_res_train = 2500 # number of locations for evaluating the PDE residual
```

```
# deeponet
branch_size = [m, 100, 100, 100, 100, 100]
trunk_size = [2, 100, 100, 100, 100, 100, 0]
activation='tanh'
# training
lr = 0.001
iterations = 30000
```

分辨率

```
Nx = 64
Nt = 100
```

```
x = np.linspace(0, 1, num=Nx)
t = np.linspace(0, 1, num=Nt)
```

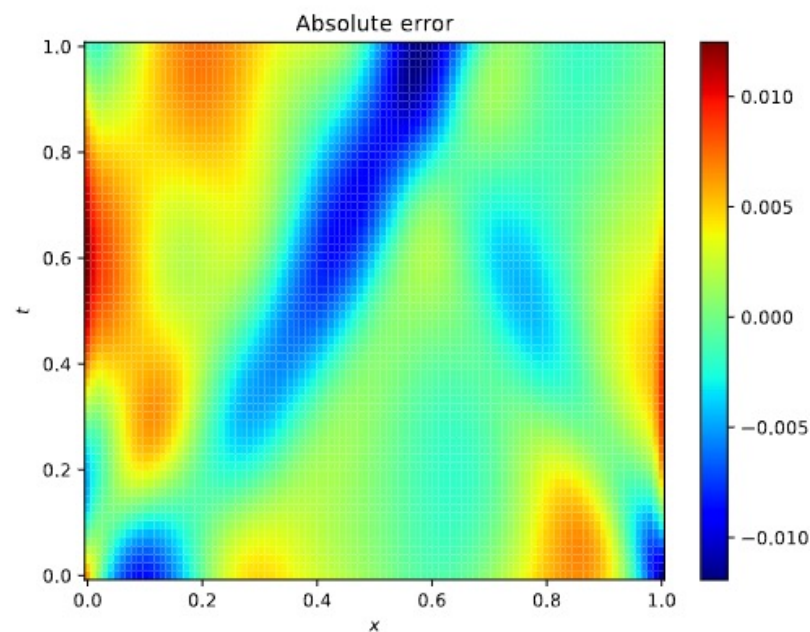
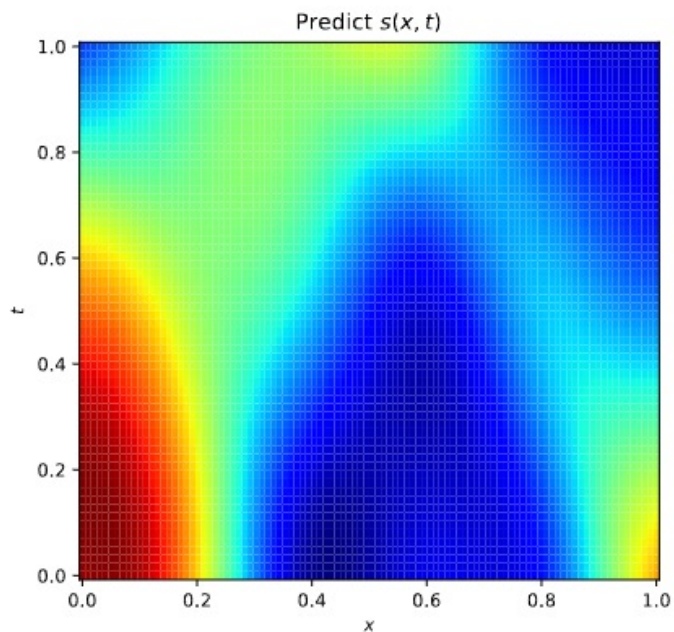
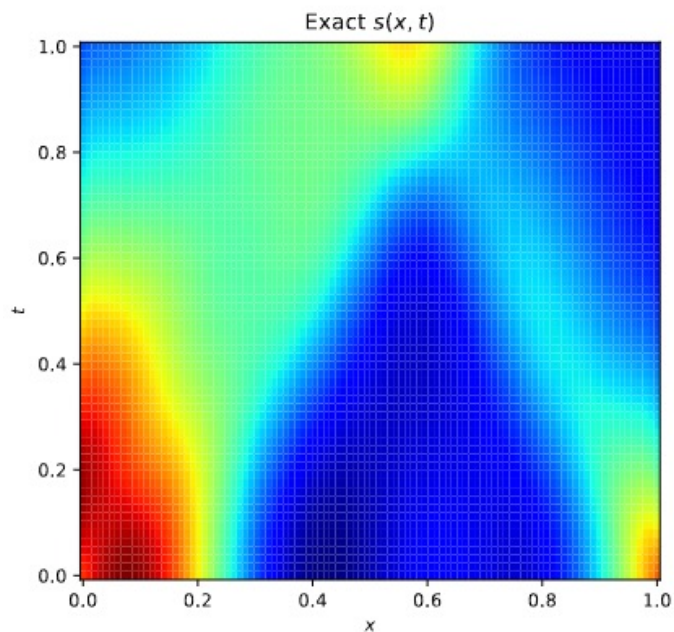
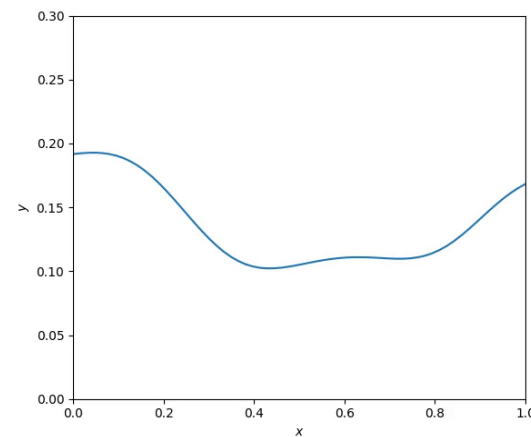
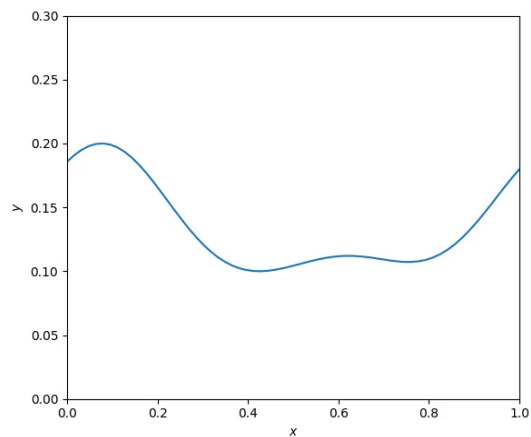




# Data-Physics Dual-Driven Model

PI\_DeepONet训练结果

Example 1 (unseen data)



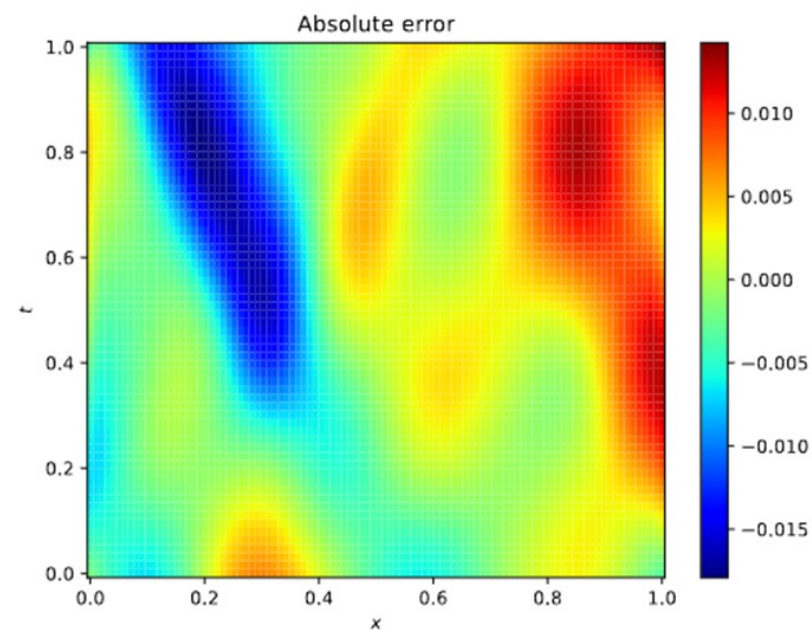
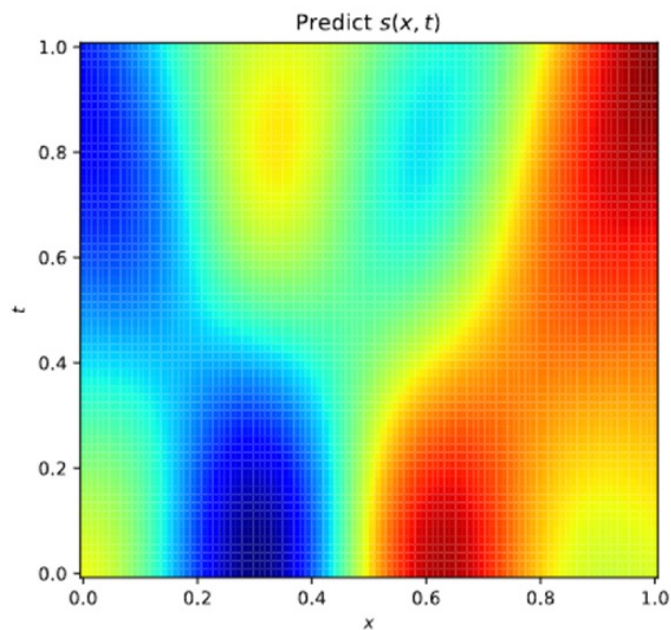
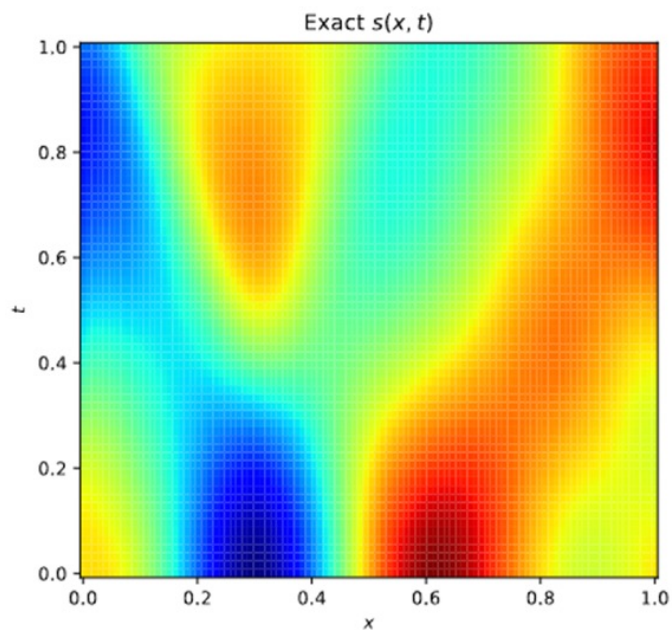
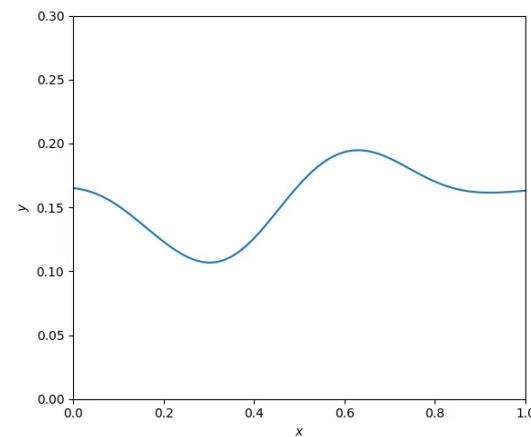
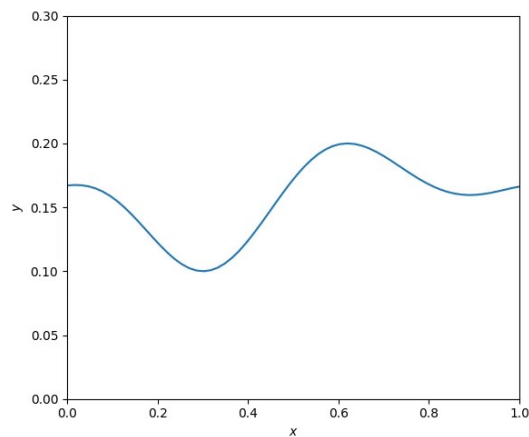




# Data-Physics Dual-Driven Model

PI\_DeepONet训练结果

Example 2 (unseen data)

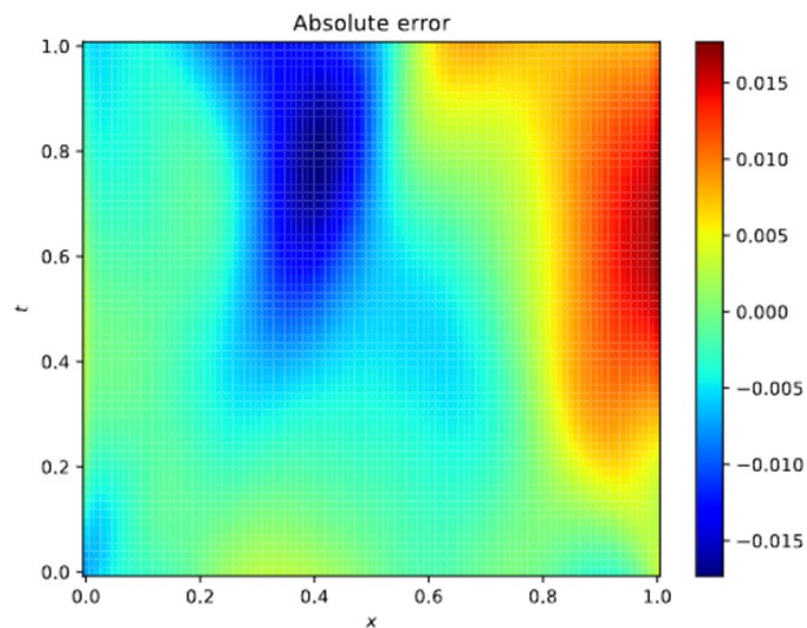
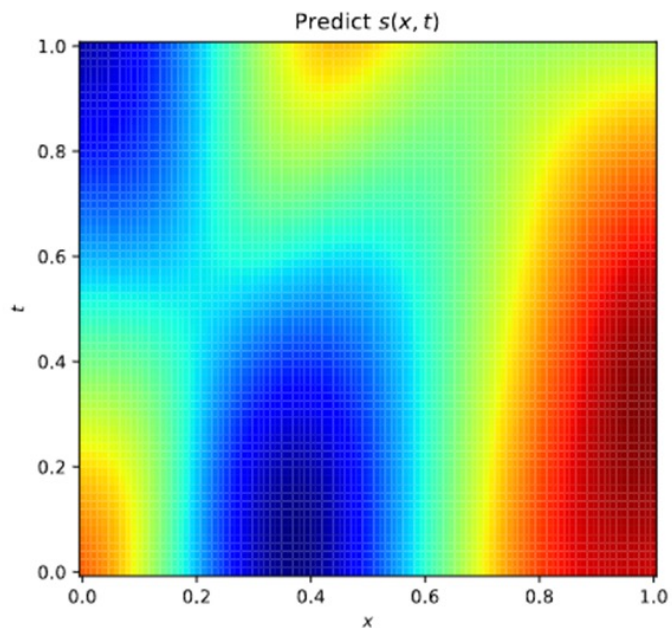
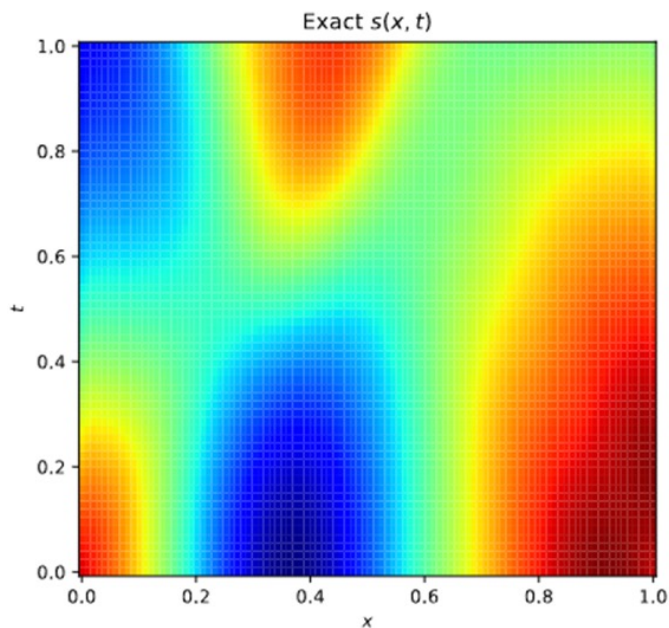
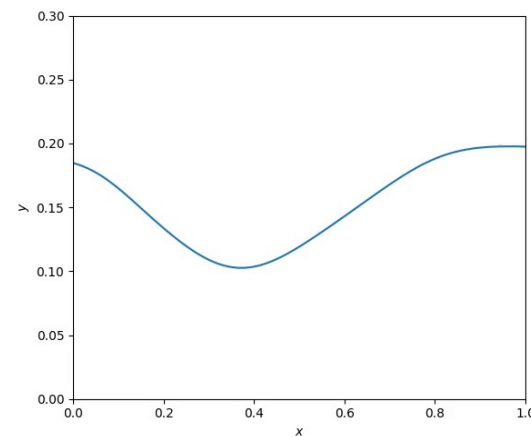
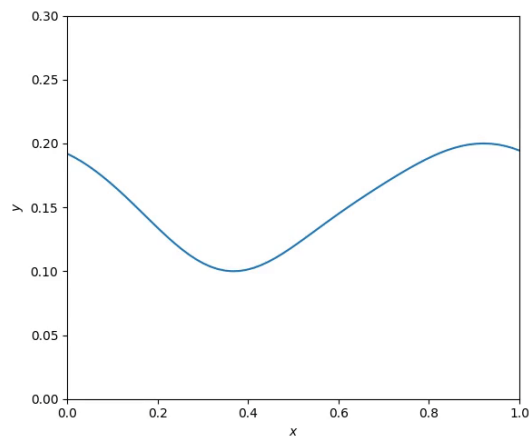




# Data-Physics Dual-Driven Model

PI\_DeepONet训练结果

Example 3 (unseen data)





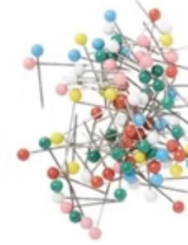
# PINN family



## An Alphabet of **PINNs**:

(Paper in Nature Physics Reviews, June 2021)

- **cPINNs**: conservative PINNs
- **vPINNs**: variational PINNs
- **pPINNs**: parareal PINNs
- **sPINNs**: stochastic PINNs
- **fPINNs**: fractional PINNs
- **BPINNs**: Bayesian PINNs
- **xPINNs**: eXtended PINNs





THANK YOU



# Multiscale problem

## Background – Multiscale Nature of Bubble Dynamics

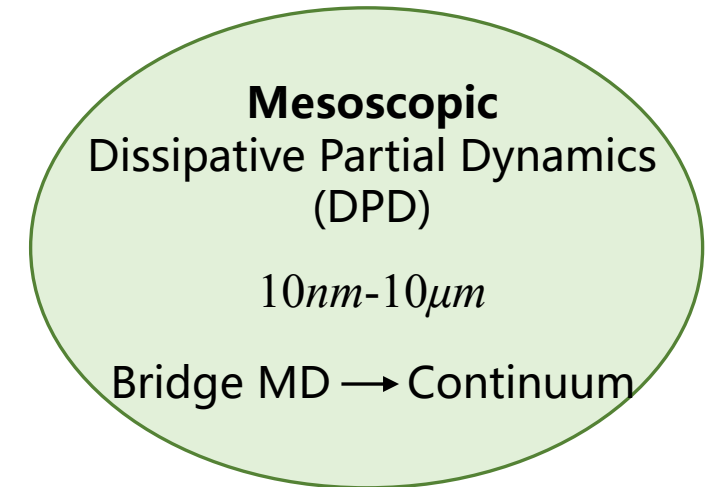
- Mesoscale connects molecular scale dynamics to continuum, mean-field models
- Retain thermal (Brownian) fluctuations. Effects of large-scale molecular fluctuations relevant to observations
- Use standard DPD for “gas” bubble phase
- Use multi-body DPD (mDPD) for liquid phase – allows for both short-range repulsion and longer range attraction needed for surface tension.

### The usual strategy – two steps:

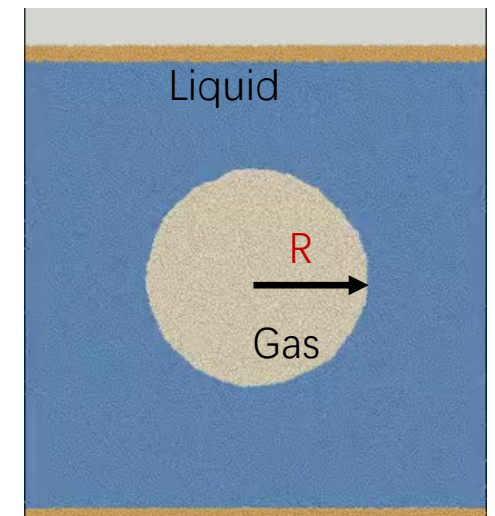
- Derive continuum level PDEs and ODEs of phenomena – Physical model has approximations and assumptions
- Solve the PDEs numerically for relevant domain – Errors from numerical scheme

### Operator Learning - one step:

- Use DeepONet (NN) to **directly** learn the mesoscale dynamics and provide the mean/continuum level result for  $\langle R \rangle(t)$ .
- Can we accommodate different system parameters, such as  $R_0$ , over a range of scales?
- Can we quantify the mesoscale fluctuations?



Gas bubbles in a liquid often sites for heterogeneous nucleation in cavitation or boiling



## Physical system – Fluctuating Bubble Growth in a Liquid

Step 1: Use DeepONet to learn the continuum level dynamics linking  $P_\infty(t) \rightarrow R(t)$

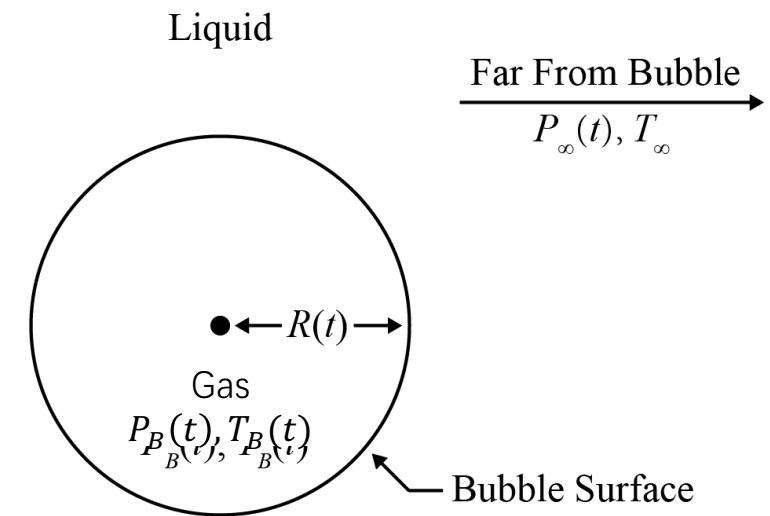
Classical continuum model: Rayleigh-Plesset (R-P) equation in 3D for spherical gas bubble

$$P_B(t) - P_\infty(t) = \rho_L \left\{ R \frac{d^2 R}{dt^2} + \frac{3}{2} \left( \frac{dR}{dt} \right)^2 + \frac{4\nu_L}{R} \frac{dR}{dt} \right\} + \frac{2\gamma}{R}$$

$$P_B(t) = P_{G0} \left( R_0 / R \right)^3 \quad \Delta P = P_\infty(t) - P_\infty(0)$$

Physical assumptions for R-P, not really valid for DPD simulations :

- Incompressible, viscous liquid: density,  $\rho_L$ ; viscosity,  $\nu_L$
- Isolated compressible gas bubble with uniform internal pressure  $P_B$ , negligible inertia
- Spherical bubble, radius  $R(t)$  generates a potential flow as it expands/contracts
- Specify far field pressure  $P_\infty(t)$
- Effects of fluid inertia and added-mass as bubble expands plus viscous damping and surface tension,  $\gamma$



No vapor, isothermal conditions  $T_B = T_\infty$ ,  $k = 1$ .

$$P_B(t) = P_V(T_B) + P_{G0} \left( \frac{T_B}{T_\infty} \right) \left( \frac{R_0}{R} \right)^{3k}$$

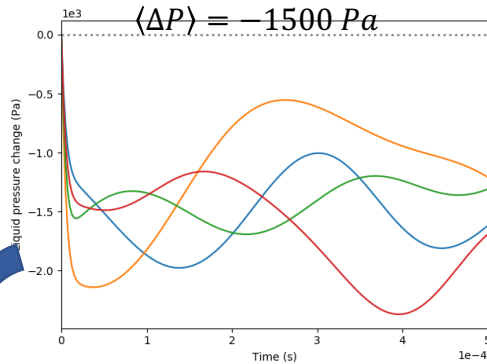


# Multiscale problem

## Outline of DeepONet for learning R-P operators

(INPUT SPACE)

$\Delta P(t)$

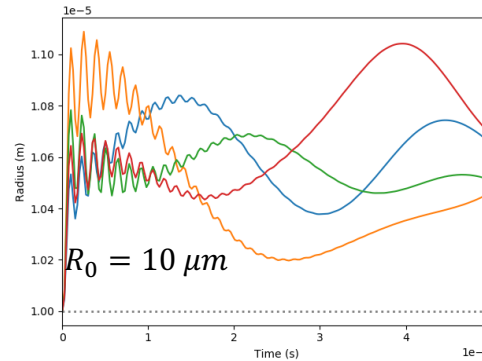


R-P  
equation



(OUTPUT SPACE)

$R(t)$



$t = 0 - 500 \mu sec$

$\Delta P$

$\Delta p(t_1)$   
 $\Delta p(t_2)$   
...  
 $\Delta p(t_m)$

Branch net

$b_1$   
 $b_2$   
...  
 $b_p$

$t$

Trunk net

$a_1$   
 $a_2$   
...  
 $a_p$

$\times$

$$R(t) = G(\Delta P)(t) = \sum_{k=1}^p b_k a_k$$

Output times  $t$

Lu, Lu, et al. "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators." *Nature Machine Intelligence* 3.3 (2021): 218-229.

Define the input space

The pressure trajectories are generated by Gaussian Random Function (GRF) with a mask starting from equilibrium pressure

$$g(t) \sim \mathcal{GP}(\mu, \sigma k_{IT}(t_1, t_2)),$$

$$s(t) = \begin{cases} \left(\frac{t}{qT}\right)^{10} & 0 \leq t < qT, \\ 1 & qT \leq t \leq T, \end{cases}$$

$$\begin{aligned} \mu &= 1500 Pa, \sigma = 400 Pa \\ T &= 0.0005 s, \\ q &= 0.1, l = 0.2 \end{aligned}$$

$$\Delta P(t) = g(t)s(t).$$

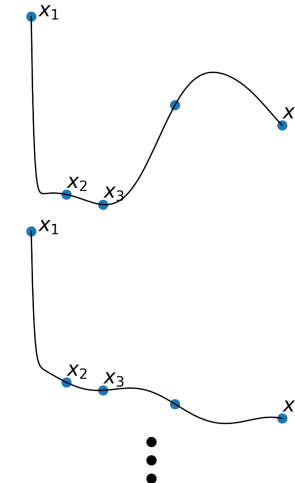
Data sensor

$\Delta P(t)$

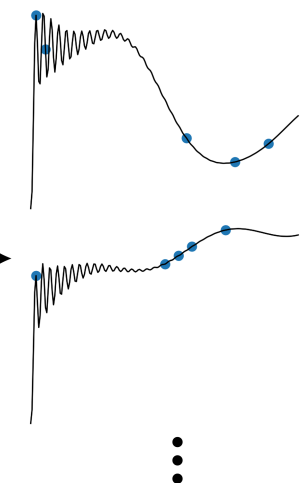
$R(t)$

Sensors on a  
uniform grid

Output at  
arbitrary locations



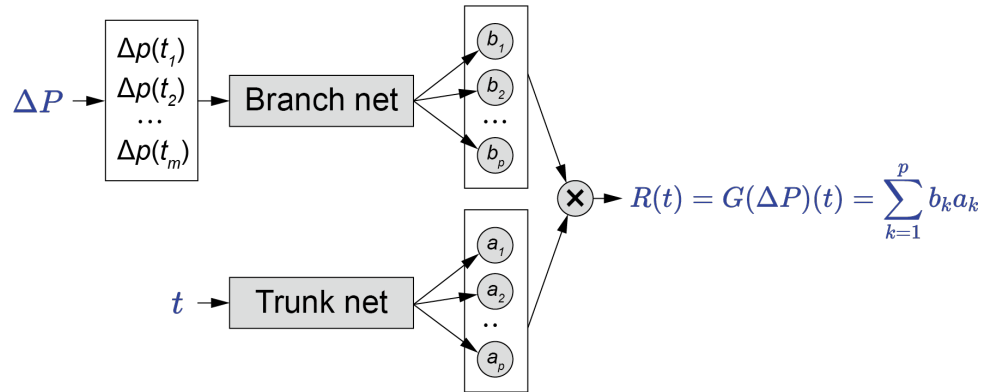
$G$





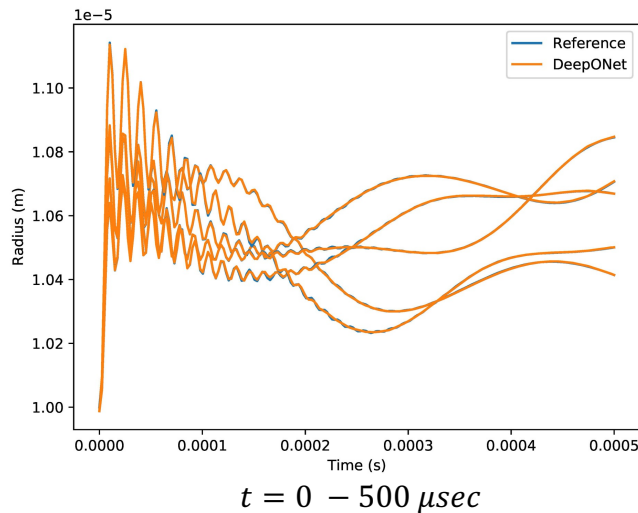


# Multiscale problem



Trajectory comparison  
of Reference and Prediction

Bubble radius  $R(t)$   
 $10 - 12 \mu m$



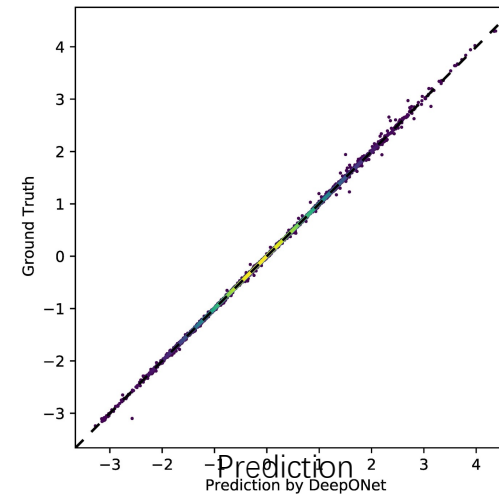
## Outline of DeepONet for learning R-P operators

Setup of DeepONet

|                                         |                                       |                                     |
|-----------------------------------------|---------------------------------------|-------------------------------------|
| Training + Test datasets<br>3000 + 1000 | # pressure sensors ( <b>m</b> )<br>50 | # radius sensors ( <b>p</b> )<br>20 |
| # Train data points<br>60000            | # Test data points<br>20000           | Branch depth<br>3                   |
| Branch width<br>300                     | Trunk depth<br>3                      | Trunk width<br>300                  |
| Activation function<br>ReLU             | Learning rate<br>0.0005               | # Epoch<br>150000                   |

Parity (scatter) plot of Prediction and Reference

Ground  
truth



Data points falls on the diagonal  
line indicate the perfect  
accuracy

- Lu, Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators." *Nature machine intelligence* 3, no. 3 (2021): 218-229.



# Multiscale problem

## Comparison of DeepONet and LSTM

- 5000 training trajectories for both LSTM and DeepONet
- Test different number of sensors on each trajectory
- DeepONet has huge advantage when it's not possible to collect dense data or time scales are unknown

Sparser training data

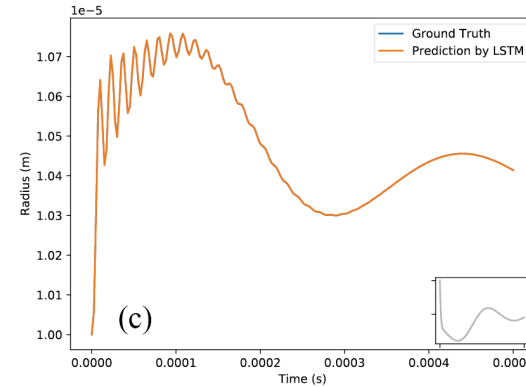
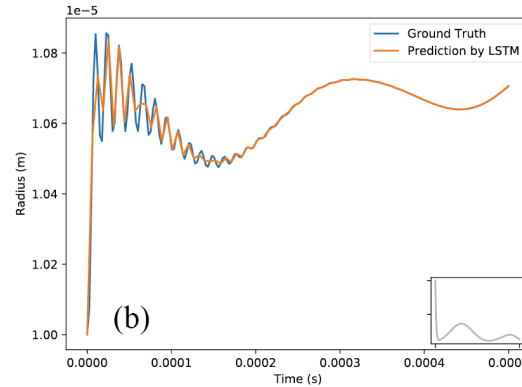
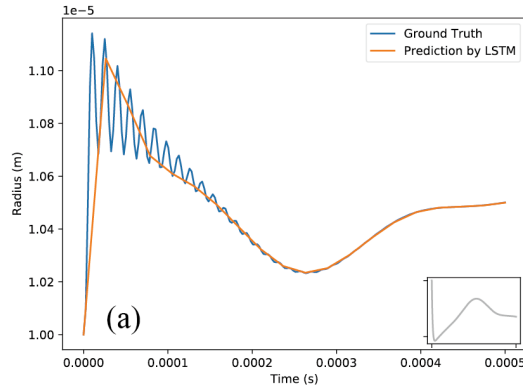
Denser training data

20 points / trajectory

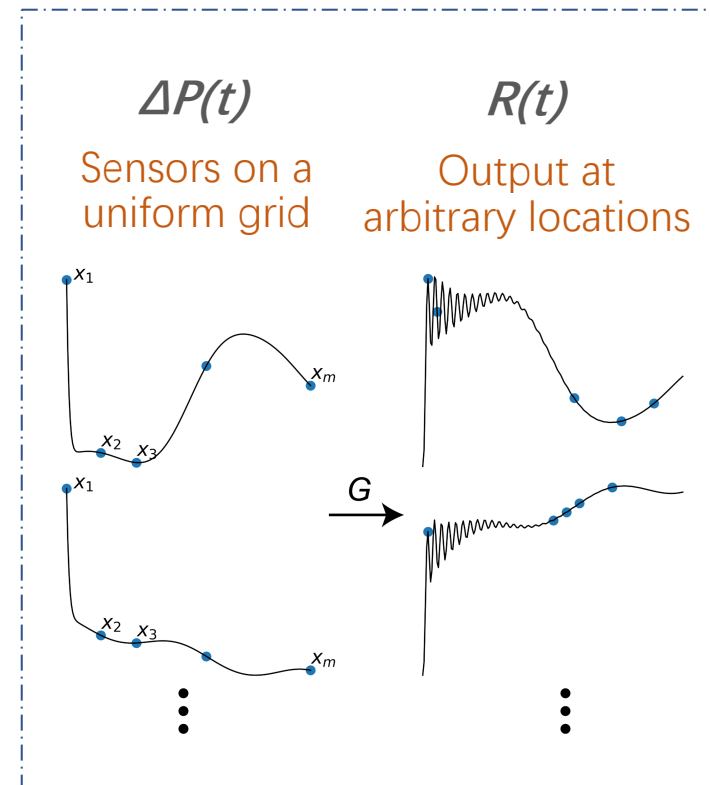
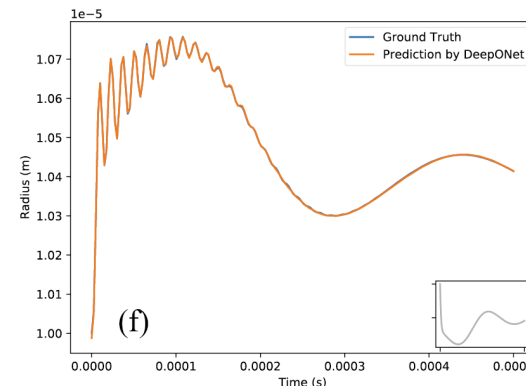
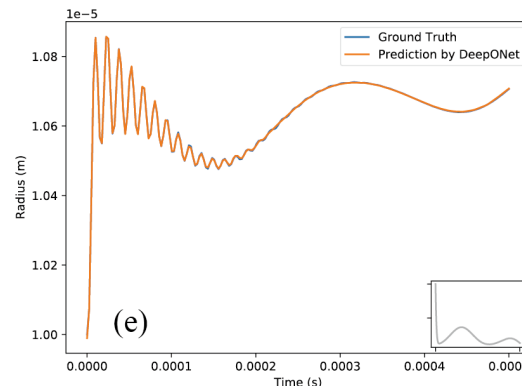
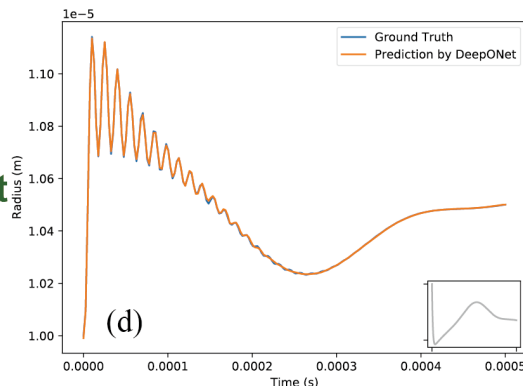
80 points / trajectory

200 points / trajectory

LSTM



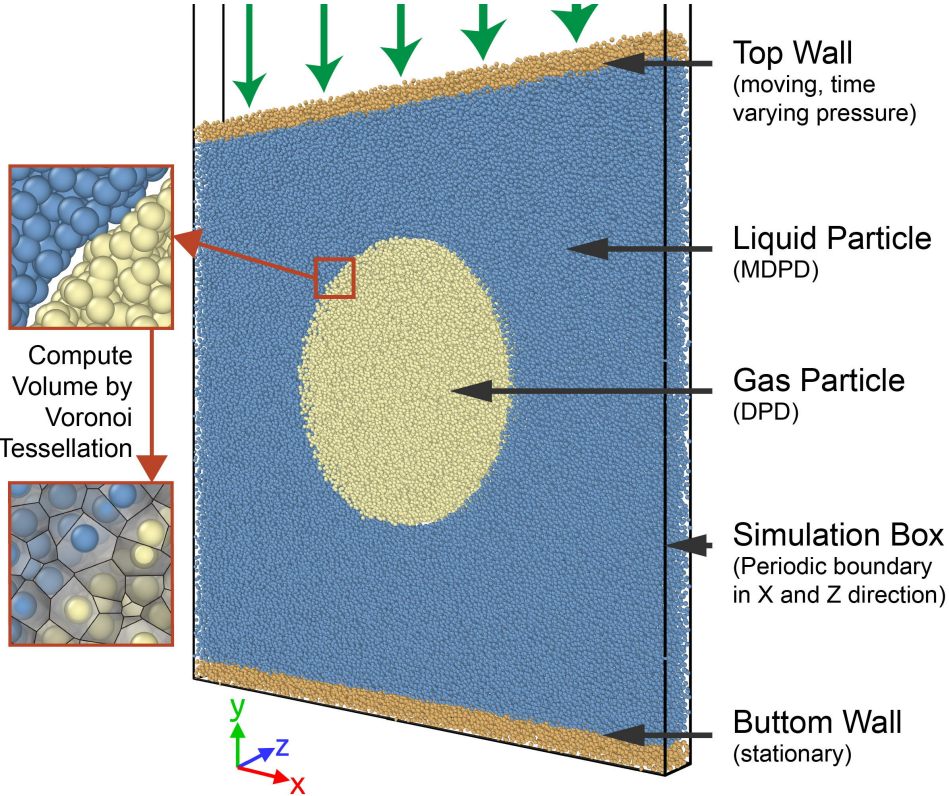
DeepONet





# Multiscale problem

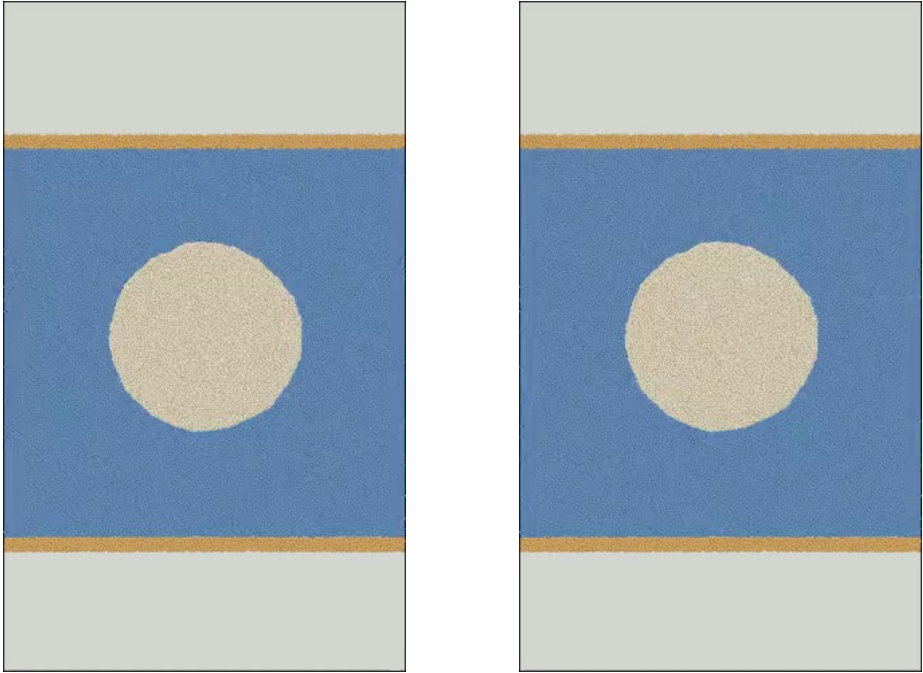
Specify mean wall pressure  $P_W(t)$ , observe bubble radius  $R(t)$  and mean radius  $\langle R \rangle(t)$



Quasi-2D simulations: mDPD (liquid) and DPD (gas)

## Mesoscale simulations with mDPD and DPD for bubbles as ground truth

|                      |                   |                                                                  |
|----------------------|-------------------|------------------------------------------------------------------|
| Dimensions of domain | 60 X 60 DPD units | $5.0 \times 10^{-6} \text{m} \times 5.0 \times 10^{-6} \text{m}$ |
| Bubble radius        | 14 DPD units      | $1.2 \times 10^{-6} \text{m}$                                    |
| Simulation time      | 2000 DPD units    | $3.6 \times 10^{-6} \text{s}$                                    |



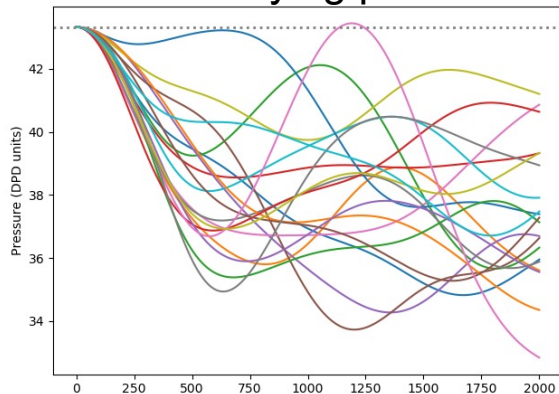
Videos 1 (not much drift) and 2 (some drift)



# Multiscale problem

Samples of time-varying pressure on top wall

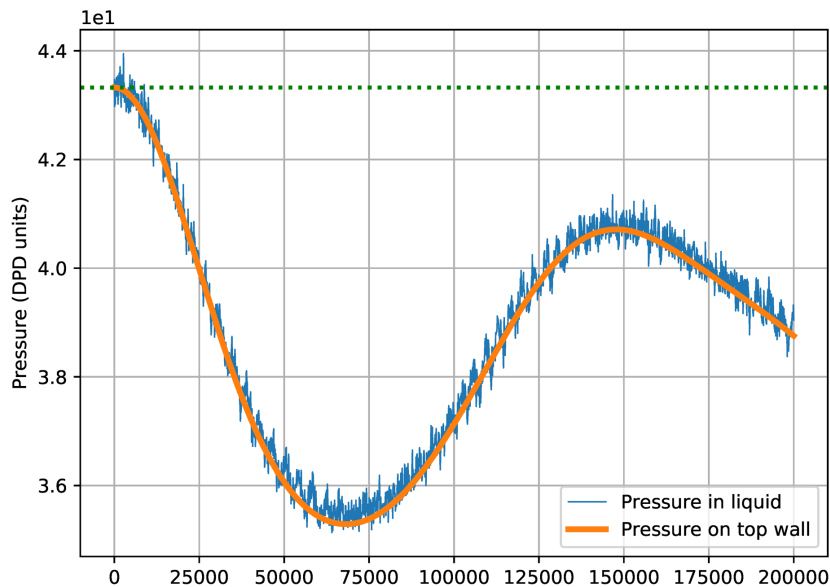
$P_W(t)$   
32 – 43 DPD units



$t = 0 - 2000$  DPD units

$P_W(t)$   
 $P_L(t)$

Example 1:



$t = 0 - 2000$  DPD units

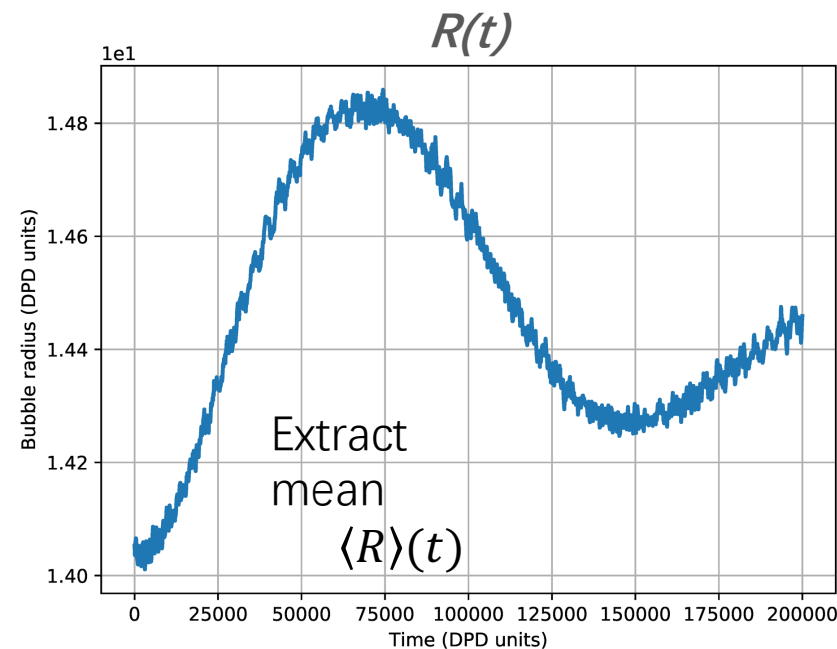
## DeepONet applied to mesoscale DPD simulation data: Learn $P_W(t) \rightarrow \langle R \rangle(t)$

Setup of DeepONet

|                              |                                      |                                    |
|------------------------------|--------------------------------------|------------------------------------|
| # Training + Test<br>500+100 | # pressure sensor ( <b>m</b> )<br>50 | # radius sensor ( <b>p</b> )<br>20 |
| # Train data<br>10000        | # Test data<br>2000                  | Branch depth<br>2                  |
| Branch width<br>200          | Trunk depth<br>2                     | Trunk width<br>200                 |
| Activation<br>ReLU           | Learning rate<br>0.0005              | # Epoch<br>50000                   |

Raw data

DPD  
simulation



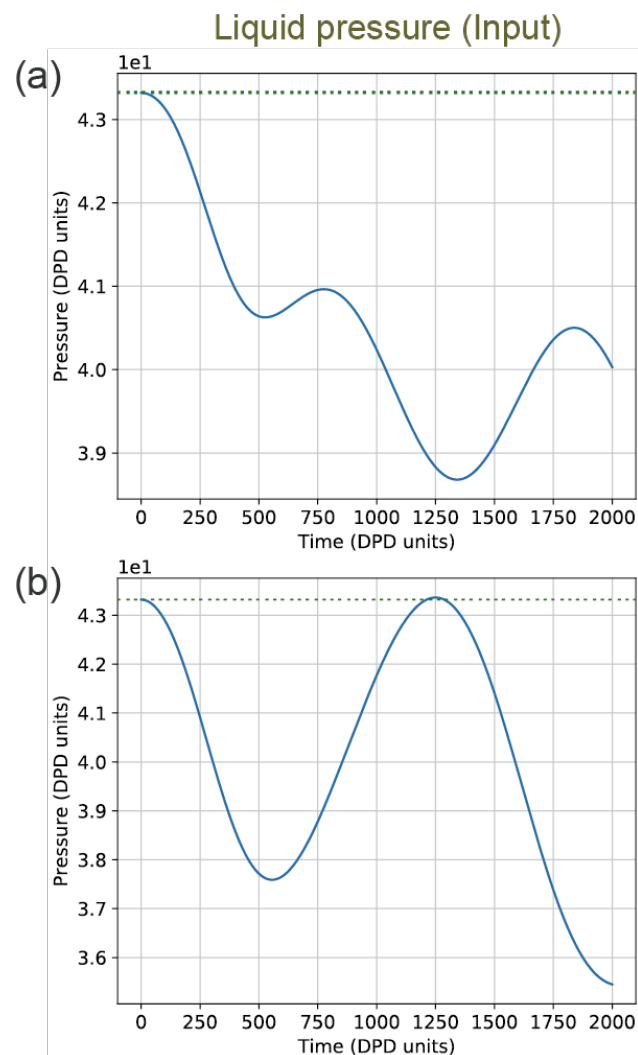


# Multiscale problem

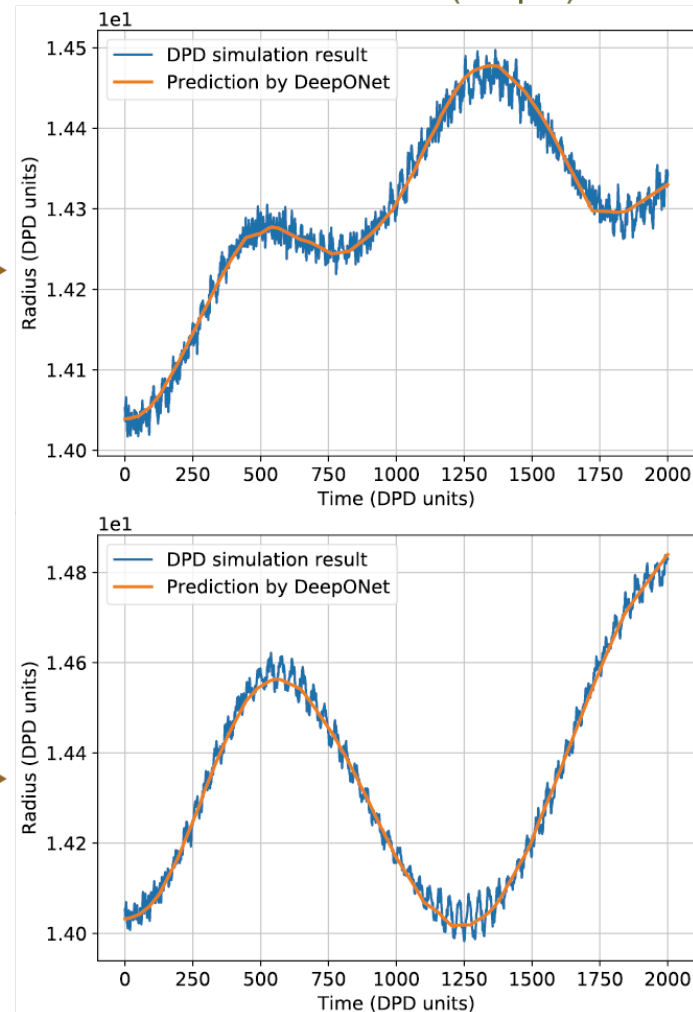
DeepONet applied to mesoscale  
DPD simulation data: Learn

$$P_W(t) \rightarrow \langle R \rangle(t)$$

Input  
 $P_W(t)$



Bubble radius (Output)

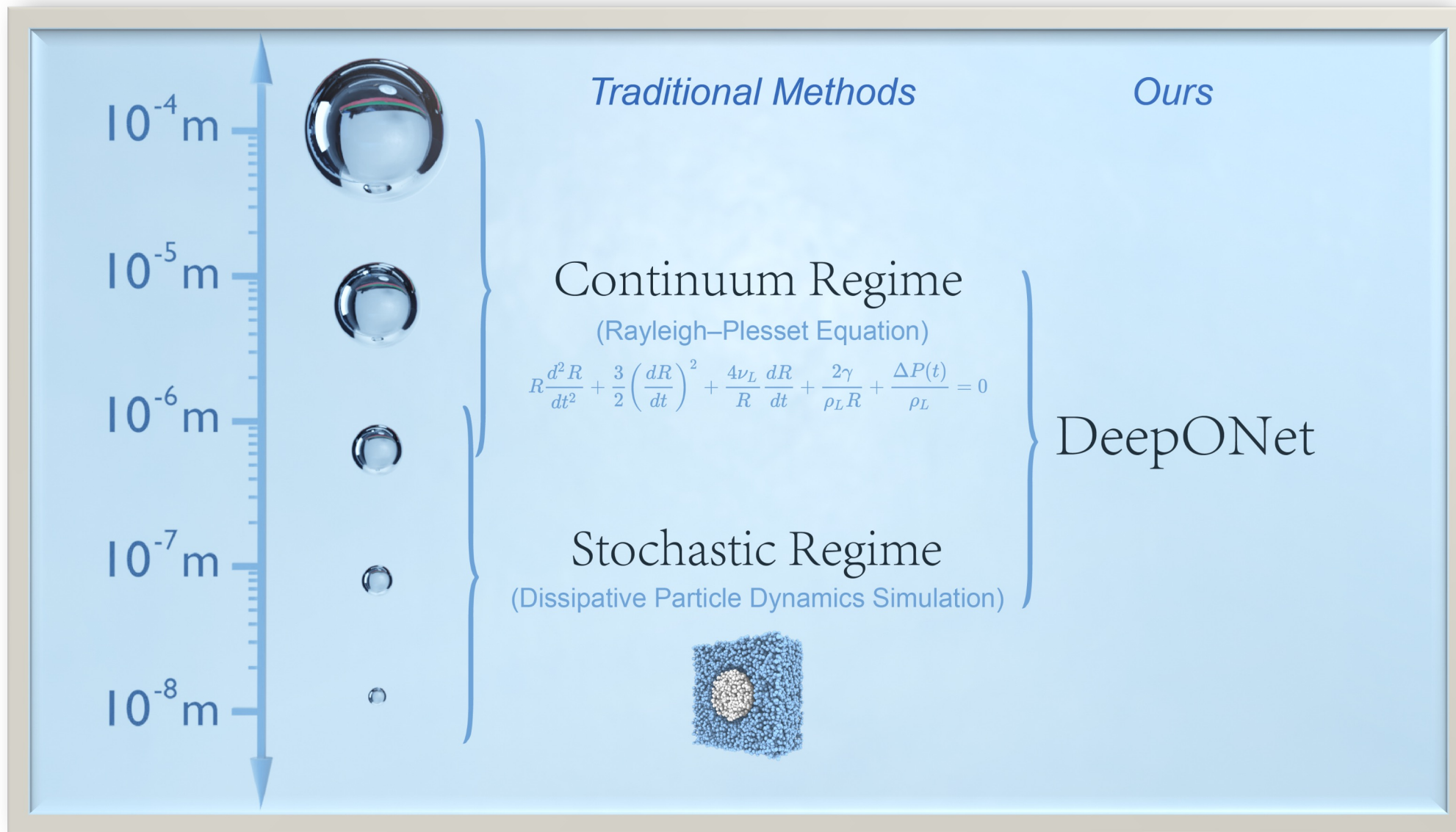


DPD results  
 $R(t)$

DeepONet  
Prediction for  
average  
 $\langle R \rangle(t)$



# Multiscale problem

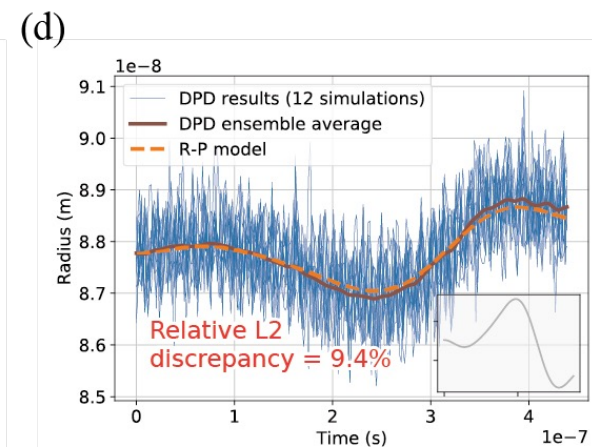
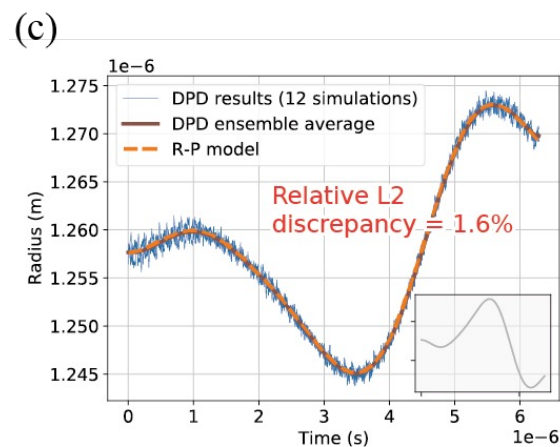
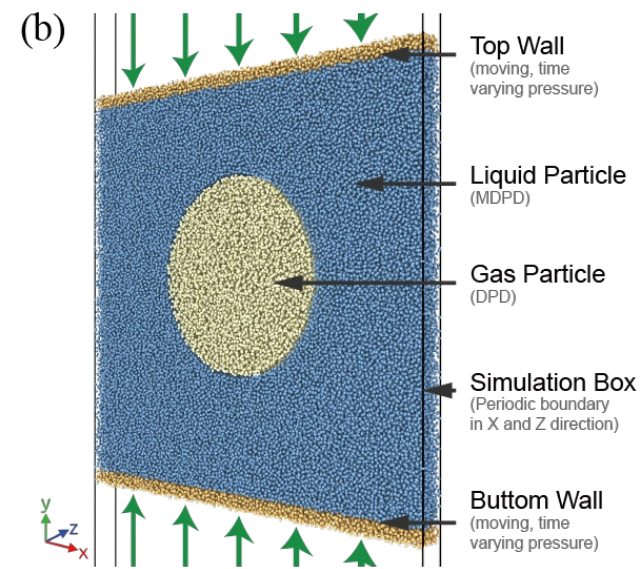
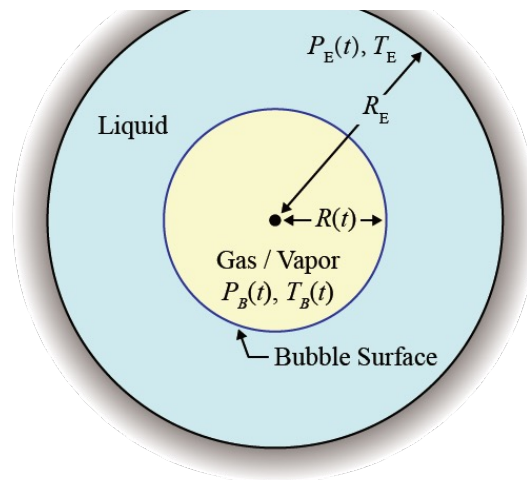




# Multiscale problem

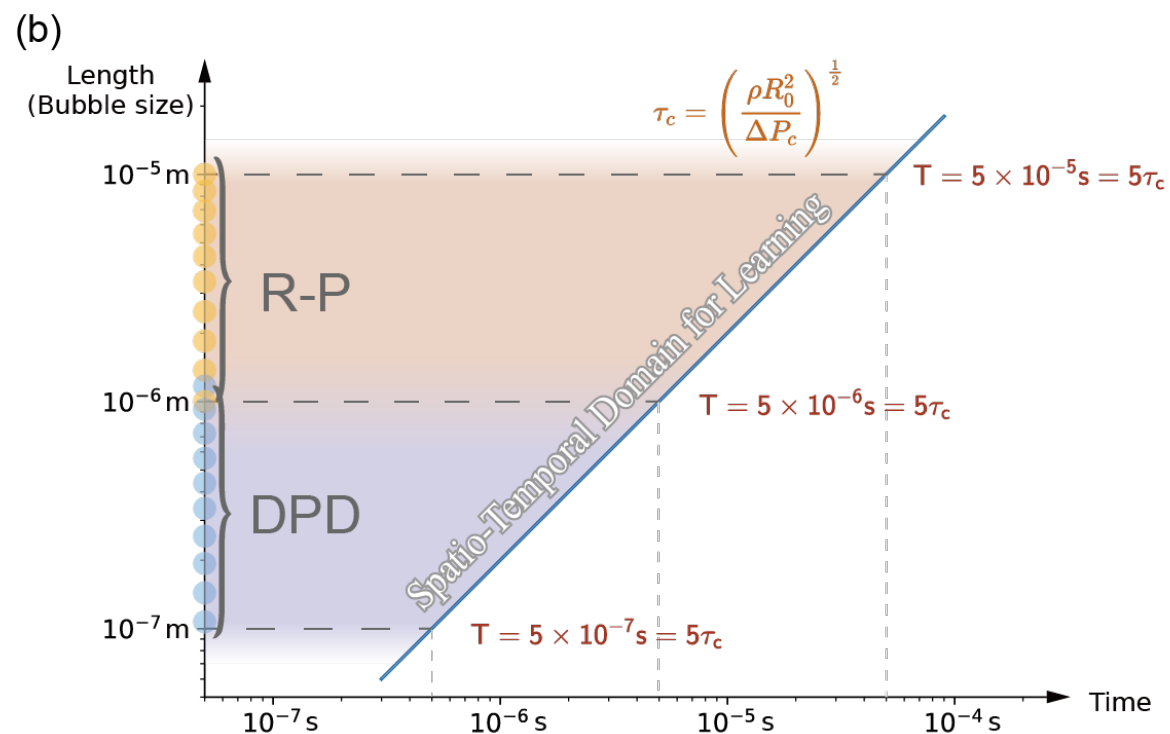
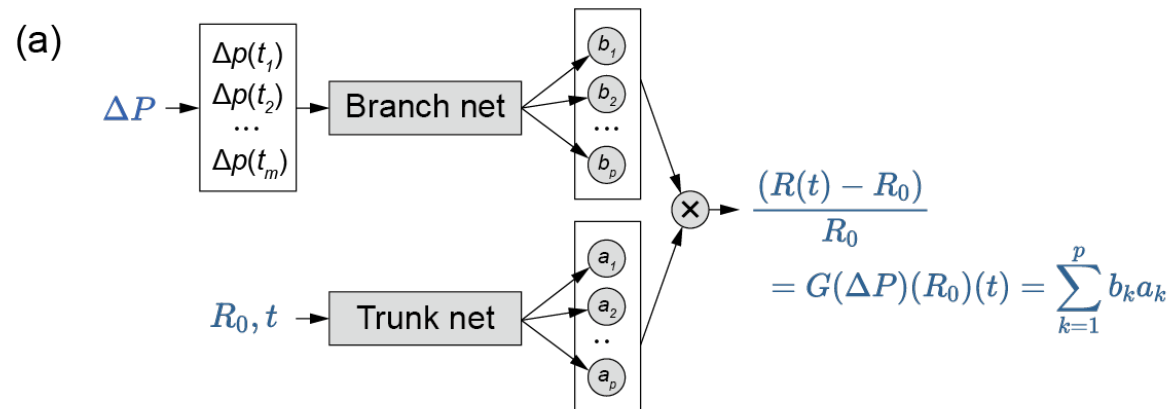
- Considering the high computational cost of DPD simulation, we consider a quasi-2D system in this proof of concept case.
- The R-P equation is a derived 2D version of classical R-P equation.
- At 1e-6m scale, the discrepancy of two models is 1.6%
- At 1e-7m scale, the discrepancy of two models increase to 10%.
- We set 1e-6m as the hand shaking scale.

$$P_B(t) - P(R_E, t) = \rho_L \log_e \left( \frac{R_E}{R} \right) \frac{d}{dt} \left( R \frac{dR}{dt} \right) - \frac{1}{2} \rho_L \left( 1 - \frac{R^2}{R_E^2} \right) \left( \frac{dR}{dt} \right)^2 + 2\mu_L \frac{1}{R} \frac{dR}{dt} + 2\mu_L \frac{1}{R} \frac{dR}{dt} + \frac{\gamma}{R}.$$



# Multiscale problem

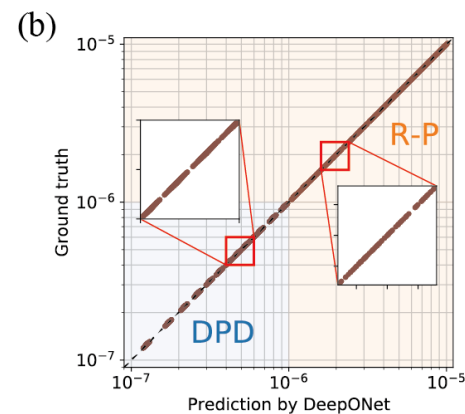
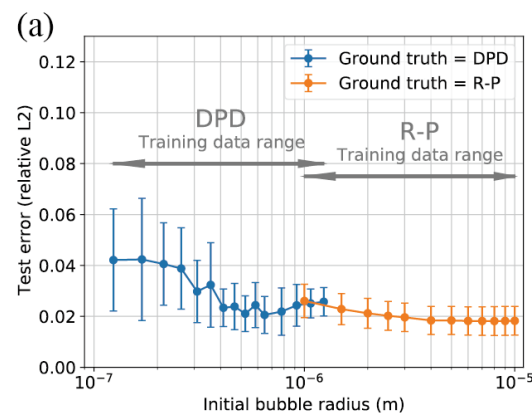
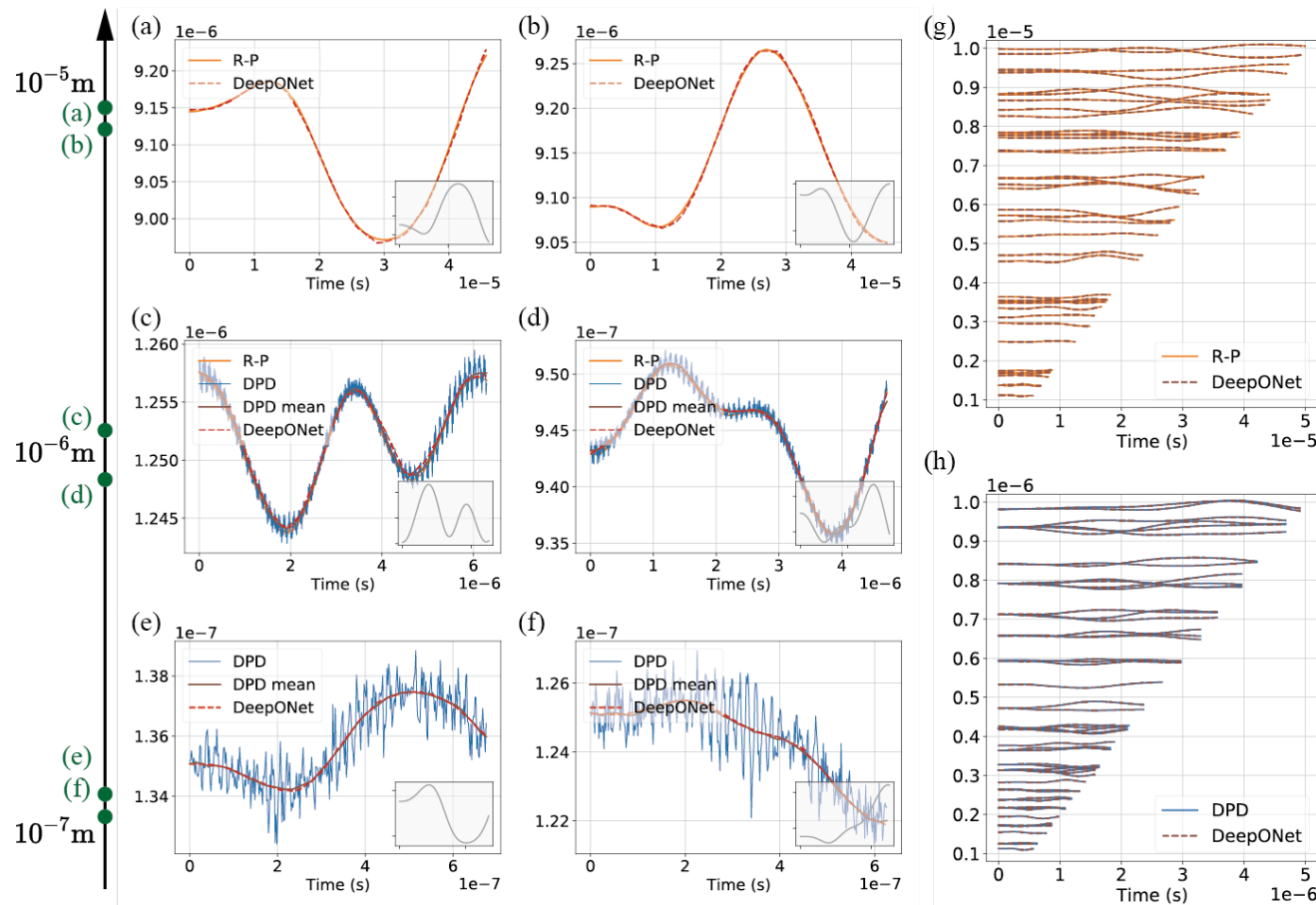
- We design the spatio-temporal domain for learning like this.
- The time scale is linked to bubble size. We use the concept of characteristic time (dimensionless) to normalize the time interval.
- The training dataset consists of two parts. The R-P part and the DPD part. The handshaking region has overlapping data from both methods.
- The input of network: pressure change, initial bubble radius, time
- The output of network: relative size change





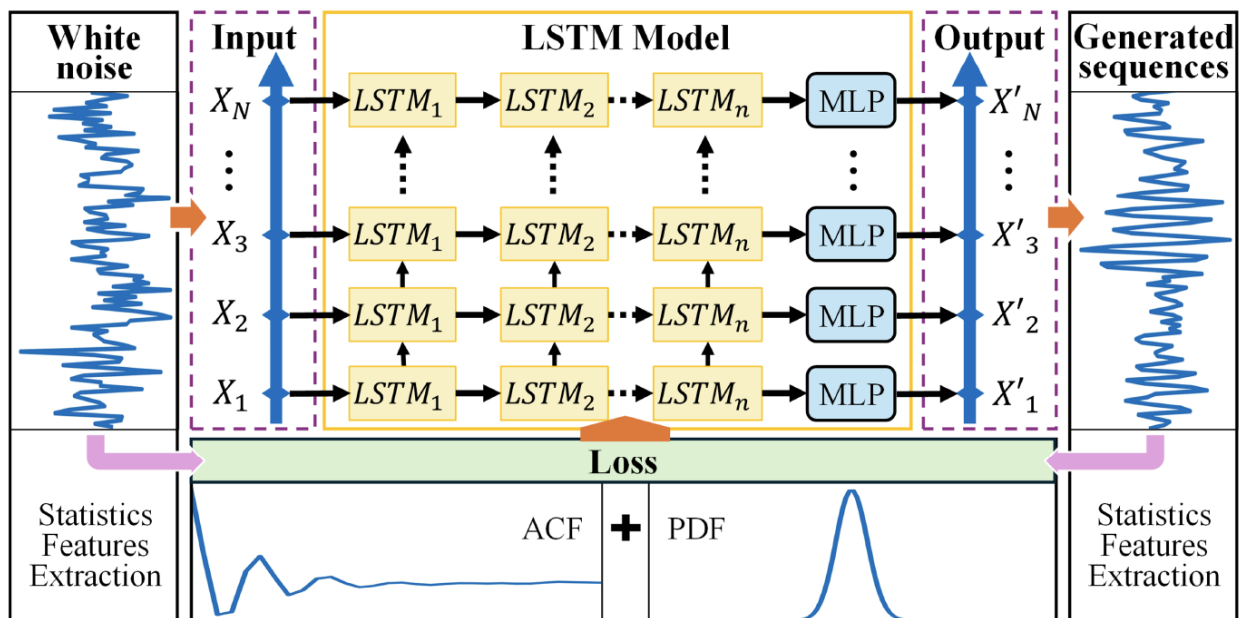
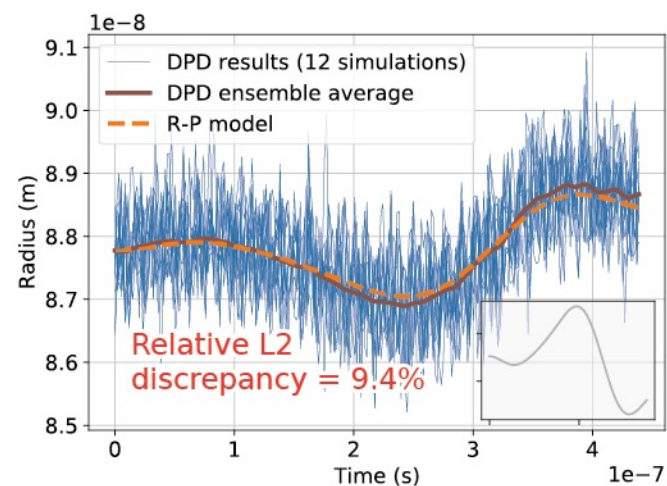
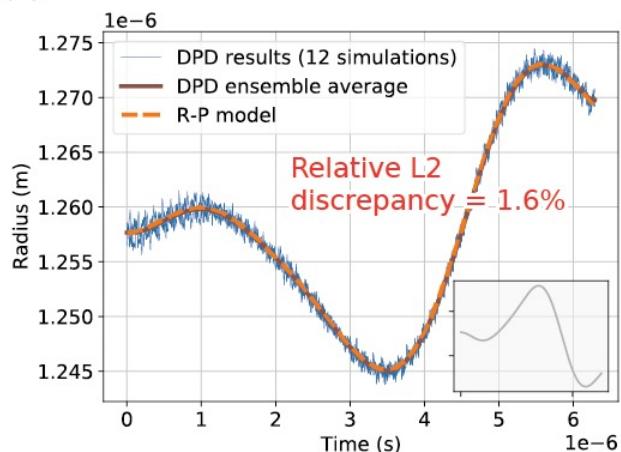
# Multiscale problem

- Once trained, DeepONet can predict bubble size within fraction of a second.
- We show examples of DeepONet prediction vs R-P results and DPD results at different scales.
- The test error (relative L2) shows the DeepONet has relative stable and low errors ( $<0.04$ ) across scales.

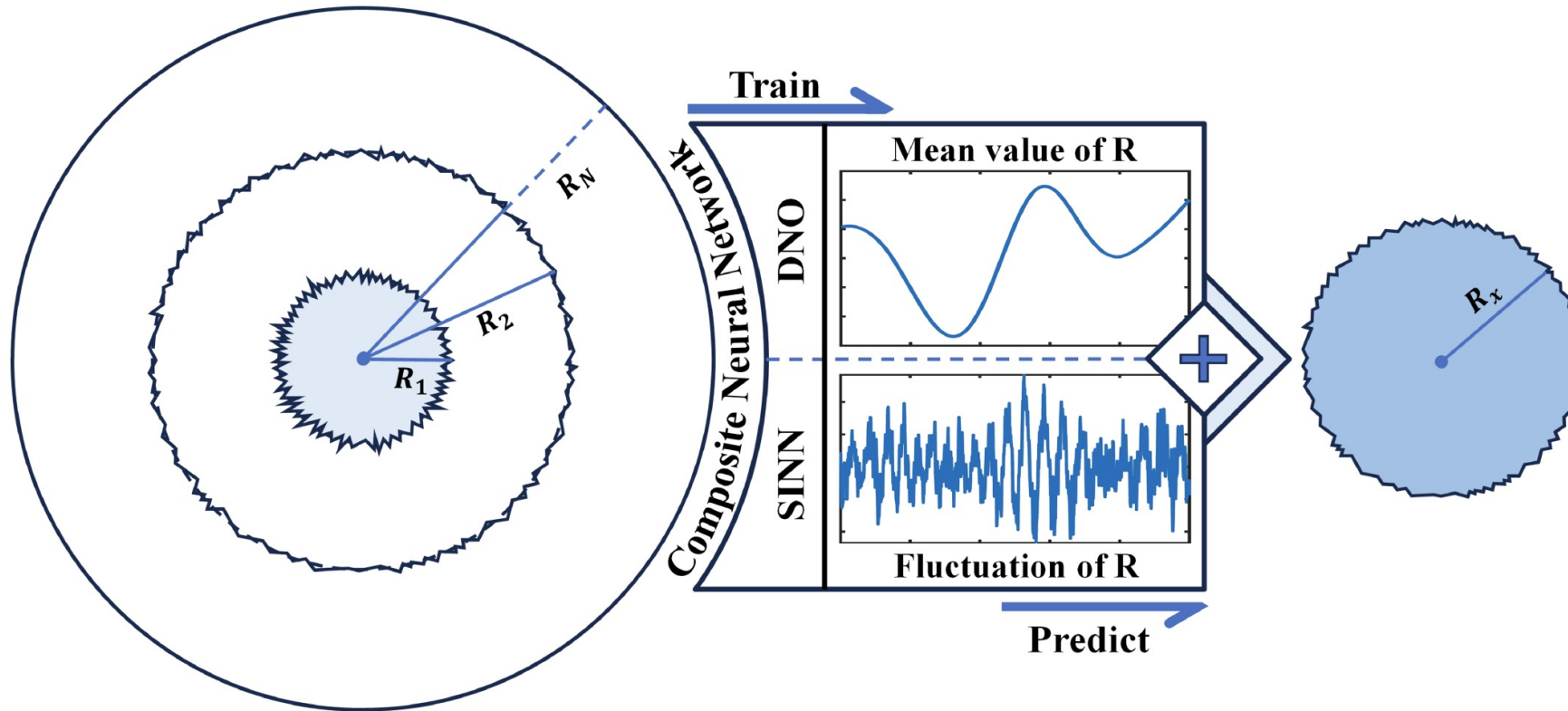


# Multiscale problem

- The thermal fluctuations (noise) is one of the characteristics of microscopic processes.
- Next, we involve the thermal fluctuations in the learning process.
- Instead of learning the raw noise trajectory, we learn the Statistical properties (Probability density function and Autocorrelation function) of the noise.



# Multiscale problem



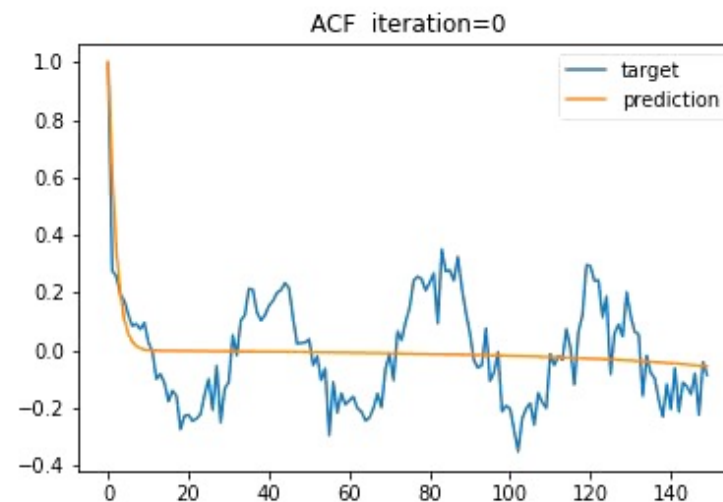
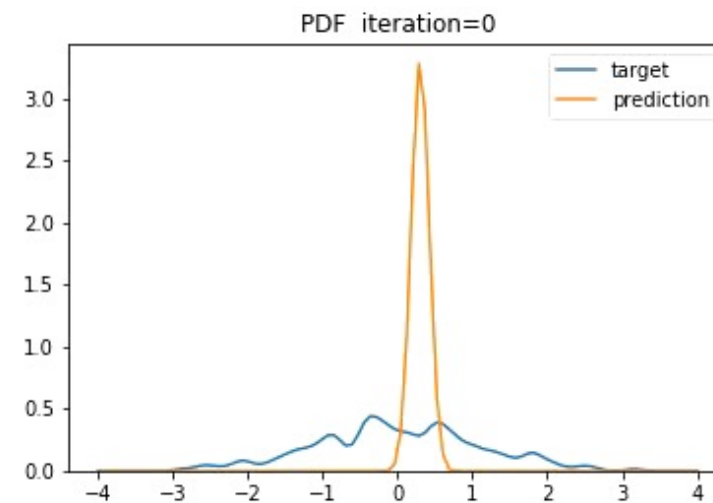
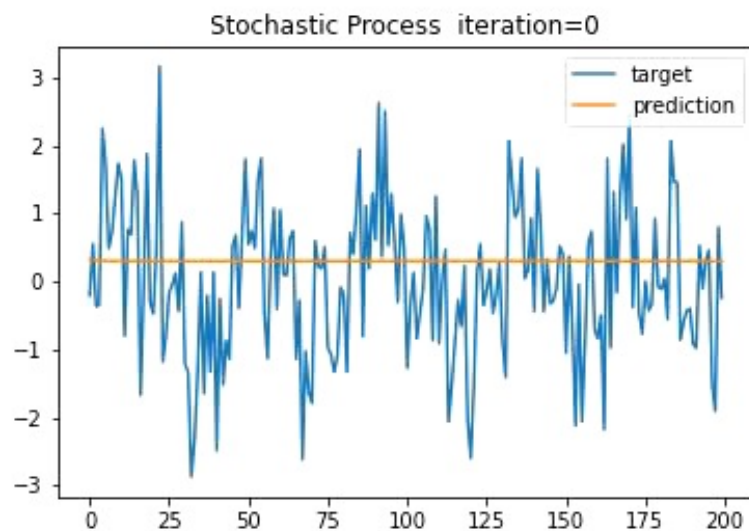
## Schematic of the Composite Neural Operator Network.

It contains a deep neural operators (DeepONet) network and a statistics-informed neural network (SINN). The entire network is trained on computational data generated from the continuum-based RP model and microscale mDPD simulations.



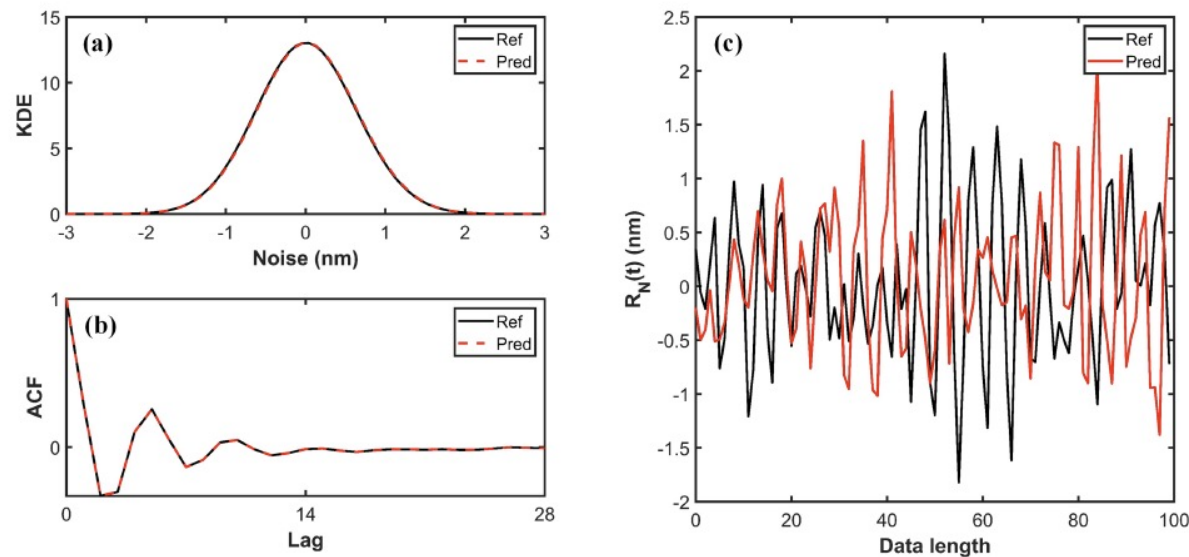
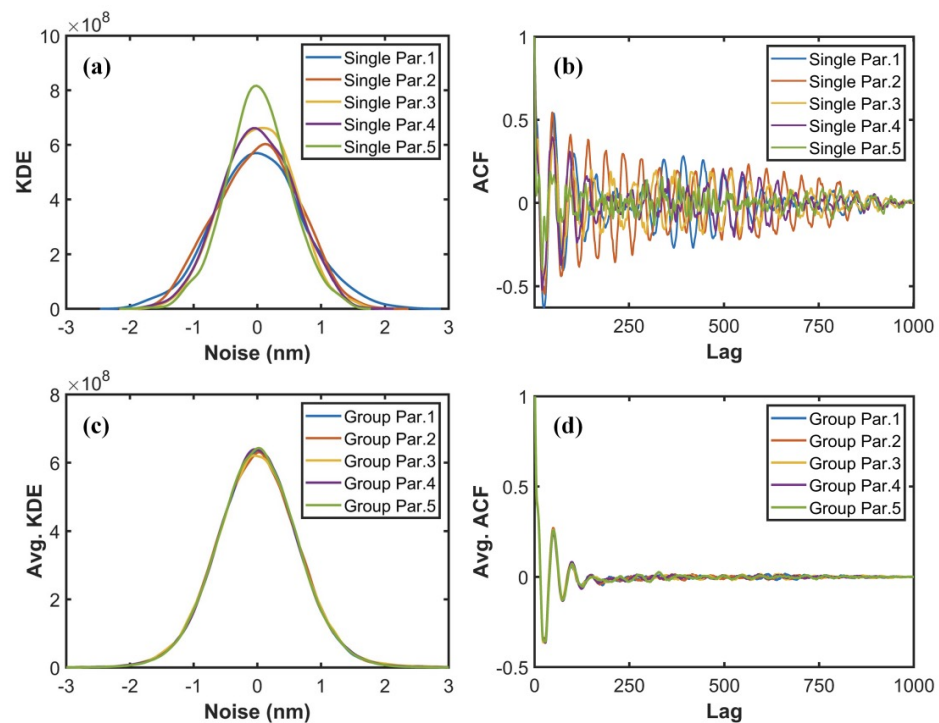
# Multiscale problem

- An illustrative example of SINN training process on target mesoscale fluctuations



# Multiscale problem

PDF and ACF from single run and group of parallel runs



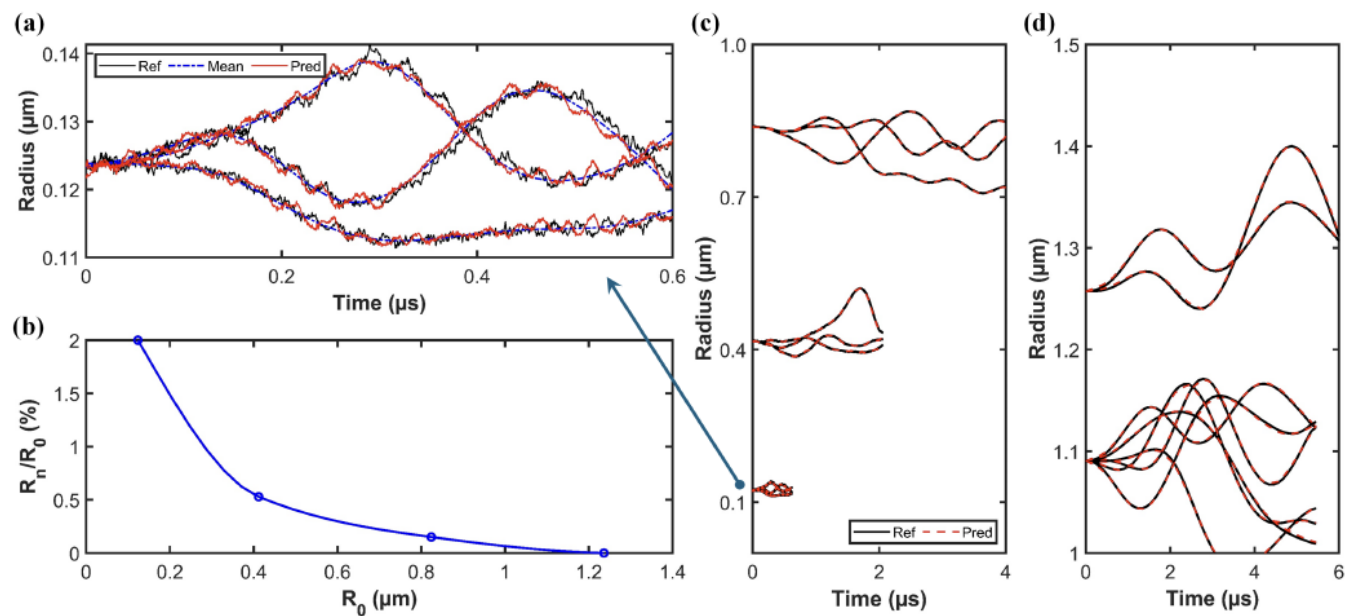
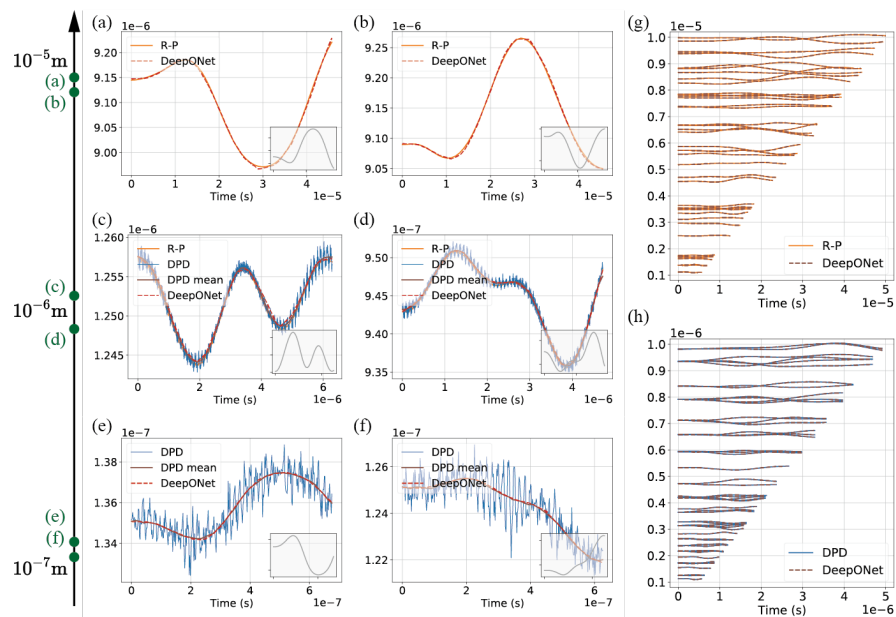
Example of one generated noise trajectory from the trained model compared with the reference noise trajectory from mDPD system

## DeepONet

# VS

## Composite model

(DeepONet + SINN)





THANK YOU



# Multiscale problem